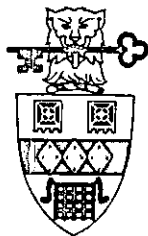
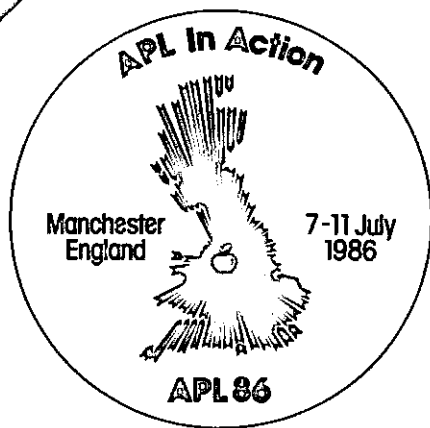


VECTOR



*The Journal of the
British APL Association*

A Specialist Group of the British Computer Society

Vol.3 No.2 October 1986

Contributions

All contributions to VECTOR should be sent to the Editor at the address given on the inside back cover. Letters and articles are welcomed on any topic of interest to the APL community. These do not need to be limited to APL themes nor must they be supportive of the language. Articles should be submitted in duplicate and accompanied by as much visual material as possible, including a photograph of the author. Unless otherwise specified each item will be considered for publication as a personal statement by its author, who accepts legal responsibility that its publication is not restricted by copyright. Authors are requested wherever possible to supply copy in machine-readable form ideally text files on a 5¼" IBM-PC compatible diskette. For other standards, please contact the Editor beforehand. Program listings should indicate the computer system on which they have been run. APL symbols should be displayed on a separate line and not embedded in narrative. Except where indicated, items published in VECTOR may be freely reprinted with appropriate acknowledgement.

Membership Rates 1986-87

Category	Fee p.a.		VECTOR copies	Passes
	£	\$		
Nonvoting student membership	5		1	1
UK Private membership	10		1	1
Overseas private membership	18	27	1	1
Supplement for airmail (not needed for Europe)	8	12		
Corporate membership (UK)	85		10	5
Corporate membership (Overseas)	140	210		
Sustaining membership	360		neg	5

The membership year runs from 1st May to 30th April. Applications for membership should be made on the form at the end of the journal. Passes are required for entry to some Association events and for voting at Annual General Meetings. Applications for student membership will be accepted on a recommendation from a course supervisor. Overseas membership rates cover VECTOR surface postage and must be paid in £UK.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive multiple copies of VECTOR and are offered group attendance of Association meetings. Partaking individuals need not be identified but a contact person should be nominated for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in the editorial section of the journal and opportunities to inform APL users of their products via seminars and articles.

Advertising

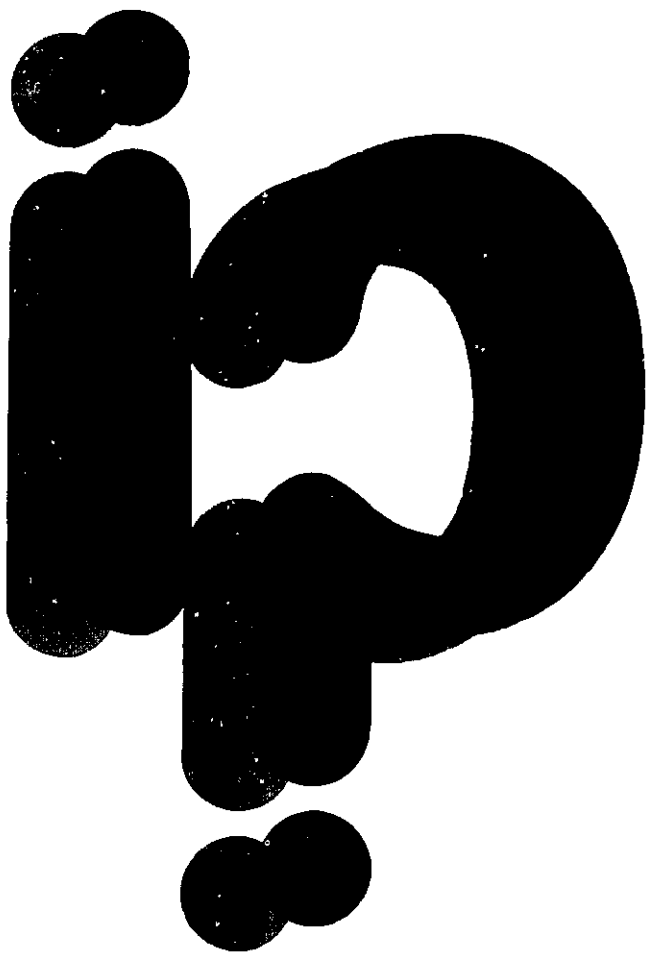
Advertisements in VECTOR should be submitted in typeset camera-ready A5 portrait format with a 20 mm blank border. Illustrations should be black-and-white photographs or line drawings. Rates are £250 per page. A6 and A7 sizes are offered subject to layout constraints.

Deadlines for advertisement bookings and receipt of camera-ready copy are given beneath the Quick-Reference Diary.

Advertisements should be booked with and sent to Steve Lyus, whose address is given beneath the Index of Advertisers.

CONTENTS

GUEST EDITORIAL: APL86 – What Next?	Dick Bowman	Page 3
APL NEWS		
Quick Reference APL Diary		5
General Correspondence		7
British APL Association News		15
Report and Accounts		15
News from Sustaining Members	Steve Lyus	17
British Computer Society news	David Preedy	21
International APL News	David Preedy	23
APL Product Guide	Steve Lyus	27
APL Book List		36
The Education VECTOR	Norman Thomson	37
RECENT MEETINGS (AGM)		
Introductory Notes	Adrian Smith	43
APL and Relational Database	Phil Chastney	44
The Split in APL	Ken Iverson	45
		47
APL-86 CONFERENCE REPORTS		
Photo Montage		51
Opening Plenary – CORE APL	Ken Iverson	52
Product Forums	David Preedy	59
A.I. at APL86	Neil Mitchison	61
The Lighter Moments of APL86	Philip Goacher	65
Pick of the Week (1)	Roy Sykes Jr.	67
APL86 Exhibition Report	Martin Malin	69
Pick of the Week (2)	Adrian Smith	71
APL86 Fun and Games	Aelred Tobin	77
Implementations of Enhanced APLs	Anthony Camacho	80
Quotes of the Week	David Ziemann	81
Thoughts on APL Design	Bob Pullman	86
Closing Plenary	Anthony Camacho	87
		89
TECHNICAL SECTION		
Stop Press: Amstrad PC1512	Adrian Smith	91
Technical Editorial	Barman & Ziemann	92
Technical Correspondence		93
Prize Competition (Result)	David Ziemann	94
APL-86 Competitions Review	David Ziemann	95
Prize Competition (Caption Contest)	David Ziemann	96
APL-86 Standards Report	David Ziemann	108
APL/PC 2.0 Auxiliary Processors	Manuel Alfonsaca	110
Development of APL Through Standardisation	David Livingstone	112
Using Quad-FX with Auxiliary Processors	David Piper	115
Public Domain Interpreter (Proposal)	Anthony Camacho	123
Comparison Tolerance	Dick Gray	127
		131
PUBLIC DOMAIN SOFTWARE LIBRARY		
(Keyword Index)		134
INDEX TO ADVERTISERS		138
		139



**THE SOLUTION TO
YOUR APL QUESTIONS**
on IBM PC or Mainframe.
INNER PRODUCT

For further information, contact JAMES MANNING, Marketing Manager, Inner Product, Eagle House, 73 Clapham Common Southside, London SW4 9DG, or telephone 01-673-3354.

Guest Editorial: APL86 – What Next?

by Dick Bowman

In a sense, it's all over – after more than two years of planning and meetings the Committee went to Manchester for a week and found that everyone else had decided to go as well. As to the degree of success, I'll leave others to comment, but I hear no reports of physical assault so I guess we got out with our skins intact.

What are we going to do with ourselves now?

Before answering that question it seems appropriate to acknowledge the efforts of all of the APL86 committee in the run-up and during the event itself. By the nature of these things some people do jobs which make them more visible than others, but let's all recognise that this was a cooperative effort in which every member of the committee has contributed vast amounts of time, effort and perspiration. The British APL Association has a great deal to thank Anthony Camacho, Mel Chapman, Cathy Dargue, Phil Goacher, Christine McCree and Dave Ziemann for. Also we would all recognise the additional contributions made by Peter Marchant and Roy Tallis, the ladies of the BCS and by the APL86 Stewards.

So, what do we do now?

In the short term, the work of the APL86 Committee is still not over – we have a number of administrative loose ends to tidy up, not least of which being to get the financial account settled and what we hope will be a worthwhile surplus available for fulfilling the long-term objectives of the British APL Association.

Because, at the end of the day we decided to organise APL86 not just to keep the Committee off the streets and out of trouble but because it was going to be good for the British APL Association. We now are clearly much more visible as a leading body in the APL world, one of the factors being VECTOR itself, which we launched on the back of the Loughborough conference in 1983. APL86 has helped reinforce our position and will in turn enable us to launch other projects over the coming months.

What are these projects?

Watch this space.

What can I do to help?

Lots – volunteer your services, the BAA Committee and Working Groups have never yet been known to turn helpers away; write technical articles for VECTOR; offer to talk about your work, ideas and projects at our regular meetings; get us more members, persuade your colleagues to join, or get your company to become a Corporate or Sustaining member. Remember, APL is what made computer programming fun – spread the word.

But to return to APL86, I think we all found it an enjoyable experience; the tradition of annual APL conferences is a worthwhile one – if you've been you know, if not then why not find out in 1987. See you in Dallas.

Dates for future issues of VECTOR

	Vol 3 No 3	Vol 3 No 4	Vol 4 No 1	Vol 4 No 2
Copy date	17 Oct 86	30 Jan 87	24 Apr 87	24 Jul 87
Ad. booking	14 Nov 86	20 Feb 87	22 May 87	21 Aug 87
Ad. copy	21 Nov 86	27 Feb 87	29 May 87	28 Aug 87
Distribution	January 87	April 87	July 87	October 87

BACK NUMBERS OF VECTOR

Back numbers of VECTOR are available from the BCS. If you don't have them all, now is the time to complete your collection. Apart from the technical contents, every issue includes book and product reviews, letters, news and a competition. Send in your order before they run out. These will one day be unobtainable collectors' items, like the early issues of Quote Quad.

The prices inclusive of postage and packing are as follows:

	UK	Prices in Pounds Sterling	
		Surface (inc. Europe)	Airmail (outside Europe)
Single issues	3	3.75	5.75
Volume 1	10	14.00	22.00
Volume 2	10	14.00	22.00

Please send sterling cheques or money orders (payable to The British APL Association) to the Treasurer:

Mel Chapman, 12 Garden Street, Stafford ST17 4BT.

Don't forget to include your name and address and to be clear which VECTORs you want.

Quick-reference diary

compiled by David Preedy

Date	Venue	Event
1986		
17 October	London	British APL Association meeting
27-30 October	Miami Beach	Conference on Computer Languages, organised by IEEE Computer Society
21 November	London	British APL Association meeting
1987		
6-7 January	London	APL in Engineering & Information Science, organised by South Bank Polytechnic
16 January	London	British APL Association meeting
20 March	London	British APL Association meeting
11-15 May	Dallas	APL 87 - APL in Transition
15 May	London	British APL Association AGM & meeting
18 September	London	British APL Association meeting
16 October	London	British APL Association meeting
20 November	London	British APL Association meeting
1988		
15 January	London	British APL Association meeting
18 March	London	British APL Association meeting
20 May	London	British APL Association AGM & meeting
16 September	London	British APL Association meeting
21 October	London	British APL Association meeting
18 November	London	British APL Association meeting

All British APL Association meetings are to be held at the Royal Over-Seas League, Park Place, near Green Park tube station, and start at 2pm.

ARE YOU

TIRED of TRYING

to get GRAPHS from some
COMMERCIAL PACKAGES?

- *Our PLOT/PC can match most of the graphical output of other highly priced packages at a MUCH LOWER PRICE!*
- *3D & 2D PLOTS TEXT ANNOTATION and AUTOMATIC SCALING*
- *Supports SCREENS, PRINTERS and PLOTTERS*
- *Includes INTERACTIVE DIAGRAM EDITOR*
- *COMPLETE with DOCUMENTATION*
- *LICENCE TERMS AVAILABLE*
- *UNLOCKED CODE*

£ **99**.00 (inc. VAT)

MTS **MetaTechnics Systems Ltd**
Unit 216, 62 Tritton Road,
London SE21 8DE, 01-670 7959

The Systems Builders

General Correspondence

The VECTOR working group welcomes correspondence on any topic affecting the APL community. All such letters should be addressed to the Editor and should indicate whether they are intended for the general or technical section. Letters containing APL code will normally appear in the Technical section of VECTOR, and authors are asked to observe the requirements on the inclusion of APL code stated on the inside cover. The Editor reserves the right to edit any letter unless the writer states that the letter is to be published in full or not at all.

APL popularity

From C E Williams

24th June 1986

Sir,

VECTOR frequently carries comments bewailing the lack of popularity of APL, and blaming the character set. Has anyone considered the equally plausible reason, the high cost of entry to APL for the home computer user?

MicroAPL alone appear to be taking effective action over this problem, but the QL/APL (at very reasonable cost) has too many hardware drawbacks – the atrocious keyboard, absence of roll-over, non-standard mass storage, and now the sell-out to Amstrad. The next entry point, the Atari, jumps to some £700 and hardware reliability appears to be suspect.

What is required, I believe, is inexpensive software on disc (perhaps bundled with BASIC on Amstrad and BBC machines in place of LOGO ?) for the 8-bit machines which far outnumber the 16-bit machines at the present time, giving the opportunity of much easier access to APL, and hopefully increasing its popularity.

There was such a subset of APL for the TRS80 Model I, Level II, called APL80, written by Phelps Gates and published in the USA by RAM software. It was advertised in the UK for £30 about 1980/81. If anyone knows whether this is still available, or where a copy may be obtained, it will be very much appreciated.

Yours faithfully,

C E Williams,
Cold Keld,
Loweswater,
Cockermouth,
Cumbria.

(Editor: As a parent, I have long felt that the combination of APL and LOGO would be a very good base to encourage children to appreciate the concepts behind computers, and would second your call for an inexpensive interpreter. I am sure we all applaud the plans for a public domain interpreter announced at APL86 – see elsewhere in the issue of VECTOR for details – and wish the brave volunteers every success in their venture.)

Porta-APL benchmark

From Mark Bassett

25th June 1986

Sir,

In reviewing the APL68000 for the Macintosh, I think I may have found an error in the Porta-APL review (VECTOR, Vol 3, No 1). One of the benchmarks in the first set calculated should involve finding the tenth root of an integer vector. In fact your write calculated the square root, which is considerably easier by a factor of about three!

Yours sincerely,

M S Bassett,
Metapraxis Ltd.,
Hanover House,
Coombe Road,
Kingston.

APL Blossom Time, and how to send money to Zeke Hoskin

From Zeke Hoskin

26th May 1986

Sir,

The original "APL Blossom Time" was written by Mike Montalbano, under the pseudonym of J C L Guest. He brought it to the APL81 conference in San Francisco, and the APL Blossom Choir formed quite spontaneously to sing it at the banquet. (I was not in the damned choir, having come to the damned conference without my damned mandolin!) The lyrics were subsequently published in APL Quote Quad, probably early in 1982. The tune is lifted from a country song named "The Battle of New Orleans", lyrics by Jimmy Driftwood, popular in around 1961. Driftwood in turn stole it from a fiddle tune named the Nth of January, N being somewhere between 11 and 14. The fiddle tune is probably originally Irish, as is fitting for a song about shooting the bloody British. (In the early-60s, reaction to the song forced somebody to write a pro-British war song called "Sink the Bismarck", but this is a bit far afield.) I can send you the tune if you really want it.

At APL85 in Seattle, I asked the organisers who was running the APL Blossom Choir and they said, "You are." Overwhelmed by the honour, I sat down at somebody's Macintosh to make a set of overhead slides and added a verse and chorus of my own (to go before the original last verse and chorus).

Bob Smith and Bob Bernecky agreed to disagree.
They knew they had to build a nest but couldn't choose a tree. They took a thousand
strands of code and wove them round and round,
But one was at a NARS-ty height and one was on the ground.

They both got arrays and hung them on the networks,
And neither had a notion what the other one would do,
And one was composed and one was pervasive,
And meanwhile Jim Brown was writing APL2.

That is the only verse and chorus sung at APL85 that was not part of the Montalbano epic. I got Jim Brown's and Bob Bernecky's permission to publish it in Quote Quad, but was unable to get hold of Bob Smith or Mike Montalbano.

There are two riders: The APL85 version was a thousand "lines" of code, not "strands". Also Bob Bernecky points out that "neither had a notion what the other one would do" is (of course) inaccurate. The dictates of meter preclude the more correct version:

"The APL Standards committee wished that they (Bob and Bob) wouldn't do anything until they (the committee) decided what they (Bob and Bob) should do."

Thank you for the 10 pounds. It was more than twenty Canadian dollars before the bank charged a dollar and a half for converting it. The same thing happens when I try to write a pounds money order to renew my membership in the BAA. Therefore should I happen to win and/or earn anything from VECTOR, please apply the pounds directly to my membership. I'd suggest including that as an option in future contests.

Have a nice APL86 . . . iota rho to Manchester but the C is too wide.

Yours more or less sincerely,

Zeke Hoskin,
Control Z Software,
3065 E 24th Ave.,
Vancouver, B.C. V5R 1E7,
Canada.

(Editor: Your letter probably passed VECTOR Vol 2, No 4, mid-Atlantic, so I expect that know by now that we can accept subscriptions in US dollars, and this may reduce your currency commission charges. As regards prize-money being off-set against next year's subscription, I'm sure our treasurer will be delighted to receive advance payment whenever you win, but please make sure you repeat your request for this treatment on each competition entry, because the logistics of tracking a different procedure for each individual can become tortuous!)

A plea for help

From Joe McGrellis

18th March 1986

Sir,

We are about to start a major development project based wholly on an IBM PC/AT. It will be developed using STSC's APL*PLUS Application Development and will use as its kernel their STATGRAPHICS package. This letter is a plea for help.

What we need is a standalone device driver for the Hewlett-Packard HP7475A plotter which will allow us more freedom to expand our scope beyond STATGRAPHICS. We have a tentative promise of a new TOOLS workspace that may be available later this year, but our project can't wait that long.

Being stranded out here in the APL desert, our only contact with the outside world is through VECTOR, but due to the vagaries of the Saudi postal service it can take up to 3 months before an issue of VECTOR is received. Perhaps you could throw my dilemma open to the wider APL community.

It is an excellent magazine, long may it continue, but perhaps a bit earlier to us out here in Saudi Arabia.

Yours sincerely,

Joe McGrellis,
c/o Arabian-American Oil Company,
P.O. Box 9712,
Dhahran 31311,
Saudi Arabia.
Tel Saudi Arabia (3) 875-6940 (office)
873-8298 (home)

(Editor: Thanks for your comments, and for your tact in blaming the irregular supply of VECTOR entirely on the Saudi postal service! I hope someone answers your plea.)

Polygon shading

From Clement Taylor

12th May 1986

Sir,

For several months I have been searching for a suitable algorithm for shading polygons. I would be most grateful if any of your readers could provide an algorithm expressed as one or more APL functions to shade a simple polygon. I have a Sinclair QL computer with the MicroAPL APL.68000, and I have been playing around with a Tektronix 4661 flat-bed plotter. Using APL it is very easy to draw all sorts of mathematically definable shapes. I have never been able to figure out a workable method of shading the polygons which are being plotted.

On another note, I attended the recent Commodore show in London and I was very pleased to see that MicroAPL are planning an APL implementation for the new Amiga computer. I will be returning to the USA this summer and I am planning to purchase one for myself as soon as I am settled. I understand that in the States you can pick up an Amiga with a quality colour monitor, the plug-in 256k memory expansion and an external disk drive for \$1300. An impartial source told me that at that price the Amiga was outselling the Atari by three to one. I also saw a two megabyte memory expansion for around \$1000 (£599). The APL.68000 package will be pegged at under \$300. To me this could mean real APL on a

real personal computer for under \$2600, a price about which APL devotees might do some serious thinking.

Yours faithfully,

Clement Taylor,
Esher,
Surrey.

(Editor: I know that Clement will have returned to the States before this letter is printed. Any readers who can provide a suitable algorithm should send it to myself. To add spice to the challenge, the last such algorithm I saw shaded any polygon, including re-entrant polygons, and drew alternate lines in the opposite directions in order to optimise plotting time. It was written by a sandwich student who knew no APL when he started his year's industrial placement!)

Our technical team have tracked down the following reference on this and closely related topics in the APL literature:

Polygon clipping, Quote Quad 15,3 March 1985

(correction in Quote Quad 16,2 December 1985)

Parallel shading of polygons, Quote Quad 13,4 & 16,1 September 1985)

Hidden Plane Elimination and Surface Shading of Polygons Representing Buildings, Cornell & Julian, APL 81 Proceedings.)

A reader's response

From Mr J Sullivan

16th July 1986

Sir,

I couldn't disagree more strongly with Mr Devaney's letter in VECTOR Vol 3, No 1. I will answer his points in order:

- (a) Management control: where do you draw the line between the costs of program development and the costs involved in insisting on proper management control? The facts that APL is hard to control and that most managers cannot understand it, are no reason for abandoning it when you calculate the costs of the alternatives. As an example I have just rewritten a badly-working and prone-to-failure PL/I program in APL. The PL/I program still didn't work properly after six months' solid work and was about 1000 lines long including all subroutines. The APL version works properly after half a day's solid work (including designing it from scratch) and uses only 80-100 lines of code. This is not an isolated example.

According to Mr Devaney I should not be using APL because my management can't control it; according to my management I should be using APL because it is very much cheaper and I can clear my applications backlog much more quickly. As for APL being 'unable to support properly established libraries ...', has he not heard of IP Sharp's LOGOS? And if he can't use LOGOS, can't he design something similar but simpler, as we have in our environment?

- (b) Unintelligibility. To whom? Has Mr Devaney ever tried to decipher 'typical' FORTRAN code? I know that FORTRAN-77 is better for writing clearer structured code than FORTRAN-66, but not much, and that doesn't help me try to understand some of the programs we have bought in from 'reputable suppliers' that are indecipherable. Rather than carry out maintenance on these programs, I convert them to APL, which makes further maintenance much easier. The point is that you can write unintelligible code in any programming language, so why single out APL?
- (c) Lack of portability. 'The only portability for APL that I know of is typing it all in again!' (Mr Devaney's exclamation mark.) I suggest that Mr Devaney does not know much about APL. An APL workspace is interpreter- and/or machine-dependent in much the same way that an executable program (load module) is machine-dependent. You would never dream of transferring a load module from one machine to another; you just transfer the source program and compile and link-edit it again on the new machine. So it is with APL; you put your functions in some kind of transfer file – via IBM's)OUT command or something similar; (if your APL hasn't got this command, write a function to simulate it – it's easy. Then you transfer this file by magnetic tape and do the reverse process. If $\square$ AV is incompatible, there is no problem; write a translate function.
- (d) Closed, secure systems. Mr Devaney's last paragraph is incomplete; the user can meddle with ANY program if he/she knows how to. What sort of security against intrusion is he after? Most mainframe manufacturers provide security which will protect anything – data, APL workspaces, etc. – against most prying eyes and fingers, but the most determined intruder will get you in the end. If you want to stop your users meddling with your APL functions, you should:
- lock all the functions in the production workspaces (keeping the development versions unlocked);
 - error-trap your workspaces so that your users can't get into 'raw' APL.

If you don't know how to do these you're not much of an APL programmer and I suggest that you contact one of our Sustaining Members for training!

Yours sincerely,

J Sullivan,
Senior Research Officer,
Statistics & Market Research,
Business Development Division,
National Westminster Bank plc.,
41 Lothbury,
London EC2P 2BP.

(Editor: John's response is obviously deeply felt; indeed he was so quick off the mark that I think he is the first correspondent whose letter has reached us in time for inclusion in the next copy of VECTOR! I expect replies from other readers to Mr Devaney's original letter in time for the January issue.)

NEW MAINFRAME SOFTWARE FOR IBM'S® APL

Now available in the UK, two new offerings from STSC that enhance IBM's mainframe APL implementations

If you're staying with VS APL ...

COMPILER

The first commercial compiler for APL compiles functions individually. Results in significantly faster execution. Interpreted functions can call compiled functions and vice versa.

If you're migrating to APL2 ...

SHAREFILE/AP

STSC's popular APL component file system is now available under APL2. Multi-user, nested array storage, libraries, access matrices. Multiple file system support. International language translations.

For full information, contact the APL*PLUS™ Product Group, Cocking & Drury on 01-493 6172.

Trademarks/Owners: IBM/International Business Machines Corporation · APL*PLUS/STSC, Inc.

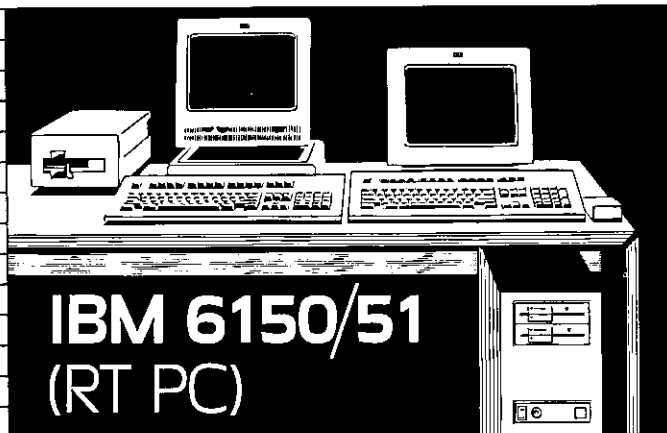


COCKING & DRURY LTD.
THE APL PROFESSIONALS

16 BERKELEY STREET · LONDON · W1X 5AE
Tel: (01)493 6172 · Tlx: 23152 MONREG

dyalog APL

IS AVAILABLE **NOW** FOR THE HIGH PERFORMANCE



IBM 6150/51
(RT PC)

— AND FOR 22 OTHER **UNIX*** SYSTEMS! —

The IBM 6150/51 is a new family of high performance 32-bit multi-user micro computers and workstations. It is based on reduced instruction set (RISC) technology and AIX**, IBM's enhanced version of UNIX System V; and is an excellent basis for a Dyalog APL system.

Dyalog APL on the IBM 6150/51 is fast, especially with the floating-point accelerator, and there is practically no limit on the size of APL objects and workspaces.

The entire range of standard IBM screens and printers for the 6150/51 is fully supported, as are most of the common APL/ASCII vdu's.

All AIX facilities can be accessed from within a Dyalog APL function and results may be captured in an APL array. Compiled subroutines written in C or Fortran can easily be incorporated into applications, and interfaces between Dyalog APL and several IBM supported software packages such as SQL and the Professional Graphics Series are available.

Dyalog APL adheres to the draft ISO standard for APL and includes many of the features of APL2. Nested arrays, and corresponding extensions to primitive functions and operators, greatly simplify the processing of lists and non-rectangular data structures. Dyalog APL contains a host of useful features including a session manager, full-screen editor, and full-screen data manager.

The IBM 6150/51 with Dyalog APL is available direct from Dyadic. Prices, including Dyalog APL, start from around £15,000.

For further information, please contact **Peter Donnelly**
Dyadic Systems Ltd., Park House, The High Street, Alton,
Hampshire GU34 1EN, United Kingdom
Telephone: Alton (0420) 87024 (10 lines). Telex 858811

*UNIX is a trademark of AT & T Bell Laboratories. IBM is a registered trademark of International Business Machines Corporation.

**AIX is a trademark of International Business Machines Corporation.

British APL Association – 1986 Report and Accounts

Income and Expenditure for year to:		30.4.86		30.4.85	
		£	£	£	£
Income:					
Subscriptions:	Full (including students)	5790		3383	
	Corporate	1720		752	
	Sustaining	2600		2000	
	Library	<u>315</u>	10425	<u>60</u>	6195
Special Events:			-		1642
Vector Sales			80		-
Bank Interest			<u>145</u>		-
Total Income			<u>10650</u>		<u>7837</u>
Expenditure:					
Meetings:			1069		1193
Vector (less advertising income):			1897		11830
Committee:	BAA	773		713	
	APL86	<u>-</u>	773	<u>305</u>	1063
Mailing:			1572		1583
Education:			638		-
BCS HQ Services (in excess of BCS budget):			1519		-
Professional Fees:			35		60
Written off:			<u>220</u>		-
Total Expenditure			<u>7723</u>		<u>15729</u>
Surplus of Income over Expenditure			<u>2927</u>		<u>(7892)</u>
Balance Sheet at Year End					
Cash at Bank			4835		8828
Debtors			<u>6488</u>		<u>3532</u>
Less creditors			11323		12360
			<u>(5758)</u>		<u>(9722)</u>
Net Assets			<u>5565</u>		<u>2638</u>
Accumulated surplus previous year			2638		10530
Surplus current year			<u>2927</u>		<u>(7892)</u>
Net reserves			<u>5565</u>		<u>2638</u>

The accounts, to BCS requirements, were prepared and signed by Mel Chapman as treasurer and audited by Roger Francis M Sc. B Sc.(Econ) ACIS.

May 1986

APL.68000 FOR THE ATARI ST

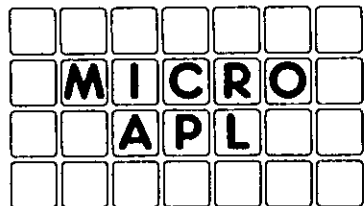
The introduction of the Atari ST marks the beginning of a new era in microcomputing: high performance with no compromise in a compact but attractive 68000 implementation. The Atari version of APL.68000 is no-compromise APL: fast and extremely powerful. Atari ST and APL.68000: a no-compromise and low budget combination.

FEATURES:

- Full interface to Atari ST graphics facilities
- Arbitrary I/O via serial ports
- Session manager allows editing of screen lines
- Workspaces can be set as run-time applications
- Applications can use standard ASCII keyboard
- Applications can detect mouse position and state
- Uses standard Atari ST user interface
- Built-in full screen function editor
- Access to Atari ST DOS native files
- Common system commands as pull-down menus
- Full printer support
- Runs in its own window
- User-defined pull-down menus
- User-defined Dialog and Alert boxes

Price: £170.00 excl. VAT

APL.68000 is a trademark of MicroAPL Ltd.
Macintosh is a trademark licensed to
Apple Computer, Inc.



MicroAPL Limited

Sales Office: 19 Catherine Place, London SW1E 6DX
Telephone: 01-834 9022

News from sustaining members

Compiled by Steve Lyus

Dyadic Systems Limited

Dyadic Systems has now released Dyalog APL Version 4.0. The new release contains a number of significant enhancements and performance improvements. Foremost among these is a facility for user-defined operators, which is 100% compatible with IBM's APL2. Version 4.0 also supports function assignment and display. Not only is this feature invaluable for debugging operators, it has many interesting uses and will be a great aid in teaching APL.

A new system function, □ MONITOR, measures cpu and elapsed time spent in particular functions and lines of code, and will help Dyalog APL users to speed up their applications, by identifying those parts which consume the most resources. A number of enhancements to the session manager, forms manager, and full-screen editor are provided; and there is a new translate-table facility for extended terminal and printer support. Performance has been improved by an overall factor of 2 compared with Version 3.0, with even greater gains in speed for particular operations.

Mercia Software Limited

Mercia Software are continuing to expand their range of APL-based software and services. As well as supplying and supporting STSC's range of software based on APL*PLUS/PC, they now have four new products which may be of interest to VECTOR readers. Their multi-user/multi-tasking APL*PLUS/PC uses Multilink Advanced software and Mercia's own APL utilities, enabling you to run up to nine APL terminals/tasks on an AT under standard PC DOS and standard APL*PLUS/PC. EXEC*U*STAT is a business analysis and graphics package written in APL*PLUS by the authors of STATGRAPHICS. There are also demonstration disks available for Unitab – an APL spreadsheet and data entry system – and for The APL Debugger, for even greater APL programmer productivity.

On the training front, Mercia's APL courses are still proving popular, and they anticipate a good response to their new course on "System Design with APL*PLUS/PC".

I. P. Sharp Associates

I.P. Sharp and STSC have agreed on a joint development project to create a complete version of SAPL in 'C' to be run under UNIX. Work is already in progress and a test version of the new product is expected early in 1987. In addition I.P. Sharp have negotiated special distribution rights for STSC's APL*PLUS PC product. This may involve a selection of IPSA's mainframe applications becoming available for PC APL.

LOGOS, a complete APL application development environment, is now available. LOGOS provides all the controls and facilities that are necessary for developing and maintaining large production systems. This is an area in which APL has traditionally been weak and one for which it has been severely criticised by DP management. LOGOS answers these criticisms and should enable the range of applications for which APL is appropriate to be greatly expanded. Despite the appeal of LOGOS to management, it also offers many benefits to the programmers themselves. The experience within IPSA is that programmers appreciate the tools available and that their ability to produce reliable systems on time is improved.

Details of Release 19 of IPSA's mainframe APL product are now available and include an interface to DB2 (AP127), a new system management information facility with direct links to SMF, and new security software (SACF) which provides a general interface to site specified access control facilities such as RACF, ACF2 or TOPSECRET. In addition a number of significant improvements to the efficiency of the interpreter, scheduler and other system software have been made. Also the option of running workspace slots in pageable memory has been provided. This allows sites to minimise page fixed memory requirements while still offering the high performance required by their most critical applications.

APL People Ltd

Our latest venture – a software company called predictably enough 'APL Software Limited' – generated so much interest at APL'86 that we have had our hands full evaluating all the weird and wonderful software packages which we were offered! For those of you who don't yet know, the objective of APL Software is to develop and market THE VERY BEST in APL system software not only throughout the UK but world-wide! In order to achieve this, we are joining forces with APL Software Technology Ltd (AST) of Sweden, and have persuaded Phil Goacher to join us and become Managing Director of the new Company.

Present offerings include:

APL2 software for screen handling, APL2 report generators, Powertools, RDS on mainframe VSAPL and on the PC, and STSC APL on the PC enhancements to pack data and to fix functions faster.

We hope there will be lots of exciting new software available very shortly to add to this list!

In addition, we are to become the UK agents for the American Software House, Interprocess, and it looks like our next move might be to set up an office 'States-side'!

Meanwhile, down in rural Bath, things have been pretty hectic as we try to follow up all the enquiries we received at APL'86 – writer's cramp and telephonist's elbow are beginning to set in!

Cocking & Drury Ltd.

At a successful APL86 conference in Manchester recently, Cocking & Drury launched several new significant products related to APL on micro computers and mainframes.

Release 6 of STSC's APL*PLUS PC now supports objects greater than 64k bytes, as well as introducing faster soft character set and support for the HP Laserjet printer. An upgrade from release 5 will be available early in September.

STATGRAPHICS Release 2, the already highly successful Statistics and Graphics package for IBM PC's and compatibles, was also shown for the first time in the U.K. New features include enhanced data management and graphics, new statistical procedures and significant improvements in performance and user interaction.

On the mainframe side, new products were announced for both VSAPL and APL2. For VSAPL, an SQL interface to STSC's APL*PLUS enhancements is now available offering similar capabilities to that under APL2. For APL2, delta-FMT introduces the recognised industry standard method of formatting data for reporting.

The COMPILER for VSAPL announced recently in the U.K. by Cocking & Drury is generating interest at large APL users, several of which are evaluating it.

Bookings are now being taken for Cocking & Drury's new four day Intermediate APL*PLUS PC course. The course covers a wide range of PC application development skills, from pure APL through to full-screen and design techniques.

In addition, delegates of the APL*PLUS PC course and all other courses offered by Cocking & Drury, will in future be given a free copy of POCKET APL, STSC's Introductory APL for the IBM PC.

MicroAPL Ltd

The summer months have seen MicroAPL developing its international distribution network for APL.68000 on the Macintosh, Atari ST and Amiga. In the US, the Spencer Organization has been preparing a substantial marketing campaign for the products, and MicroAPL's Sales Director, Grant Scriven, has been busy setting up distributors in France, Eire, Sweden, and Australia. Meanwhile, the Unix versions of APL.68000 have also been selling well, especially on the Sun and NCR Tower machines.

Meanwhile, in the UK, MicroAPL has been concentrating on its core business of providing customized hardware/software solutions to major clients, in some cases taking APL applications outside the traditional areas into process control and real-time financial dealing systems. In addition, MicroAPL's cooperative venture with International City Holdings to market their Treasury Support System (written by MicroAPL consultants, in APL) has been attracting a great deal of interest amongst corporate treasurers of large companies.

The pressure for space at MicroAPL's headquarters in Nine Elms became so intense that the company has now moved its sales and consultancy operations into pleasant new offices in Victoria. Those familiar with MicroAPL's early history will already know the location, which is in the same building at 19, Catherine Place, SW1, where the company started. The telephone number at Catherine Place is 01-834 9022.

APL86 saw the first public showing of MicroAPL's new implementation of APL.68000 for the Atari ST. The announcement attracted a lot of interest, since it represents a major step in making APL available on very inexpensive, popular machines. Not only is the product a full implementation of APL at a very low cost, but it also follows MicroAPL's policy of making APL easier to use, by taking full advantage of the Atari Windows/Icons/Mouse/Pull-down menus ("WIMP") interface. The implementation is similar to the very successful Macintosh version of APL.68000, which was also attracting much interest at APL86.

Also shown for the first time at APL86 was Version 6.0 of APL.68000, running on MicroAPL's recently-launched supermicro, the Aurora. We boasted that the combination was the fastest APL running at the exhibition, and a flurry of benchmarking followed. One APL user simply did not believe that APL.68000 was as fast as his benchmarks showed, but eventually he had to concede that we had not cheated by changing the clock speed on the machine! The impressive performance of the Aurora system has been achieved by using multiple 68020 processors, each with an optional 68881 floating-point processor, and also by optimizing the code of APL.68000 since we acquired it last year.

The MicroAPL stand also featured a special version of APL.68000, called MultiAPL and developed by the Spencer Organization in the US, which runs on a 68000 plug-in card in an IBM PC/XT/AT system. This gives very impressive performance and large workspaces, and is an ideal upgrade for users whose applications are too large to run well under APL* Plus/PC. In addition, the full multi-user file system of APL.68000 means that MultiAPL runs extremely well in networked applications using the Novell networking system. The IBM-AT which we are using to demonstrate MultiAPL was linked using MicroAPL's IAS software to the Aurora system, allowing side-by-side comparisons of speeds on the AT and on the Aurora, as well as demonstraing the ease with which IBM PC-family applications can be integrated with MicroAPL's supermicro networks.

British Computer Society News

compiled by David Preedy

BCS Bulletin

The BAA Chairman has received the following letter from David Knight, the review editor of BCS Bulletin.

"I am writing to ask for your help in reviewing books for the BCS Bulletin. Over many years we have built up a large index file of potential reviewers, which is unfortunately thoroughly out of date.

"Many Specialist Group Chairmen have been kind enough to help by offering to take books and distribute them to interested Group members. It would be very useful if this concept could be extended to include more Groups. Otherwise, could you please canvass your membership to see who would be willing to have their names placed on the reviewers list?"

Any BAA members who would like to be involved in this activity should contact David Knight at the following address:

Computer Centre,
University College London,
Gower Street,
London,
WC1E 6BT.



H.M.W. PROGRAMMING CONSULTANTS LTD

Why not discover more about

- ★ **Consultancy/Support Service**
- ★ **APL on the IBM PC**
- ★ **VM/CMS Packages**

Ring Ken Jackson at

H.M.W. PROGRAMMING CONSULTANTS LTD

142 FELTHAM HILL ROAD,
ASHFORD, MIDDLESEX TW15 1HN
Telephone: Ashford 41232

International APL news

compiled by David Preedy

APL 87 – APL in transition
11-15th May 1987
Dallas, Texas

At the time of the APL '81 conference (the high-water mark of conference attendance), the primary operating environment of an APL system was a large, time-shared, mainframe computer. By the time of the APL '85 conference, the personal computer had become a major operating environment for APL. The APL time-sharing industry has shrunk substantially in terms of revenue and employment. More importantly, other languages and operating environments have begun to offer facilities that until recently were the sole province of APL. Features such as array handling and interactive de-bugging facilities are being found in both interpretive and compiled languages.

While this transition has opened up exciting new possibilities for spreading the use of APL, it has also spawned some very real challenges. APL is now competing for the beginner's attention with BASIC (a give-away on most PCs) and with functional C and PASCAL compilers that cost in the neighbourhood of \$50. APL is also competing with many spreadsheet and database packages. This conference will attempt to address these issues.

The conference will have three main themes:

APL as an Application Development system – Addressing such issues as user interface, transportability, ease of development, and maintenance and efficiency in operation.

APL and its Environment – Papers on the interaction of APL and its host operating systems, other languages and packages. Also papers on using APL in distributed processing or multiple CPU (or mainframe-PC) applications, and using APL with non-APL file systems.

Future Developments in APL – Including ISO standards, enhancements to the language, Nested Arrays, maintaining APL's technical superiority over alternate languages. Also including papers relating to strategies for increasing the use of APL, particularly among beginning users.

While these topics are only guidelines, the Program Committee encourages and will consider papers dealing with any subject relating to APL. The success of this conference depends on the participation of the entire APL community.

Informal abstracts are due to the Program Chairman by September 15, 1986. Completed papers are due by November 1, 1986, and are to be in English. Final copies of accepted papers are due by April 20, 1987, and must be suitable for photographic reproduction.

Program Chairman: Frank Cavallito,
1812 Sherrye,
Plano, Texas 75074,
U.S.A.

Rochester SIGAPL group

Bob Pullman has contacted us from Rochester regarding his inclusion in the VECTOR list of overseas APL groups. He writes:

"In VECTOR I was listed as President, Rochester ACM-SIGAPL. Although the local chapter is dormant and I am no longer a member of ACM or SIGAPL, I still get phone calls now and again from APLers who call around. I would like to maintain the listing of the Rochester group in VECTOR, but under the different address:

The Rochester Group Inc.,
325 Park Avenue,
Rochester,
New York, 14607"

Swiss APL User Group (SAUG)

Despite our pleas for an massive contribution from overseas groups for the APL86 edition of VECTOR, news seems to trickle in slowly. The following contribution from Switzerland just missed our deadline, but we are pleased to publish it in this issue.

"The Swiss APL User Group welcomes this opportunity to address itself to the international community, as it does not share the somewhat xenophobic view of the world prevalent among many mid-of-the-mill Swiss citizens (see latest popular decision regarding participation in the United Nations Organization). The Swiss APL User Group was created in December 1981 under the impetus of APL aficionados at the Central Post Office in Bern, who believed then – we still share this belief today – that APL deserved a wider exposure in Switzerland and was the ideal language for strong-minded individuals. (This is a poor translation of the title of an article written by Dr O Itzinger and published in MONITOR 4/84; the title itself deserves a loud hurrah and reads 'APL – Eine Weltsprache für Individualisten'.)

"Since then the steering committee and a few dedicated SAUG members have kept the group going over dale and hill. The number of members is relatively small in the absolute sense (60), but large compared to the Swiss population as a whole. In these conditions the steering committee decided that the major effort would be centred on communication between members via a journal to be published regularly 3 or 4 times a year, including any material dealing with APL in all its aspects, first in Switzerland and second in the world at large.

Luckily for us, at about the same time, VECTOR surfaced in a format which inspired respect and awe, but turned out to be a very exciting source of information on what was happening on the world APL scene. We would like to acknowledge this debt here and now and are confident that, as time goes on, more and more of the material published in our APL-Notizen will be original and Swiss, whatever that means. (Following Georges Mikes and his 'Switzerland for Beginners', there is no such thing as the Swiss, and we feel it strongly here, as the German-speaking majority is always slightly on edge at having to tolerate a loud French-speaking group, with minority and majority rivalling endlessly in their propensity to use EDP jargon, i.e. bad English as borrowed from our friendly neighbours beyond the Atlantic.)

"All this to let you know that we do exist and that we welcome new members at home and abroad (regardless of mother tongue). Our address is given below and the membership is only 40 (overvalued) Swiss Francs. Anyone who wishes to join will receive our warmest welcome."

Enquiries about SAUG should be addressed to:

Dr J L Metzger,
President, SAUG,
c/o Diametal AG,
Solothurnstrasse 136,
CH-2504 Biel,
Switzerland.

Sydney APL Users Group

John Searle has written to tell us about the local APL group in Sydney. Its address is:

Sydney APL Users Group,
GPO Box 1425,
Sydney,
N.S.W., 2001,
Australia.

APLBUG newsletters

We continue to receive regular updates of the Bay Users' Group newsletter from California with details of their monthly meetings. There is obviously a very active group there. As well as these reports, there are occasional notes on idioms and other general topics. The latest theme of interest is the burning topic, in California at least, of the dramatic impact of the Earth's decreasing rate of spin. The problem was to work out how long it would take before the diurnal drift would have dribbled away a day, given that the length of a day increases by 1-2 milliseconds per century.

Another interesting development is that the group plans to have an APL booth at the 3-day PC FAIRE in San Francisco, this September. With all the current emphasis on widening awareness of APL (see various APL86 reports in this issue), this seems to be a highly practical way to contribute. The "Bay Area" recruiting ground is around San Francisco, and the group has active members scattered between Santa Rosa and Gilroy. Applicants should contact Curtis A Jones at 210 South 12th Street Apt 1, San Jose, CA 95112. The membership dues are \$15 for a full year or \$8 if you apply between July and December.

APL ♦ 385

Background

APL has been a major development language at Rowntree Mackintosh since 1979. During that time we have built a comprehensive library of screen-handling and data-handling code. It is designed for quick development, easy maintenance and efficient running.

Recently we have moved several applications from VS APL to a variety of PCs. This has led us to develop portable versions of our screen-drivers so that we can transport systems with the minimum of fuss.

APL ♦ 385 makes this code available to the APL community at large. The software is licensed on a 'Freeware' basis, only the documentation is copyright. The price (£50 for the PC version, £125 mainframe) is as low as I can make it if I am to cover the cost of advertising and distribution.

Adrian Smith
Brook House
Gilling East
York

Ampleforth (04393)-385

FSM ♦ 385

What can it do for me?

- *** it will give you a really professional toolkit for building complex screen-based systems.
- *** your screen-handling code will be at least 4 times faster than a similar system using the IBM FSDESIGN routines. If you choose to exploit the GDDM page facilities you can get a further dramatic improvement.
- *** you can make full use of character painting, pop-up help, and GDDM graphics as a natural part of your system design.
- *** your code will move down to the PC with minimal modification.

Please note that FSM ♦ 385 started life on VS APL. It runs very adequately on the PC, but of course it does not fully exploit all the available features. If your interest is solely in PC-based systems you might do better to look at a dedicated PC package (such as Powertools/PC).

APL Product Guide

Compiled by Steve Lyus

VECTOR's exclusive APL Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We do depend on the alacrity of suppliers to keep us informed about their products so that we can update the Guide for each issue of VECTOR. Any suppliers who are not included in the Guide should contact me to get their free entry – see address below.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage.

The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages. Where no UK distributor has yet been appointed, the vendor should indicate whether this is imminent or whether approaches for representation by existing companies are welcomed.

For convenience to readers, the product list has been divided into the following groups:

- ★ Complete APL Systems (Hardware & Software)
- ★ APL Timesharing Services
- ★ Other services
- ★ APL Interpreters
- ★ APL Visual Display Units
- ★ APL character set printers
- ★ APL-based packages
- ★ APL Consultancy
- ★ Vendor addresses

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

Note: 'poa' indicates 'price on application'

All contributions to the APL Product Guide should be sent to:

Steve Lyus
Metapraxis Ltd.,
Hanover House
Coombe Road, Kingston
KT2 7AH

(N.B. Change of Address again!)

COMPLETE APL SYSTEMS

COMPANY	PRODUCT	PRICES £	DETAILS
Analogic	The APL Machine	\$60,000+	AP500 array processor, 4 Mb data memory, 80 Mb disk drive.
APL Software Technology	Ericsson PC	2,000 - 5,000	Authorised Ericsson dealer supplying complete APL systems based on Ericsson PC - fully IBM compatible including hardware add-on boards.
Cocking/Drury	MicroAPL SPECTRUM SAGE II SAGE IV	6,000 - 35,000	Supplied as part of a turnkey system. See MicroAPL entry.
Gen. Software	Myriade	poa	Ti computer + APL & APL operating system
Inner Product	IBMP C	2,000 - 6,000	IBM PCs supplied for turnkey applications
M.B.T.	MBT Series 10 TORCH	poa poa	UNIX/68010 based multi-user APL system 68000/Z80 multiprocessor
MetaTechnics	—	3,000+	Details on application
MicroAPL	Aurora	23500	Multi-user APL computer using 68020 CPU. Std. configuration 2Mb RAM, 16 RS232 ports, 66 Mb hard disc, 720K diskette
	Sinclair QL with QL/APL	300+	Fully expandable APL system with colour graphics.
	SPECTRUM	11,000 - 15,000	Expandable multi-user APL computer using Motorola 68000. Std. configuration 1 Mb RAM, 12/36 Mb disc, 12 ports.
	STRIDE 440	8,500	Multi-user APL computer, 1 Mb RAM, 12/18 Mb disc.

APL TIMESHARING SERVICES

COMPANY	PRODUCT	PRICES £	DETAILS
Boeing	Mainstream APL	poa	Enhanced IBM VS APL (CMS)
Mercia	APL*PLUS	poa	STSC's Mainframe Service - MAILBOX etc.
I.P. Sharp	SHARP APL	poa	International Network application systems and public databases.

APL VISUAL DISPLAY UNITS

COMPANY	PRODUCT	PRICES £	DETAILS
Farnell	Tandberg TDV 2221	995	Ergonomic design APL terminal, 50-19200 baud, 15" anti-reflex screen, low profile keyboard
Gen. Software	Tandberg TDV 2271	1,195	Combined APL/ANSI ergonomic terminal as above.
	Mellordata Elite 3045A	400	Second-hand
M.B.T.	various		Contact MBT for details
MicroAPL	Insight VDT-1	795	Inexpensive APL VDU
	Insight GDT-1	1,450	With monochrome graphics
	Tektronix 4105	3,545	High resolution colour graphics supporting APL character set on MicroAPL hardware.
	Concept 201	1415	APL VDU with 8 page memory
	Concept 201G	1935	Graphics VDU
Shandell	AVT-APL+ GVT-APL+	1,200+	ANSI 3.64; DEC compatible; full overstrike chars; 4/8 pages; 2/3 comms, ports; 80/132 cols; windowing; 12" screen; 46 PF keys; Tek 4013 graphics compatible.
	HDS 200	1,200+	As above plus 15" screen; viewports; smooth scroll; NVM storage; 55 PF keys; Tek 4014, VT640/DQ640 & Visual 500 graphics compatible.
Textronix	4114B	13,500+	19" D.V.S.T.: Graphics: 3120 x 4096 displayable; Intelligent: up to 800K memory; APL keyboard (option 4E)
	4125	21,550+	19" 2D colour graphics; Workstation (1280 x 1024); Intelligent: up to 800K memory; APL keyboard (mod AP)
	4128	26,822+	As 4125 plus 3D wireframe

APL PRINTERS

COMPANY	PRODUCT	PRICES £	DETAILS
DataTrade	Datasouth DS180+	1,295	180 cps matrix printer with 4K buffer, 9 x 7 dot matrix and APL option.
	Datasouth DS220	1,695	Letter quality; graphics capability, APL option (both available with IBM Twine or Coax interface).
Inner Product	Epson FX80	500	Soft char. set, 160 cps, 80 column
	Anadex 9620	1,150	200 cps., 132 col., tractor feed
	Siemens PT88	620	180 cps., 80 col., silent
	TGC Starwriter	1,180	40 cps., letter quality
M.B.T.	Facit 4565	poa	40 cps letter-quality
	Facit 4510/11/12	poa	Matrix printers
MetaTechnics	Quen-data	295	Low-cost APL Daisy-wheel printer
MicroAPL	Datasouth DS180+	1,295	See DataTrade entry
	Philips GP300	1,924	Matrix printer with letter & draft quality and APL.
	Qume Letterpro20	549	APL/ASCII Daisy-wheel printer

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES £	DETAILS
APL People	Employment Agency	poa	Permanent employees placed at all levels. Contractors supplied for short/long-term projects, supervised.
Mine of Information	APL Book Service		See booklist
I.P. Sharp	Productivity Tools	poa	Utilities for systems, operations, administration & analysts; auxiliary processors, comms software, international network.
	Databases	poa	Financial, aviation, energy and socioeconomic.

APL PACKAGES

COMPANY	PRODUCT	PRICES £	DETAILS
APL ◇ 385	FSM 385	PC: 50	Screen development
	DRAW 385		Screen design
	DB 385	Mainframe: 125	Relational W.S.
	GEN 385		Utilities
APL Software Technology	POWERTOOLS/PC	295	Assembler written replacement function for commonly used CPU-consuming APL functions. Includes extended full-screen Forms Processor.
Beta-plan	BETA-FONT	poa	Multiple font PC character generator. Dealers required for non-Scandinavian countries.
Boeing	TABAPL	poa	Hierarchical Planning System
Butel	Merlin	5,000	Mainframe APL spreadsheet runs under VM/CMS, TSO, VSPC
	Merlin/PC	poa	Version for APL*PLUS/PC
Cocking/Druy	<i>Mainframe</i>		
	STSC's SHAREFILE	poa	Component files, quad- functions & nested arrays for IBM VSAPL under VM/CMS & MVS/TSO
	SHAREFILE AP	15,000	STSC's sharefile for APL2
	FILEMANAGER	poa	STSC's database package.
	FORMAT	2,250	Enhanced report formatting.
	WSAIDS	795	Workspace doc. & development aids.
	COMPILER	30,000	First APL compiler. Available with APL*PLUS enhancements and Sharefile under VM/CMS & MVS/TSO
	SQL	poa	An interface between SQL and APL*PLUS for VSAPL
	FMT	poa	Full featured FMT for APL2
	DATAPORT	poa	Powerful Information Centre spreadsheet incorporating data exchange between APL and FOCUS, IFPS, SAS, APL/DI, ADRSII, LOTUS123, VISICALC, MULTIPLAN, DIF files
	<i>Microcomputer</i>		
	STATGRAPHICS	595	Powerful Statistics and graphics on IBM PC's, PC/at's and compatibles
	Release 2		
	Update	poa	Update from release 1 to release 2
APL*PLUS PC Tools		VOL 1	325 Incl. 327x IRMA support, RAM disk, full screen data entry, menu input, report generation, games.
		VOL 2	125 Incl. file documentor, screen editor, exception handler.
	APL*PLUS PC Fin & Stat. Library	350	Financial & statistical routines
	MetaScreen	245	See MetaTechnics entry.
	MetaPack	95	See MetaTechnics entry.
	POWERTOOLS	295	See APL Software Technology entry.
	SPREADSHEET MANAGER	195	APL-based spreadsheet for APL*PLUS/PC. Cell arithmetic; transfers to ASCII, LOTUS
E&S	PROTOPAK consisting of:		Packages for prototyping management information systems - PC & mainframe
	RMS	Modules	Relational databases.
	AMS	250+	Multi-dimensional arrays
	RAMS		Combined RMS & AMS.
	BMS		Dynamic financial modelling & forecasting
	FMS		Full-screen handler for APL*PLUS/PC. (AP 124-based)
	CMS		Communications package.
	SOS	poa	Scheduled ordering and stock control.
Gen. Software	PROPS	500+	Spreadsheet system for Product and/or Project Planning.

H.M.W.	INPUT PRINTPAK VIEWPAK	poa poa poa	Matrix manipulation package for data entry & report generation Block printing for VM/CMS AP124 Protocol emulator for IBM/PC	
Holtech	CASH	3,500 -10,000	Accounting package & hotel management system on MicroAPL SPECTRUM & SAGE CPUs.	
Inner Product	Viewcom APL/dBASE II APL/dBASE III APL/LOTUS APL/WORDSTAR APL/MULTIPLAN CEMAS	150 150 150 150 150 150 3,500	Control Viewdata from APL Interface APL with dBase II Interface APL with dBase III Interface APL with Lotus Interface APL with Wordstar Interface APL with spreadsheet EEC monetary and agrimonetary analysis.	
M.B.T.	RHOMBUS HASLEMERE	poa poa	Integrated Office System Hotel Accounting System	
Mercia	STATGRAPHICS APL*PLUS tools	995	Integrated stat. graphic system for PCs.	
		VOL 1	325	IBM PC Utilities: IRMA3270 comms, full screen, RAM Disk report generator
		VOL 2	125	File documentation, screen editing, Exception handling
	FINANCIAL AND STATISTICAL LIB. INFO CENTRE	325	Financial and Statistical analysis	
		2,000	Full-screen entry, display & multi-dimensional analysis. Interfaces to other I.C. products. Runs under VM VSAPL on IBM mainframes.	
	EXECUCALC	-20,000	Mainframe Spreadsheet with VisiCalc and Lotus 1-2-3 functionality requires VSAPL under TSO or VM.	
	EXECUPLOT	4,000	Mainframe Graphics display system with VisiPlot functionality requires VSAPL under TSO or VM and GDDM.	
	EXEC*U*STAT	3,200	Easy to use Statistics for management.	
	MICROSPAN	495	Comprehensive APL tutor	
		250		
MetaTechnics	MetaScreen	245	Full-screen handler for APL*PLUS/PC, based on VSAPL AP124	
	MetaPack	495	Comprehensive utilities package for APL*PLUS/PC. Includes MetaScreen, MetaWS, Browse, Toolbox, Numeric Editor	
	ADAPTA DLS	poa	Production & purchasing scheduling for process manufacturing.	
	ADAPTA MSP	poa	Job-shop loading & scheduling for multi-stage production.	
MicroAPL	MicroTASK	250	Product development aids	
	MicroFILE	250	File utilities and database	
	MicroPLOT	250	Graphics for HP plotters etc	
	MicroLINK	250	General device communications	
	MicroEDIT	250	Full screen APL editor	
	MicroFORM	250	Full screen forms design	
	MicroSPAN	250	Comprehensive APL tutor	
	MicroGRID	poa	Ethernet & other networking	
	APLCALC	400	APL spreadsheet system	
	MicroPLOT/PC	250	For APL*PLUS/PC product	
	MicroSPAN/PC	250	For APL*PLUS/PC product	
	PC TOOLS Vol 1	495		
	STATGRAPHICS	695		
Parallax	ExecuCalc	\$5,000	Mainframe-based electronic spreadsheet for VM/CMS & MVS/TSO with links to micro products.	
	ExecuPlot	\$5,000	Mainframe-based colour graphics with micro links.	

I.P. Sharp	ACT	poa	Actuarial system
	APS	poa	Financial Modelling
	BOXJENKINS	poa	Forecasting technique
	CONSOL	poa	Financial Consolidation
	COURSE	poa	APL Instruction
	EASY	poa	Econometric Modelling
	FASTNET	poa	Project Management
	GLOBAL LIMITS	poa	Exposure management for banks
	MABRA	poa	Record maintenance/reporting
	MAGIC	poa	Time series analysis/reporting
	MAGICSTORE	poa	N-dimensional database system
	MAILBOX	poa	Electronic Mail
	MICROCOM	poa	Mainframe to micro link
	SAGA	poa	General graphics, most devices
	SIFT	poa	Forecasting system
	SNAP	poa	Project management
	SUPERPLOT	poa	Business graphics
	VIEWPOINT	poa	4GL - Info centre product
	XTABS	poa	Survey Analysis
Sugar Mill	Stat 1	\$129.95	Statistical tool/box, menu driven

APL CONSULTANCY

(prices quoted are per day unless otherwise marked)

COMPANY	PRODUCT	PRICES £	DETAILS
APL People	Consultancy	poa	All levels, most APL systems
APL Software Technology	Consultancy	poa	Technical & business systems, micros, networking & communications a speciality.
Boeing	Consultancy	poa	
Camacho	Consultancy	poa	Specialising in programming & manual writing.
Cocking/Drury	Consultancy	120-150 140-200 185-300 275-400	Junior consultant Consultant Senior consultant Managing consultant
	Courses	375 330 595 poa poa	3 day Fundamentals 3 day Advanced 5 day System Design Introduction to APL2 APL2 in depth
Delphi	Consultancy	poa	Specialising in management reporting systems and APL on microcomputers.
Dyadic	Consultancy	poa	APL system design, consultancy, programming & training for Dyalog APL, VSAPL, APL*PLUS, IPSA APL etc.
E & S	Consultancy	150 -250	System prototyping: all types of information system.
FASTCODE	Consultancy	poa	Specialise in improving performance of APL applications on micros & mainframes.
Gen. Software	Consultancy	100+	
H.M.W.	Consultancy	100-250	System design consultancy, programming.
Inner Product	Consultancy & Training	200	On-site micro-mainframe APL, PC/DOS & Assembler
Lloyd Savage	Consultancy	poa	Decision support, particularly specialising in Sales & Marketing systems.
M.B.T.	Consultancy & Courses	poa	
Mercia	Consultancy & Training	poa	APL*PLUS & VSAPL consultancy. Onsite & Offsite APL courses.
MetaTechnics	Consultancy	poa	Management Information & Production. Management Systems & Process Control.
	MetaSys	poa	Prototyping APL-Hybrid Custom Programming Service.
MicroAPL	Consultancy	poa	Technical & applications consultancy.
M.T.I.	Consultancy	poa	Specialise in Maintenance and development of existing APL systems
Parallax	Consultancy & Training	\$750 poa	Introductory APL, APL for End-user & Advanced Topics in APL
QB On-Line	Consultancy	200	Specialising in Banking, Financial & Planning Systems.
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL
I.P. Sharp	Consultancy	poa	Consultancy & support service world-wide.

APL INTERPRETERS

COMPANY	PRODUCT	PRICES £	DETAILS
APL Software Technology	APL*PLUS/PC	600 -695	See Cocking & Drury entry. Release 5 now available. Discounts available on multiple copies.
Cocking/Drury	APL*PLUS/PC	475	STSC's full featured APL for IBM PC, PC/AT and compatibles
	Release 6		
	Release 5	100	Extension from release 5 which incorporates +64K object support
	Release 4 update	150	Extension upgrade from release 4
	Release 3 update	110	Extension upgrade from release 3
	Run-time	poa	Closed version of APL*PLUS PC which prevents user exposure to APL.
	APL*PLUS UNX	poa	STSC's 2nd generation APL for IBM PC/AT, DEC, AT&T and other Unix computers.
Dyadic	Dyalog APL	1,000 -8,000	2nd gen. portable APL for UNIX systems, e.g. IBM PC AT, SUN, MC68000, HP9000, NCR, VAX, Gould, Amdahl.
Gen. Software	APL*MYRIADE	poa	Runs on Texas Instruments TI990 range.
IBM UK Product Sales	IBM PC APL	poa	Event-handling & APs for full-screen I/O disks, diskettes, asynch. comms.
Inner Product	VIZ::APL	250 -350	8-bit Ziog Z-80 CP/M
	APL*PLUS/PC	600	See under Cocking & Drury
M.B.T.	Dyalog APL	poa	See Dyadic Systems entry
	MBTAPL	poa	Enhanced Dyalog APL for MBT hardware.
	VIZ::APL	poa	Customized for TORCH hardware
Mercia	APL*PLUS/PC	695	STSC's full-feature APL for IBM PC, PC/AT Compaq, Olivetti, Wang, Apricot, Ericsson etc
	Release 5.0 upgrade to 5.0	150	
	APL*PLUS/UNIX	poa	Interpreter for UNIX systems: WICAT, CADMUS, CALLAN, FORTUNE 32:16, HP, 9000/500, OLIVETTI 13B2
MetaTechnics	APL*PLUS Release 4	695	Discount on quantity.
MicroAPL	APL.68000	1,000+	Full implementation with component files, error trapping etc. for SPECTRUM, SAGE & other MC68000-based computers.
	QL/APL (keyword)	87	Full keyword APL for QL with many extra features.
	QL/APL (APL chars)	87	VSAPL compatible APL for QL with many extra features.
	MAC/APL	257	
	AMIGA/APL	326	
	ATARI/APL	131	
	APL*PLUS/PC - REL 5	495	
Portable	PortAPL	\$195 \$275 \$2,995	IBM PC Software Mackintosh DEC VAX
I.P. Sharp	Sharp APL/PCX	2,575	For IBM XT/AT
		1,000+	For IBM mainframes
	Sharp APL/PC	325	For IBM PC or PC/XT

VENDOR ADDRESSES

COMPANY	CONTACT	ADDRESS & TELEPHONE No.
Analogic Corporation	Denise Favorat	8 Centennial Drive, Centennial Industrial Park, Peabody, Mass. U.S.A. 01961 ☎ 617-246-0300
APL 385	Adrian Smith	Brook House, Gilling East, York. ☎ 04393-385
APL People	Valerie Lusmore	17 Barton Street, Bath, Avon. ☎ 0225-62601
APL Software Technology	John Hagger or Per Hullen	14 Rosewood Avenue, (UK) Ltd. Alveston, Bristol BS12 2PP. ☎ 0454-415737 46 Vicarage Hill South Benfleet, Essex SS7 1PB. ☎ 03745-50501
Beta-plan APS	Kim Andreasen	Stengrade 75, DK-3000 Helsingør, Denmark. ☎ 45 221 48 48
Boeing Computer	Anne Harding	19 Fitzroy Street, Services (Europe) Ltd., London W1P 5AB. ☎ 01-631 0808
Butel Technology Ltd.	Mike Munro	Butel House, 3 Great West Rd., London W4 5QJ. ☎ 01-995-1433
Anthony Camacho		2 Blenheim Road, St. Albans, Herts AL1 4NR. ☎ St. Albans 60130
Cocking & Drury Ltd.	Romilly Cocking Brian Drury	16 Berkeley Street, London W1X 5AE. ☎ 01-493 6172 155 Friar Street Reading RG1 1HE. ☎ 0734-588835
Datatrade Ltd.	Tony Checksfeld	38 Billing Road, Northampton, NN1 5DQ. ☎ 0604-22289
Delphi Consultation Ltd.	David Crossley	Church Green House, Stanford-in-the-Vale, Oxon SN7 8LQ. ☎ 03677-384
Dyadic Systems Ltd.	Peter Donnelly	Park House, The High Street, Alton, Hampshire. ☎ 0420-87024
E & S Associates	Frank Evans	19 Homesdale Road, Orpington, Kent BR5 1JS. ☎ 0689-24741
Farnell International Instruments Ltd.	R. Fairbairn or Roger Attard	Jubilee House, Sandbeck Way, Wetherby, W. Yorks. ☎ 0937-61961 Davenport House, Bowers Way, Harpenden, Herts. ☎ 05827-69071
FASTCODE	Andrew Dickey	P.O. Box 281, Croton-on-Hudson, New York 10520, U.S.A. ☎ (914) 271-3200
General Software Ltd.	M.E. Martin	22 Russell Road, Northolt, Middx. UB5 4QS. ☎ 01-884 9537
H.M.W. Programming Consultants Ltd.	Ken Jackson	142 Feltham Hill Rd, Ashford, Middx. TW15 1HN. ☎ 07842-41232
Holtech Ltd.	Jan Bateman	'O' Block 4th Floor, Metropolitan Wharf, Wapping Wall, London E1 9SS. ☎ 01-481 3207
IBM UK Ltd	Chris Sell	PO Box 32, Alencon Link, Basingstoke, Hants. RG21 1EJ. ☎ 0256-56144
Inner Product Ltd.	Dominic Murphy	Eagle House, 73 Clapham Common Southside, London SW4 9DG. ☎ 01-673 3354
Lloyd Savage Ltd	Phillip Johnson	Cambridge House, Oxford Road, Uxbridge, Middx, UB8 2UD. ☎ 0895-59826
Mercia Software Ltd.	Gareth Brentnall Barrie Webster	Aston Science Park, Love Lane, Birmingham B7 4BJ. ☎ 021-359 5096
MetaTechnics Systems Ltd	John Stanbridge David Toop	Unit 216, 62 Triton Road, London, SE21 8DE. ☎ 01-670 7959
MicroAPL Ltd.	Jonathan Matthews Paul Thornton	Unit 1F, Nine Elms Industrial Estate, 87 Kirtling Street, London SW8 5BP. 01-622 0395
Mine of Information	Richard Ross-Langley	PO Box 1000, St. Albans, Herts AL3 6NE. ☎ 0727 52801
Modern Business Technology Ltd. (MBT)	Michael Branson	P.O. Box 87, Guildford, Surrey GU4 8BB ☎ 04868-23956
M.T.I.	Ray Cannon	7 Pine Wood, Sunbury-on-Thames, Middx. TW16 6SH ☎ 09327 80848
Parallax Systems Inc.	Kevin Weaver	60 West 9th Street, New York, New York 10011, U.S.A. ☎ 212-475-4001
Portable Software	Richard Smith	60 Aberdeen Ave, Cambridge, Mass. U.S.A. 02138. ☎ 617-547-2918
QB On-Line Systems	Philip Bulmer	5 Surrey House, Portsmouth Rd Camberley, Surrey, GU15 1LB. ☎ 0276-20789
The Rochester Group	Robert Pullman	164 Pinnacle Rd., Rochester NY 14620 ☎ 716-461-3169
Shandell Systems Ltd.	Maurice Shanahan	12 High Street, Chalfont St. Giles, Bucks HP8 4QA. ☎ 02407-2027
I.P. Sharp Associates Ltd.	David Weatherby	10 Dean Farrar Street, London SW1. ☎ 01-222 7033
Sugar Mill Software Corp.	Lawrence H. Nitz	1180 Kika Place, Kailua, Hawaii 96734 ☎ (808) 261-7536
Tektronix UK Ltd.	Paul Morgan	Fourth Avenue, Globe Park, Marlow, Bucks SL7 1YD. ☎ 06284-6000

APL BOOKLIST

Title	Price £	UK Post & Pack (£)
An Interactive Approach, Gilman & Rose	24.35	2.65
APL: An Introduction, H. Peelle	15.00	2.25
Introducing APL to Teachers, K. Iverson	3.00	.40
Introduction to APL for Scientists & Engineers	3.00	.40
An APL Notebook, Barrie Wetherill	1.90	.50
A Source Book in APL, A. Falkoff & K. Iverson	10.00	1.80
APL: Design Handbook for Commercial Systems, A. Smith	13.10	1.50
APL and Insight, P. Berry and G. Bartoli	4.50	.55
APL in Practice, STSC	40.00	2.50
APL: The Language and its Usage, R. Polivka & S. Pakin	36.35	2.70
APL Quote-quad the Early Years	23.00	2.70
APL Business Technology '83 Proceedings	11.20	2.70
APL SV Reference Card	.50	.50
Designing APL Systems, Toronto Press Procs '82 Vol 2	8.50	1.80
FinnAPL Idiom Library	11.20	1.30
Sharp APL Reference Book	1.50	.50
Sharp APL Reference Manual, P. Berry	10.50	2.70
Yale University Idiom List	2.00	.50
APL in Exposition, K. Iverson	3.00	.50
Algebra, K. Iverson	10.00	2.30
Solutions to Algebra, K. Iverson	3.00	.50
Elementary Analysis, K. Iverson	8.00	1.50
Resistive Circuit Theory, R. Spence	20.00	2.80
Star Map, P. Berry & J. Thorstensen	6.00	.50
Reliable Software through Composite Design, Myers	15.00	1.30
APL Lapel Pin with gripping backplate	2.00	.50
Whizzbangs, Volume II, R. Sykes	10.00	1.80
Whizzbangs, Volume I, R. Sykes	10.00	.40
Information Centres Toronto Procs '84	8.00	1.80
Application Systems in APL, Gibson Levine Metger	30.00	2.70
A Dictionary of APL, K. Iverson	2.50	.50
A Concise Dictionary of APL, K. Iverson	2.00	.50
APL '86 Tutorials, A. Camacho (9.30 members)	12.00	2.00
APL '86 Conference Proceedings, D. Ziemann (13.30 members)	17.30	2.25
Mathematics and Programming, K. Iverson	8.00	.95
Applied Mathematics for Programmers K. Iverson	8.00	1.35
APL Toolkit, (CIPS APL SIG), R. Levine	4.50	.95
Special APL 86 Wallet offer (see separate advert)	22.50	5.00

Please order direct from Mine of Information:

APL Book Service, PO Box 1000, St. Albans, AL3 6NE, U.K. ☎ 0727-52801

Prices are subject to change without notice, Access & Visa accepted, or sterling cheque with order.

Outside UK add an **extra** amount per book –

£1 to Europe, £2.50 Africa & Middle East, £5 elsewhere (all sent airmail).

The Education VECTOR

by Norman Thomson

Welcome to a new feature of VECTOR. In it we shall report developments in the progress of APL in the world of education. And what richer source of material for the opening column than the Education Day organised by Sylvia Camacho as a curtain-raiser to APL86. This event attracted over 100 of those most intimately connected with the combination of APL and education. The attendance included educators *of* APL, educators *with* APL, educators *without* APL, educators of educators, educational researchers, educational marketers. With twelve formal speakers covering such a spectrum, plus a discussion forum, it wasn't surprising that every aspect of education and APL (and its competitors) was touched on at some time, and quotable sayings abounded:

"APL is easy, fun and of great practical use" (Romilly Cocking)

"APL helped me to see the problem, and not only that, it worked!" (Bernadette McAlinden)

"APL is part of the continuing evolution of symbolic notation, and thus of the evolution of scientific thought" (Ray Polivka)

"Students accept it, staff do not" (Bob Spence)

"The biggest constraint to the growth of APL is the people who know APL" (Romilly Cocking)

"APL removes the computer barrier" (Norman Thomson)

"Computer people are learning to be baffled by parallel thinking" (Anthony Camacho)

"The language for problems – pure and symbol"

The opening address on "Executable Analytic Notation in Teaching Mathematics and Programming" was delivered with typical Iverson flair. Attempts to integrate other programming languages with mathematical instruction have failed – only APL has been able to combine executability with analytic power. Its real rival is CMN (i.e. Conventional Mathematical Notation – please memorise new acronym!). There are no courses given in CMN – likewise APL can be introduced "in passing" in a wide range of contexts, given the right concomitants of a dictionary, and a computer to experiment with.

This view of APL as an "emergent" language was reinforced by the talks of Professors Hawkes and Spence, who separately related the naturalness with which APL *emerges* as the instructional medium for their respective courses with a minimum of initial formal instruction.

Roger Knott from the Centre for Mathematical Educational Technology at Loughborough University also sang the virtues of an emergent language. The emphasis of his Institute is on mathematical investigation, an area in which candidate languages, e.g. LISP, APL, LOGO, must compete on the grounds of power, expandability, good support features and the capability to add excitement. In spite of the clear advantages of a computer as a super-calculator and as a calm, uncriticising "student's friend", there remained a residual, irrational hostility of teachers towards it.

Further sound advice in this direction came from Roy Atherton of Bulmershe College of Education, Reading. The promotion of APL in schools has to be supported by an honest case and good materials. Explicit thought should be given to the target audience – is it the brightest pupils or all pupils; have pilot projects been considered; and is there a realistic view of the time frame for success. Taking structured programming and modern mathematics as analogues, a decade seemed about par for introducing something new, surely a sobering corrective to the impulsive energy and impatience of APLers who above all think in terms of making things happen *now*!

Romilly Cocking aired his concern at the failure to produce new APLers at the level of secondary and tertiary education, and added advice as to why APL has not taken off as it should. Avoid the dangers of oversell, he advised, and the temptation to over-reach the audience. Occasionally the APL message gets through quickly, but more often hostility, conservatism and indifference result in scepticism (APL too good for its own good!), horror stories, or "reasoned" objection, handed down and absorbed in lieu of knowledge.

These experiences found echoes in Ken Johnson's talk. Ken is from the Artificial Intelligence Unit at Edinburgh University, where he has promoted the introduction of LOGO into the Scottish secondary school curriculum, the rationale for which is similar to that for introducing APL. Where officialdom does not understand, its response is to ignore – note the total absence of LEA representatives and maths advisors present, in spite of an intensive mailing and advertising campaign. Ken's talk led to an interesting discussion about the break-off point between "easy" LOGO (accessible to 8-year olds), and "difficult" LOGO (beyond school level). There is a similar break-point in APL, but surely this comes at a much later stage making APL a much more suitable language for backing the secondary curriculum. It was good to have the two foremost promoters of APL in American public education present, namely Linda Alvord and Howard Peelle, and some comfort to know that the Americans have similar problems at this level.

Anthony Camacho put forward a view of programming, in which the power of forthcoming parallel processors will be unusable except in APL because of the inability of traditionally educated programmers to think in parallel. The theme of parallelism was also taken up by Dr. Frank Evans of South Bank Polytechnic in examples of how multi-dimensional arrays and other APL features naturally match problems arising in the chemical and oil industries. APL produces a unique programming environment, he concluded. It was interesting to hear this academic view balanced a short while later by Bernadette McAlinden's admission that in the real world of Tesco Stores (18,000 product lines!), the purity of multidimensional arrays with 525 planes was sometimes tempered with a "scissors and Sellotape" approach!

One fact that nobody could doubt by the end of the day was that shortage of money is the biggest barrier to the progress of APL in Education. In the concluding talk, Ray Polivka declared starkly that money is in Industry, and it thus behoves us all to increase APL's exposure by writing letters to the technical press in order to stamp out misconceptions and to let the Captains of Industry be in no doubt that APL is a language of industrial strength.

The above were just some of the riches to which I added my plea as Education Officer for action to get something moving in British schools to make the APL terminal an unobtrusive presence in the classroom and thus realise some of its great possibilities. This, I said in apology to overseas visitors, was a U.K. concern – how wrong I was! The events of the ensuing week winkled out the parallel concerns of our American cousins, and led to the formation of a group dedicated to the publication of a Public Domain APL for small machines, e.g. the BBC Micro, on both sides of the Atlantic. The joint Chairmen are Anthony Camacho (UK) and Ed Cherlin (USA). British members are Dave Ziemann, of Cocking and Drury, Sylvia Camacho and myself; US members are Howard Peelle and Linda Alvord. We hope to raise sufficient funds to have Paul Chapman implement the portable product. Meanwhile Inner Product have announced that they are putting the VIZ::APL product into the public domain with immediate effect to the benefit of those with 8-bit CP/M machines.

I shall be happy to hear from anyone in the educational world about positive things that they are doing with APL, which I may not already know about; please write to me at:

17 St. James Terrace,
Winchester, SO22 4PP.

Roy Sykes said in his closing address at APL86 that the next phase for APL was not more technical progress, but wider dissemination. The most important aspect of this is Education, through which, given the medium of public domain APL, we shall in time address an audience not of thousands, but of millions!

Teaching Statistics through APL

by Professor Alan Hawkes and Dr Alan Sykes

APL is alive and in action at Swansea University Statistics Department – that was the message of Alan Hawkes' tutorial, and to prove it Richard Green's fingers roamed nimbly over Barco-connected keyboard, illustrating the words of his professor as he talked through an outline of the APL-based course given to all first-year statistics students. There is no formal APL instruction at this stage – APL is learnt in the context of statistics by having the students use a workspace of utility functions which enable them to draw histograms, obtain standard descriptive statistics, calculate expected values, produce binomial distributions, and construct Operational Characteristic Curves for acceptance sampling.

In the second year, there is an optional course on Statistical Computing, which includes a more comprehensive study of APL. In the third year, projects can be based on APL, and at this point Richard took the floor to talk about the influence of APL on the project he has just completed on nesting sets of rectangles within each other.

Alan then talked through a first-year case-study involving the analysis of predicted birth dates of Glamorgan babies based on ultra-sonic scanning as one method, and mothers' forecasts as another.

In all, this turned out to be a convincing demonstration that APL in education works!

Topics for a second course in APL

by Stephen Jaffe

In Mobil Labs, APL is the medium for numerical literacy of the research scientists. Following a basic course and a little experience, upgrade skills are necessary and Stephen provides these through a series of three lectures on advanced APL, given at weekly intervals. The first is on controlling multi-dimensional arrays; the second on special uses of grade and scan; and the third on building interfaces from APL to other parts of the system.

This tutorial was a compressed version of these three lectures, emphasising how real-life examples from the Labs are used to illustrate and motivate. Those who have taught advanced APL must have picked up many echoes from their own experience, e.g. the difficulty of explaining in English why double grade-up produces rank in the combinatorial sense! Stephen's informal rules for handling multi-dimensional arrays, and his practical tips for juggling with axis lengths, are part and parcel of the business of teaching APL. Although this material could have been written ten years ago, this is the first explicit statement of such pedagogic considerations, and the accompanying paper in the Proceedings should be a valuable source for APL teachers.

APL.68000 FOR THE APPLE MACINTOSH

APL.68000 on the Macintosh offers a uniquely friendly environment in which to program in APL. It offers the simplest method of writing applications which take full advantage of the Macintosh features, allowing systems written in APL to set a new standard of professionalism and integration with other Macintosh software.

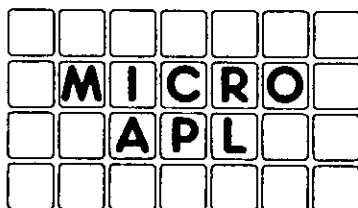
FEATURES:

- Uses standard Macintosh user interface
- Built-in full screen function editor
- Access to Macintosh native files
- Built-in VT52-like APL/ASCII terminal emulation
- Full clipboard support for data exchange
- APL applications can be started from desktop
- Common system commands as pull-down menus

Price: £256.52 excl. VAT

APL.68000 is a trademark of MicroAPL Ltd.
Macintosh is a trademark licensed to
Apple Computer, Inc.

- Full support for Imagewriter and Laserwriter
- Standard non-Apple printers also supported
- User-defined pull-down menus
- User-defined Dialog and Alert boxes
- Full interface to Quickdraw graphics facilities
- Arbitrary I/O via serial ports
- Session manager allows editing of screen lines
- Workspaces can be set as run-time applications
- Applications can use standard ASCII keyboard
- Applications can detect mouse position and state



MicroAPL Limited

Sales Office: 19 Catherine Place, London SW1E 6DX
Telephone: 01-834 9022

NEW PRODUCTS – **ONLY** FROM

MERCIA

Software Limited

MULTI-USER/MULTI-TASKING

APL★PLUS/PC.

From £595

EXEC★U★STAT

– Business Analysis and Graphics

£425

UNITAB

– APL Spreadsheet and Data Manager

£495

APL★PLUS/PC DEBUGGER

– Program Testing Aid

£195

**UNITAB and DEBUGGER demo disks
are now available at just £5 each
Order yours now!**

Orders are now being taken for APL★PLUS/PC Release 6. Make sure you are first to break the 64K barrier.

Mercia also offer a full range of APL education and consultancy services.

To order software or demo disks, or for more details –

★ Write, or call Mercia Software ★

Aston Science Park, Love Lane, Birmingham B7 4BJ
Telephone 021-359 5096. Telex: 334535. Fax: 021-359 0433.

RECENT MEETINGS

This section of VECTOR is intended to document the seminars delivered at recent meetings of the Association, particularly for the benefit of those members based away from London who often find it hard to find the time to attend. It also covers other selected events which are likely to be of interest to the wider APL community.

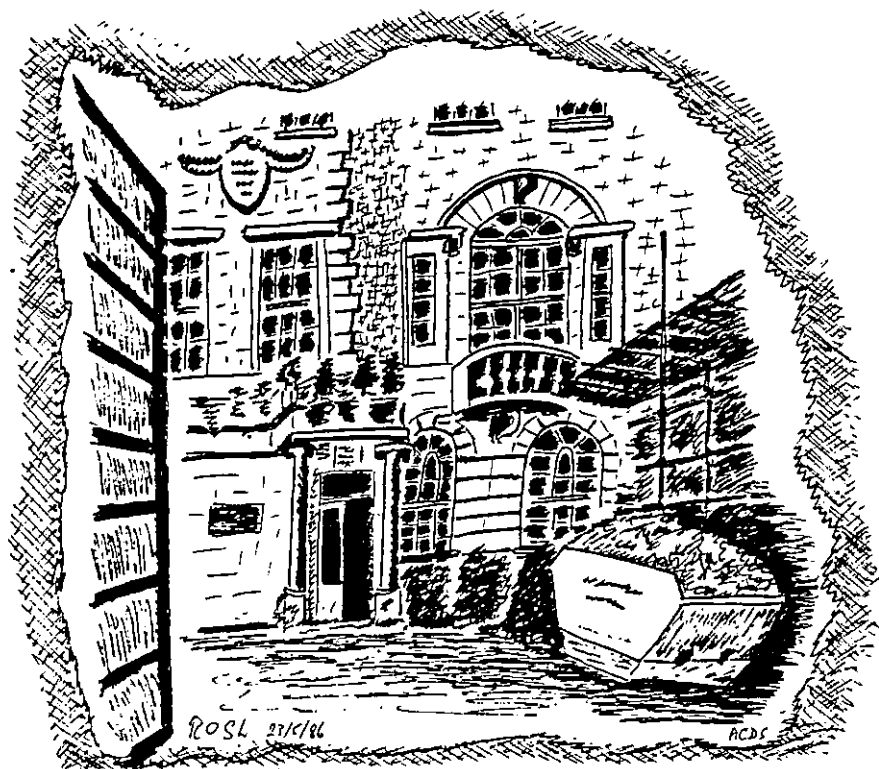
We are dependent on the willingness of speakers to provide us with a written version of their talk, and we would remind them that "a picture's worth a thousand words". Copies of slides and transparencies will enhance their articles.

The Activities Officer (see inside back cover) will respond enthusiastically to offers from individuals who wish to contribute seminars and supporting papers.

Introductory Notes

by Adrian Smith

For once I have had the luxury of being able to use my own notes! The AGM at the Royal Over-Seas League was a stimulating and enjoyable technical meeting, as well as fulfilling its more formal role. In particular we were fortunate to get a pre-run of Ken Iverson's APL86 talk on 'The Split in APL'; I am including comprehensive notes on this, because it was in fact a very different (more technical) presentation from his APL86 plenary (for full details see the Tutorial volume of the Conference Proceedings).



APL and Relational Database

May 23rd 1986

Reading Chicken Bones (Oracle, SQL and APL)

Phil Chastney

By way of preamble, Phil stated the assumptions that most of us knew APL, but were less familiar with the ideas of RDMS. APLs (mostly) come equipped with some kind of component file system, which is a good start, but is definitely not a database. To get at external data without lots of code (and lots of loops) you need a relational system:

- for religious reasons (!)
- because it will be rectangular
- because it will *not* be hierarchical. This is essential because you don't know the paths in advance.

Some Properties of Tables

They have no duplicate rows or columns. The row order doesn't matter. They have no repeating groups (i.e. ragged edges). Tables stand alone without visible links between them. The links are made on the basis of the *data values* alone.

The sort of things you can do with tabular data include:

- Extraction. This commonly employs SQL or an equivalent language (note the word *language* . . . SQL is not a package!). The structure of SQL is APL-like in the extreme:

SELECT	columns
FROM	table
WHERE	some condition is true

. . . there is an obvious parallel with:

`MASK/[1] TABLE[;COL1,COL2,COL3]`

SQL allows expressions very much in the APL mould. As with APL, the results are further tables which can be used in subsequent expressions:

```
SELECT Ename,Job from EMP
WHERE Job=
  (SELECT Job from EMP where Ename='JONES')
```

This will get a list of all the employees who have the same job as anyone called 'Jones'!

- Joining Tables. Typically you might want to pick up departmental data (location in the example below) to go with selected employees:

```
SELECT Ename,Loc from EMP,DEPT
WHERE Ename='ALLEN'
and EMP.Deptno=DEPT.Deptno
```

... no physical (or permanent) join is made ... it is all done with address calculations to give the appearance of a single table.

– Arithmetic: SELECT 12 x Salary where ... and so on. •

The Best of Both Worlds?

This is tremendous for data retrieval, but turns out to be an absolute pain in the neck for data manipulation! Now as we all know, APL is rather good at data manipulation ... can we envisage a language which will get us the best of both worlds? Unfortunately, we cannot, at least not yet. Some of the outstanding problems are:

1. The use of names. 'Select Fred,Joe' is a list of tokens, not names in the APL sense.
2. Treatment of nulls. APL has no way of handling 'no data' as a cell entry in an array.
3. Symbolic notation. SQL is headed down the 'verbosity' route.
4. Implicit joins. APL is a great one for creating all the intermediate objects.
5. SQL is truly *functional*. It has no assignment statement.

If we can't unite the two, can we embed one in the other? It is very hard to see APL sitting within SQL, so the only practical option is some arrangement where APL passes a text string to SQL, which returns an array of data. An Auxiliary Processor is marketed by Oracle to do just this; it works, but has some performance problems:

- most SQL is pre-compiled within COBOL code. Of course you can't do this in APL and you need a lot of nitty-gritty routines to keep track of storage etc.
- APL still only gets one record at a time.

In summary, APL and relational database appear in principle to be made for each other. In practice the links are not straightforward, or as efficient as they could be.

Using APL with a Variety of Databases

Paul Jackson (IPSA)

The theme of this talk was the use of an I.P. Sharp package (Viewpoint) as a way of getting at lots of different databases in a consistent way. Typically an administrator would set up 'virtual files' on a central Id., and would manage the copying of data for users to share. There were lots of nice ideas (e.g. the ability to output a report to your Mailbox), and some problems (e.g. in representing multidimensional data).

Readers are referred to the appropriate IPSA glossies for more details.

The Split in APL

Ken Iverson

Historically, APL has always suffered from splits; for example Sharp found it necessary to employ files, traps and packages to make their APL into a saleable product. Similarly IBM introduced the idea of the Auxiliary Processor with TSIO. None of these mattered a great deal; you can always cover for differences in the environment. It is changes in the language itself that should worry us, because there is no way you can cover for these. Even here, however some changes matter much more than others:

- Primitive and user-defined operators extended to all functions. This is simply an implementation of a construct that was latent in all APLs anyway!
- Enclose. A new function . . . relatively harmless.
- Strands. A syntax change.
- Specification expressions to the left of assignment. Another syntax change.
- Bracket Axis notation "MAT+[1]VEC". A new construct! This has a defined list of target functions and operators (28 and 4 respectively).

Why worry about syntax changes? In 'old' APL you only needed two rules to comprehend a line of code:

1. Take operators before functions.
2. The right argument of any function is the result of the entire phrase to the right of it.

Note the complete incorrectness of the common myth "APL executes from right to left"!

In APL2, the effort of accommodating strands has caused some drastic changes: you now need to refer to an 8-row table of left and right binding strengths, and 13 pages of special rules for the axis notation! Of course the consequences are mostly not noticeable – things keep on working for the wrong reasons. However you occasionally trip over something like:

'ABCDE'[2]	 works as expected
2 3 5 7[2]	 gives a RANK ERROR
(2 3 5)(7[2])	 is what the interpreter reads it as!

The Original Structure of APL

Before there was any implementation at all, APL had:

Variables	3 4 5	or	'ABC'
Scalar functions	+ - *		
Operators	/ . \		
Assignment	Copula		
Punctuation	()		

This structure had the twin virtues of a simple syntax and a completely uniform extension to arrays. Since that time there have been numerous departures, and the occasional return to simplicity.

Departures

Mixed functions
(names for things you don't
really understand)

MAT[I;J;K]
MAT[I;J;K] assign X

Axis specification
(e.g. rotation)

Comments and diamonds

'FRED IS';2

Return to Simplicity

Function Rank

Pick and Merge operators

Rank operator

Left and right tacks

Format function

... but of course you can never withdraw the old versions, only make them obsolescent and hope they slowly die away.

APL2 and Sharp-APL; Major Differences

It is interesting to compare the relative weights of the documentation for the respective APLs: the IBM APL2 language reference is roughly 1.5 inches thick and comprises 658 pages; the Sharp APL dictionary is nearer 0.5 inches and a mere 72 pages. Both contain some 10% extraneous information. The two APLs have approached a superficially similar set of extensions from very different directions:

APL2

Enclosed arrays and extensions
based upon them

Strands and modified
syntax

Bracket Axis
notation

Sharp APL

Return to original simple
structure ...

– Enclose
– function rank

Operators ...

– rank
– dual

Some examples from Sharp APL are:

- the Enclose function. This is a general scalar encoding function, with an exact inverse in 'disclose'. The properties of an enclosed object are: it always has rank zero; it never matches the object it was enclosed from; it always matches the original object when disclosed. Note that the second property is not true in APL2 ... if you enclose a sentence such as "I think, therefore I am" the one-letter words "I" cannot be properly represented!
- Function Rank. A primitive like Domino always has rank-2; it applies to 2-dimensional arrays only. If it is used on higher order arrays, it simply applies to all the 2-D arrays within them. Similarly the rank of 'open ended' functions (e.g. transpose) can be restricted to certain subarrays with a 'Rank' operator.
- the Dual operator. A deceptively simple creature ... "F dual G" simply applies G then F then the inverse of G. For example "Floor dual Minus" is the same as "Ceiling"; it really comes into its own with nested structures - "F dual Disclose" will disclose an array, apply F to each element and enclose the results. It effectively makes F into a scalar function.

Some Features of APL2

Here the emphasis is on easy ways to deal with enclosed arrays. Strand notation is the major example:

I J K is the same as (encl I),(encl J),(encl K)

... but why not use a Link function such as

I link J link K (link encloses then catenates)

Stranding neatly elides one symbol, but look at the fate of mathematics which saw fit to do without the "times" symbol in expressions such as "AB + CD". The unfortunate consequence of this is that "AB" always means "A x B", so there are no multi-letter names; that's why mathematicians have to raid every known alphabet, living or dead, for yet more symbols!!

Bracket Axis is another example of an apparently simple idea which exposes a minefield of hidden rules. The problem is that it requires a specific list of applicable functions and operators, and there are numerous 'special cases' which don't work. For example "[2 3 5] MAT" - the axes must be contiguous in this case.

Summary

Arguments based on simplicity are dangerous! Often they really rest on convenience or familiarity, rather than on structure or power. In particular the power of uniformity requires a great deal of study; it takes time to notice the value of a generalised inner-product as opposed to a simple matrix multiply.

Most of [Axis] is covered naturally by Dual and Rank as applied to the existing definition of Ravel; contrast this with the APL2 documentation of Ravel, which comprises 4 pages of detail on: the depth of the result, selective specification, ravel with axis (including lamination) and the empty cases. One simple paragraph would be quite sufficient if the 'old' APL simplicity were still with us.

APL CONSULTANTS

LONDON & READING

Account Managers	(6 years+)	to	23K
Senior Consultants	(4-6 years)	to	19K
Consultants	(2-4 years)	to	16K
Junior Consultants	(1-2 years)	to	11K

Are your APL skills and potential being recognised and rewarded?

Cocking & Drury consultants have been implementing successful decision support applications for 10 years, with clients who appreciate the productivity benefits of APL.

In our professional team you will experience a range of APL environments - APL*Plus, VSAPL, APL2 and Unix, on both mainframes and micros. You will also be developing systems which, increasingly, need to interface with non-APL Information Centre products.

Of course as the leading APL consultancy, in a rapidly expanding market, we offer a rewarding career with first class benefits - profit sharing, free health insurance, and a non contributory pension.



For further details call Brian Drury on 01 493 6172

COCKING & DRURY LTD.

16 Berkeley St, London W1X 5AE

APL In Action

Manchester
England

7-11 July
1986



APL 86



The British
Computer
Society

APL86 Photographic Review



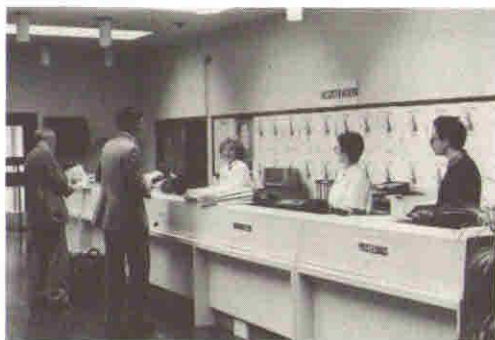
by David Ziemann

Preparing for the big day.

A view of the upper exhibition floor.



*Julia Allen of the BCS distributes APL
parasols to the delegates.*



A rare view of the APL86 registration desk during a quiet period.

Anthony Camacho introduces Ken Iverson at the opening plenary session.



Ken Iverson in the APL jungle; in his presentation of 'Core APL' Ken described the features fundamental to his vision of the core of the APL language.

Neil Mitchison and Manuel Alfonseca propose the motion 'APL is not yet suitable for AI' in one of the APL86 debates.



Neil's excellent arguments fail to convince the audience, and the motion is narrowly defeated.

Leo Groner seconds Jim Brown in successfully opposing the motion.





Romilly Cocking, Timo Seppala and Henri Brudzewski confer during the panel discussion 'APL and the Entrepreneur'. Valerie Lusmore was also on the panel, but is out of picture.



Philip Goacher chairs the panel discussion on APL implementations. Seated around Phil are James Wheeler (STSC), Luan Thomson (MicroAPL), Bob Bernecky (IPSA), John Scholes (Dyadic Systems) and Jim Brown (IBM).

Now wouldn't it be interesting to get that little group to write an APL interpreter together?

Christine Edginton surprises Dick Bowman with a cake for his birthday. Christine headed the conference organisation team from the BCS, and was largely responsible for the smooth running of APL86.



Dick Botzman seems a little surprised to have his dinner interrupted by a flower-wielding gorilla. The odd creature read a birthday poem for Dick and then presented him with an arrow from his quiver.



The real ale evening and the Pumm's party were augmented by boules and croquet on the bowling green. Naturally, the French team were unbeatable at boules, and I gave up trying to beat Adrian Smith at croquet sometime after one o'clock in the morning!

Alan Graham's contribution to APL86 extended beyond teaching us APL2. He is seen here holding up an APL symbol during a game of 'APL Name That Tune', in which two teams had to identify the symbols appropriate to popular songs.





Dick Bowman introduces Bob McLaughlin, the after-dinner speaker.

John Myrna(right) presents SIGAPL's outstanding achievement to APL award to the three people who pioneered the development of the APL standard. The recipients were Raymond Tisserand(left) who until recently convened the ISO Working Group, Alex Morrow, the editor of the standard, and Clark Wiedmann, who initiated the ANSI group's involvement.



Paul Chapman and Ed Cherlin share some APL Trivia.



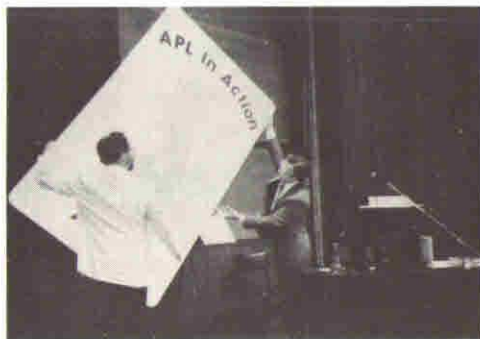
Bob McLaughlin(right), the President of the British Computer Society, proves to be a brilliant stand-up comic during his banquet speech.



Jonny Osterman gratefully receives his prize Atari computer from MicroAPL's Richard Nabavi.



Dick Bowman presents John Carpenter with a memento of APL86. John, who will be organising APL87 in Dallas next year, was heard to comment "Well, it's pretty small by Dallas standards"!



CORE APL (Opening Plenary by Ken Iverson)

report by Adrian Smith

Introduction

I can do no better than to quote verbatim Anthony Camacho's excellent preamble to Ken's opening talk:

"Ken Iverson invented APL; he is the only reason why any of us are here. I feel it would be, in a way, presumptuous for me to introduce him in the usual way. For those who don't know it, I recommend 'The Source Book of APL' - it's a wonderful piece of background.

Ken is not only the inventor, but also one of the amenders and improvers of APL, and he's not alone in that. Ken has a long thought-out set of improvements to APL, which have elegance and uniformity, and he will tell us about them today. But by asking Ken to open this conference, we're not asking you to abandon IBM, STSC, Dyalog, and other people who have improved APL in their implementations. Our reasons are different, and our reasoning is different from that of any vendor.

I hope that one of the strengths of the BAA is that it's run largely by people who use APL, rather than those who sell APL. (Although we have to acknowledge generous assistance from those who sell it.) Being independent, of course, is not particularly a virtue, unless the independence is used; and I'm using my independence to say to all the vendors of enhanced APL something which is in the interest of everyone who uses APL, whether they simply earn their living by it, or wish (as many of us do) to see it supersede conventional mathematical notation.

It's a message from everyone here (and the 20-200 thousand APL'ers who aren't here too), and the message is simple: "Don't spoil it; find a way for everyone to use the same language". In your programmes, the talk which Ken Iverson is about to give is entitled "The Split in APL". The talk he is now going to give is called 'Core APL', and the difference is that he is looking for acceptable ways for other people to accept as much as possible of the improved and revised APL he has devised.

If we can have a set of functions, written in 'Core APL' which perform as those in APL2, and a set of functions in APL2 which perform as those in 'Core APL' then we are well on the way to unifying, rather than splitting, APL.

APL is fundamentally (in my view anyway) a notation. It becomes more than that when it is implemented on a computer, because a computer can improve people's ability to think. There are two aspects to the language; its notation and its implementations. The notation is the heart of it, but as anyone will tell you, a bare APL without an editor, without a file system, without links to the operating system, without even links to machine code, is very painful to use. There's somebody in the front row who practically gave up on APL simply because of the lack of a decent editor!

APL derives its commercial strength from its implementations; if it were simply a notation and not a programming language it would never have spread as far as it has. Many features of these implementations are not what one would call elegant: □ AI, □ FMT, □ TS are very useful, but could hardly compete in a beauty contest. Somehow they never seemed threatening, because nobody fears that the nature of the language could be changed by them.

That is how we'd like to be able to see the difference between the implementations of each APL. Tomorrow we have a panel session on that very subject; in that session I would like all the speakers to remember the 200,000 APL'ers who make it possible for them to live by selling APL, the people who profoundly wish to see APL stay one language.

So I think – although what you're going to take from Ken now will be really interesting to you – that the most interesting response to this morning's talk will be from the APL vendors.

It would be very nice to see the challenge of unifying APL taken up by all of them."

Comment

Anthony is a hard man to follow! Having listened to Ken's talk both live and on tape, I feel I can only report an honest, laudable, and occasionally heroic, failure. The objective he set himself was too high, and his dislike of APL2 too strong, to allow him to reach it. I hope I am wrong in this assessment, and I would be absolutely delighted to be put right.

Over 300 people listened intently to what Ken had to say; many of those present could have a critical influence on the future of APL. I just hope they take on board his message, and will help to make his vision a practical reality.

APL86 – the product forums

All the presenters of product forums at APL86 were offered the chance to summarise their announcements for publication to a wider audience in VECTOR. The following have responded to that offer.

Butel Technology Limited

With much trepidation but with high spirits we set off on the road North to the Forbidden City, APL86. Forbidden, that is, to the likes of us mere humble software experts but not so to the APL gurus who thrive in such environments.

The reason for our expedition was to launch the English version of our APL product called MERLIN. No it's not a telephone switchboard, but a spreadsheet which exploits the advantages of APL and can be customised and expanded to match virtually any application, thus giving it truly wondrous potential.

We were sure that the ability for users to add commands, calculations, algorithms and other functions entirely to suit their own requirements, would astound and mystify all who beheld it.

As Butel is a software house specialising in System Productivity Tools for IBM Information Centres, and having only one product, we had arranged a rendezvous with an adept in the ways of APL, Bernard Legrand, who would be our guide and mentor to help us proceed in the Forbidden City without fear of discovery.

Bernard was not only the creator of MERLIN in its original French version but was also responsible for its new form in English. No one could tell you more about MERLIN's clear formats, its variable column widths, its simplistic approach to ease data entry, its note-book feature and its powerful print facilities. He could hold the attention of APL mystics with his talk of OPERGP who could consolidate calculations from several spreadsheets, of SARGRA the graphics package, of SAFI, skilled in extracting data from other sources, and of FOCUSGP who could transfer the hidden secrets of a FOCUS database into a spreadsheet.

Lest we might offend the sensibilities of the more devout we set up our wares in a room remote from the inner sanctum of the lecture halls. Though MERLIN was written to run on an IBM mainframe under VM/CMS, TSO or VSPC with compatibility to VS-APL, we had decided to demonstrate its PC version under PC DOS with APL*PLUS. To this end we installed an Olivetti M24 with colour graphics to catch the eye of the unwary, with many handouts for the curious, and for our own ease in unfamiliar surroundings, we had concealed about the room brochures on our Sysout control products, our table handling tools, and system development design aids.

We waited breathlessly for the more adventurous to stray onto our floor, attracted perhaps by the comforts of the IBM hospitality suite opposite. At last they came, first in ones, then in twos and then in groups. But did they grasp the full potential of the product, its economy, and its ease of use to the cognoscenti of APL as well as the uninitiated?

Those of you who could not find us at Manchester can look us up in the VECTOR product guide.

Mercia Software Limited

At APL86, Mercia announced the availability of multi-user/multi-tasking APL*PLUS/PC. Mercia have brought together two state-of-the-art technologies – Multilink Advanced and APL*PLUS/PC – to create an environment for multi-user APL on the IBM and compatible PCs.

The Multilink software package partitions the PC's RAM and time slices the CPU in a manner similar to a mainframe time-sharing operating system – giving up to nine terminals running DOS on an IBM PC/AT. Each terminal appears to the user to be a normal PC running DOS (with the exception that graphics are only available at the console). This enables the user to invoke most software packages written for the PC, including APL*PLUS.

To allow use of APL in true multi-user mode, Mercia have written a set of APL utilities which allow shared use of APL and native files, including component locking, inter-terminal communication and control of the multi-user environment.

Also available is LANlink – a software-driven local area network using standard serial ports on which APL users can use file locking on server APL component files in the same way as with Multilink. LANlink can also support optical fibre cabling, which can make your "local" area network two kilometres wide at about the same price as normal serial cable. For longer distances you can dial into your server machine using a modem.

Mercia also announced at APL86 the availability for the first time in the U.K. of EXEC*U*STAT – a business analysis and graphics system written in APL*PLUS/PC. EXEC*U*STAT is menu- or command-driven, and features fully integrated graphics displays and a context-related help facility. The ten chapters of the system cover a wide range of facilities: Data Management, Basic Plotting, Descriptive Methods, Estimation and Testing, Regression, Forecasting, Advanced Procedures, System Environment Control, Financial Analysis, Executive Desk.

EXEC*U*STAT will be of particular interest to people involved in business planning and analysis who are looking for a flexible, easy-to-use package with which to examine their data.

I. P. Sharp Associates

IP SHARP (IPSA) was distinctive at APL86 by its emphasis on the use of APL for large production application systems as opposed to simple personal computing. Ian Sharp, in his talk on strategic directions, reiterated IPSA's commitment to large multi-user applications (or timesharing) whether on a public bureau service or on customers' own inhouse machines. Far from dying, the demand for these timesharing systems is expanding rapidly.

IPSA provides trading support systems for the banking and energy industries which require rapid response for hundreds of users continuously signed on throughout the working day. For example, the Toronto timesharing service routinely supports over 500 simultaneous users, while one of our in-house customers is running over 1000 simultaneous APL tasks. These systems make demands that are difficult to satisfy in any language. They would be impossible in APL without the enhancements and unique environment provided with SHARP APL.

IPSA have concentrated their efforts in providing the system enhancements that make such large systems possible. However IPSA is, of course, actively interested in the latest ideas for extending APL notation. We take the conservative view that new features should require as little change to the grammar of the language as possible. Rob Hodgkinson, of IP Sharp Australia, presented a paper describing an implementation of SAPL for the HP range of minis which illustrated this.

IPSA's commitment to large scale developments does not preclude an active interest in smaller departmental and personal APL systems. Indeed, during APL86, IPSA and STSC announced a joint development to implement SHARP APL in 'C' for UNIX based machines. Such smaller machines are already being used as workstations with real-time links to larger APL systems via IPSANET. To simplify the development of these hybrid systems, IPSA have extended the idea of the shared variable to operate between any SHARP APL machines linked to IPSA's international communications network. This network shared variable processor (NSVP) is already being used to implement highly efficient and innovative international systems.

To complement SHARP APL's ability to run large complex systems IPSA announced a new product – LOGOS – aimed at the problems of developing and maintaining large complex application software. LOGOS was introduced in a paper at the conference, and in a product forum session. It provides development tools to assist programmers, in addition to creating the controls and records that are required to manage any large scale computing project. With LOGOS the APL programmer and his manager need no longer fear the approach of the system audit department.

IBM Personal Computer APL/PC Version 2.0

6391329

IBM Personal Computer APL/PC Version 2.0, is a low cost, full function APL interpreter with a high degree of VS APL compatibility. It contains a wealth of auxiliary processors for a wide range of functions and interfaces to external devices.

- Emulates 8087 or 80287 if the co-processor is not present
- RS232 support
- IEEE-488/GPIB support
- Co-operative processing via IBM 3278/9 adapter
- Interface to IBM Macro Assembler and Professional Fortran
- APL2 GRAPHPAK compatible workspace provided
- Can run DOS functions and applications under APL
- Cover Workspaces for auxiliary processors

The interpreter, workspaces and auxiliary processors are supplied on three double-sided diskettes packaged with a comprehensive manual, quick reference card and a keyboard template. The manual includes setup, installation, tutorial and reference sections.

A separate package is supplied, containing a replacement ROM for the IBM Monochrome or Colour display adapters and a ROM puller. A program to load the APL font into the IBM Enhanced Graphics Adaptor is also included.

- Available from Authorised IBM PC Dealers.



IBM (UK) International Products Ltd
West Cross House
2 West Cross Way
Brentford
Middlesex TW8 9DY

A.I. at APL86

by Neil Michison

Over the last few years "Artificial Intelligence" has become quite a buzz-phrase. And recent studies in the United States have shown that this buzz can be the noise of very large sums of money being coughed up. One good indicator of the strength of the buzz has been the creak that has recently started to accompany it, the creak of closet doors opening as big Blue allows its research workers to admit that what they've been doing for years would have been called A.I. if it had been done by anyone else. And that creak was heard loud and clear at APL86.

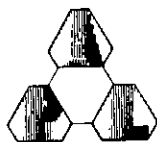
In fact AI and AI-related themes at the conference were dominated by IBM people – even although several expected participants failed to appear. And the IBMers showed a refreshing openness of mind on the question of the suitability of APL for AI work – they managed to be represented on both sides in the debate on the subject!

A working majority of the papers that *were* delivered came from Jim Brown (IBM Santa Teresa). His essential theme was that APL2 gives users all the facilities for A.I. work that LISP does, and more besides. To appreciate the force of this argument you have to realise that in the States the equation "A.I.=LISP" is more or less an article of faith. While you might not expect the Chief Architect of APL2 to be totally unbiased on a question like this, he certainly demonstrated that a great many of the traditional activities associated with A.I., such as symbolic reasoning in formal logic, can be performed very easily in APL2. He even produced a 29-line implementation of PROLOG, the Logic Programming Language, in APL2. I suspect the performance of such a PROLOG might leave something to be desired, but then IBM will always sell you a bigger computer . . . and for those concerned with performance, Manuel Alfonseca (IBM Madrid) described an auxiliary processor for the PC which performed logical deductions, comparing reasonably with PC PROLOG.

In case you needed any further convincing that APL2 can give you everything that LISP can, we were shown by Ramiro Guerreiro (IBM Brazil) that even some of the really silly features and problems of LISP can be mimicked in APL2. But he did end up by pointing out that you were much better off writing real APL2 in APL2, and not trying to pretend you were using LISP.

A paper from Gerald Sullivan and Kenneth Fordyce (IBM Poughkeepsie) described a super-spreadsheet system which used Expert System technology to manipulate the equations relating entries in the spreadsheet, so that it could work backwards or forwards to deduce the values of any variables it had not been given. And Gert Moeller gave an elegant geometric representation of predicate calculus, as an APL array with one dimension for each predicate – though again performance might be a problem if you had a modest knowledge base of 30 predicates . . .

Overall, then, there was a fairly convincing case made that APL, particularly APL2, has a place in the A.I. world. In particular Jim Brown's demystifying of the esoteric language of A.I. was very welcome. And I am sure that over the next few years even the quite extraordinarily conservative LISP community (mainly in the United States) is going to realise that A.I. cannot be defined as "writing computer programs in LISP". As we see A.I. applications going out into the commercial world, APL will be used more and more. But I still do not feel (even though I did lose the debate. . .) that APL, even APL2, is powerful enough to do everything that is needed in "real" A.I. research. Psychological modelling makes strange demands of computer systems; I really believe that I think in nested data structures with multiple entry points. . .

**APL LTD.**

17, Barton Street
Bath BA1 1HQ
Tel: 0225 62601

APL PEOPLE LIMITED

We have the widest selection of APL consultants available - at all levels right up to Project Manager! No task is too difficult for our professionals!

We are also the only registered employment bureau dealing solely in the placement of APL staff, so why not phone us today for free confidential advice about your career move!

APL SOFTWARE LIMITED

Choose from the CREAM OF APL software from around the world! Our list is increasing all the time. Or give us a ring if you think we might be interested in marketing your package!

On the Lighter Moments of APL 86

by Philip Goacher

When I was prevailed upon to produce a round-up of the stories from APL 86 and its fun and games I wondered what I was going to write about.

I need not have worried; Murphy (of the well known law) came to my aid. For example, even before the conference began, we heard that the van hired by Liz Swann to take the Cocking and Drury Exhibition equipment to Manchester arrived – only half the size and full of lemonade.

When I arrived to set up the exhibition, I looked around for the delegates' umbrellas – nowhere to be seen. At last they were found at the Securicor depot. As couriers it seems that they keep your parcels so safe that even you can't get hold of them!

In such moments is the mettle of the APL conference organiser or exhibitor truly tempered.

I would like to dedicate this to a long-suffering family who might have imagined that APL 86 was over, only to find me scribbling away whilst on holiday and asking them for impossible rhymes.

Ode to APL 86

"You are old," said the youth, "and your hair has turned white
And your body is all racked with pain,
Yet your mental gymnastics surpass even mine
For you have such a wonderful brain."

The old man looked up with a twinkling eye.
"In my youth there were many temptations
There were young girls and women, old ladies and drink
In many and strange permutations."

"These have worn out my body," the old man exclaimed,
"But my brain has been nurtured for years
And matrix inversions I'll do in my head
Any night, in the pub, for two beers."

"What's the secret old man?" said the boy, "I must know,
For I want to be clever – please tell."
"It's all in the Bible by Gilman and Rose,"
Said the man, "and it's called APL."

"Tell me more," said the boy, "of those days long ago."
"Very well," said the man, "I'll recall
What happened way back in the year eighty-six
That greatly affected us all."

"International meetings were held every year
For the APL folks of the world:
To UMIST in Manchester everyone went
Where the welcoming flags were unfurled.

The Cocking and Drury consultants we heard
Had suffered an alcohol ban
So whenever they wanted a drink, all they'd got
Was the lemonade stored in their van.

The conference opened – Ken Iverson spoke
About 'Core APL' as a tool
And for all of those present who didn't agree
He challenged them each to a dual.

There were *two* competitions the very next night,
First a quiz, then a musical game.
In a test of his knowledge (from an APL college)
A student won prizes and fame.

The musical game had two teams of renown;
Neil Mitchison led out the first.
The second was captained by Jim Brown²
But in combat it came off the worst.

The Manchester weather is fickle, we know
 So umbrellas were given 'gainst rain.
 They could also be used if the weather was hot
 For keeping the sun off the brain.

Then on Wednesday night, with Bastille day in sight,
 All the French declared war on the rest.
 'Twas their Boules team, with pride a formidable side,
 That took on and conquered our best.

Now at croquet we found many experts around
 With their mallets and hoops and skilled roquet.
 There was rumour that Adrian Smith planned to write
 A version of APL croquet.

Four teams were established: from STSC,
 I P Sharp, IBM and SigAPL
 To challenge each other in Trivial Pursuit
 And with APL questions to grapple.

At the banquet there were over five hundred guests.
 The dining room had extra staffing
 And the speaker we heard at the end of the meal
 Was the BCS boss, Bob McLaughlin.

He poked fun at the Irish, the English and Scots,
 Raquel Welch and Dick Bowman as well.
 He described Irish Dartboards (pneumatic of course)
 And his oratory cast a fine spell.

Apart from the fun and the games I've recalled
 There was much more of interest to find.
 Both exhibits and papers were excellent stuff
 With which to encourage the mind."

"Now I see," said the youth, "what it is that you did
 To develop a mind that's so sharp.
 So I'll drink lemonade and I'll learn how to play
 Concertina or flute or a harp."

"And also young lad," said the bearded old man,
 "If you want to stay youthful and handsome,
 Avoid all the women, the beer and the wine
 And talking to men like John Sansom."

Then the old man sat back to reflect on the past,
 On the APL memories he kept.
 As the boy watched he smiled with a twinkling eye,
 Put his feet up and quietly slept.

The Old Goat!



The 'IPSA' team are deep in thought as David Ziemann tries a tough APL question on them.



A tough problem for the 'STSC' team.

'Photographs by permission of Ed Cherlin, Editor APL News'

Papers I Enjoyed at APL86

by Roy Sykes, Jr.

Dave Ziemann asked me to write a review of APL86 with the above title, so I shall not dwell on other noteworthy or notworthy aspects of the conference. The APL86 committee should be congratulated on organizing and staging a well-run conference and producing an excellent programme and proceedings. In spite of culinary calamities, the hospitality was warm, the ale cool, and the content fulfilling.

Of the fifteen or so sessions and presentations I attended, all had at least some redeeming features. For some, it was merely that the speaker was awake; for others, it was a lively and engaging presentation of cogent and thoughtful ideas. I was fortunate in that my associate, Kyle, also attended most of these sessions with me. Given our propensity to avail ourselves of ale long after the formal sessions had ended, we found we needed spirited presentations just to stay awake. We were complementary: presentation style kept Kyle awake; technical content kept me awake. Unfortunately, we were sometimes fast asleep.

In his opening plenary speech 'Core APL' – misbilled as 'The Split in APL' – Ken Iverson reiterated his point of view regarding further developments in the language. Never one to be ambivalent, Ken argued forcefully for his ideas of operators and unified syntax and semantics, as contained in 'A Dictionary of the APL Language'. A relaxed and well-prepared presentation, it nonetheless disappointed many attendees who had come to hear a sage and balanced view of the differences between APL2 and Sharp APL.

Stream 2 on Monday was disappointing, mainly for lack of adequate speaker preparation. Michael Berry's presentation added nothing to that already contained in the APL86 Conference Proceedings, although his foils were of high quality. Roland Pesch (who also coauthored the Berry paper) gave a somewhat more stimulating talk on the use of 'nil' to solve problems associated with empty arrays in Sharp APL. Even in the absence of its implementation, nil rationalizes the treatment of empty frames in Sharp APL somewhat more smoothly than prototypes do in APL2. Nonetheless, IBM's more complete implementation has yet to be matched by Sharp, and it remains the best working solution.

The content of David Landaeta's paper 'a notation for manipulating arrays of operations', was controversial, particularly regarding his ideas of hyper-operators, which he calls 'elevators'. Unfortunately, he escalated his talk well beyond its 1.5-hour limit, and, having spent the first half summarizing APL2, had little time to present and clarify his central thesis.

The proponents carried the day in the debate entitled 'APL enhancements do not help the spread of APL.' The opponents (Steve Schiavo and John Myrna) failed to adequately address the central issue of the long-term impact of major enhancements, and their smokescreen was blown away by the cogent arguments of the proponents (Richard Nabavi and Valerie Lumsore). It was a sad, if surprising, result that reflected the debating skills of the advocates. The debate had its amusing moments. When the audience was invited to

voice its views, one member said "I have a question..." to which the chairman replied, "That's out of order; questions are not allowed." The attendee, not to be silenced, responded, "Well then, may I reformulate it as a statement?" eliciting a roar from the audience.

Another presentation of merit was Rob Hodgkinson's report on the notable extensions contained in Sharp APL/HP. Rob gave a provocative and engaging talk on the DEL hyper-operator and its use to construct arrays of APL procedures. His excellent preparation and fluid delivery belied little of his initial nervousness in this, his first conference presentation.

The ever-smooth Jim Brown seemed pervasive at APL86, participating in no less than four sessions dealing with his favourite APL and its applicability to logic programming, multi-user applications, and AI, too. Always well-prepared, Jim's talks are an optimistic portent of APL's future direction and usage in state-of-the-art computing.

Another group of talks, given by Clark Wiedmann, Wei-Mee Ching, and others, dealt with the exciting topic of compiling APL. Once considered a remote prospect, at least one compiler (STSC's) is now commercially available and others are in various stages of development. These papers illustrated dramatic gains in efficiency, in many cases rivaling the performance of traditionally compiled languages such as Fortran.

All in all, APL86 exhibited a bright view of APL's future. The former insularity of the APL community seems to be giving way to an aggressive pursuit of solutions to the problems holding back APL's wider acceptance. We hope that Dallas and Sydney will continue this trend.

APL86 Exhibition Report

by Martin Malin

APL in Action was the theme of the conference, and the exhibition demonstrated this in a striking way. The first thing that was immediately obvious from the exhibition was that never before has APL been in action on so many different machines in so many different forms. I counted 12 different makes (ie: manufacturers) of equipment with APL running within the exhibition. This excludes different models of machine from the same manufacturer and dial-up links to external mainframes, as well as the lengthy lists of other equipment supported displayed on several stands. APL has clearly joined the Age of the Micro in a big way! Similarly encouraging was the number of APL software products on display for use by both APL and non-APL users.

Another overall impression of the exhibition was the contrast with the conference sessions, in which APL2 and APL interpreter differences were so much to the fore. Even on the IBM stand the only mention of APL2 as such was the video featuring the familiar voice of the ubiquitous Jim Brown. Machines and operating systems supported, speed and power, software packages available and general support or ancillary functions were emphasised everywhere, but little mention of language features as such was made. Indeed the sense was of a common enthusiasm for APL in general and a determination to see it yet more widely in action, although naturally each supplier was keen to emphasise their own particular contribution and the benefits of their particular products. The dominance of micros in the exhibition was also not reflected in the number of the conference papers relating to micros, though there were some excellent contributions.

The exhibition stands gave a pleasing impression of professionalism and thorough preparation. Those on the First Floor seemed to be better off being on the same floor as all the lecture theatres, and seemed generally better frequented. Most stands, though, were busy during breaks, and had a steady flow of people even during conference proceedings. I would think most of the exhibitors should have been satisfied with the flow of people to their stands.

So what of the stands themselves – what was there to be seen? To avoid offence I will describe them in stand number order.

Stand B1 on the Ground Floor was the **bookstall**, which was useful but unexciting, having mainly the familiar titles from the APL Book Service. Orders could be given, though, so it provided a useful opportunity to buy or order all those books that you had been meaning to get for ages but had never quite got round to ordering!

Next door Stand B2 was showing the **Ampere** micro, and seemed to be relatively quiet most of the time. The Ampere is distinguished by being the first (and only??) micro with APL as the standard language, and by having not ROM or tape or floppy disk access to programs and data, but all three! One or two external 3.5in. floppy disk drives are an optional extra and microcassette drive and ROM slots are standard. It also has communications facilities with a built-in autodial speaker phone and 300 baud modem. Like many lap-tops it suffers from a hard-to-read LCD screen, particularly under the direct glare of fluorescent lights as in the exhibition. It is an individual machine, both in looks and configuration and must now face serious competition from some of the more recent PC-compatible lap-tops.

I.P. Sharp Associates had Stand B3-B4, where the theme was the Global Information Centre concept, and the emphasis on the services, facilities and packages which Sharp's offer. The latest of the software products is Logos, a powerful and interesting product for managing and administering application development and maintenance. A logical extension of the familiar function file (or "paging") concept, Logos provides a variety of tools for developing code through a full-screen editor, documenting it and storing prior versions after editing, and assembling test or production workspaces from any combination of objects on the Logos files. It keeps track of the occurrences of every object and can disseminate updates, as well as documenting them. It is of value, therefore, to both the programmer and the manager in providing assistance to the former and control for the latter through the discipline and documentation it imposes. Viewpoint is the Sharp Information Centre product and a worthy, and more attractive, competitor for IBM's Infocenter/1. The sad thing for non-Sharp users is that Sharp APL is a prerequisite for these products, and this plus the cost involved makes it a major strategic decision to make use of them. IPSA, of course, still has its extensive international network and timesharing facility, familiar packages such as Magic and Mabra and a vast range of numeric databases. There is also the Mailbox facility, and Sharp terminals were much in demand for reading the daily mail by the Sharp users present! It is a personal annoyance to me to hear so much talk of the wonders of Electronic Mail, particularly in relation to PC's, when I was using Mailbox, with telex interface some 10 years ago! Whilst the IPSA private network is a different concept from a national service like Telecom Gold, I do feel that the impression given is that electronic mail is new, and we know differently, don't we! Altogether the Sharp stand was effective in appearance and interesting in content.

The next 3 stands B5 to B7 were publicity stands for the **British APL Association**, our worthy hosts, **ACM (Association of Computing Machinery)**, our worthy sponsors, and **APL87**, our willing hosts next time in Dallas! The call for papers was going out from the APL87 stand, but although without a stand the organisers of APL88 in Sydney were doing their best, with glossy brochures and newsletter registration, to upstage even the Texans – no mean feat!

IBM occupied spacious accommodation on Stand B8-11, with the latest releases of PC APL and Application Prototyping Environment (APE) and Infocenter/1 (IC/1) the main products on display. The new release of PC APL seems to be a big improvement over the original one, and now perhaps compares with APL*PLUS/PC, but it may be too late to recover much of the market which seems to have gone largely to the latter up to now. Infocenter/1 is IBM's main Information Centre product, and is based around former products ADRS2, ADI and FPS coupled with, under APL2, the interface to SQL. With full-screen front-end encompassing all these facilities as well as graphics, it is an obvious and useful progression for former users of one or more of the previous products. It is also a good front-end for those wishing to make extensive use of the SQL interface. The list of prerequisite software includes APE, whose new release is available exclusively in APL2, the first IBM product in this area not to support VSAPL. APE is an application development tool, providing extensive facilities for screen design, graphics and query

building. For relatively simple applications it provides a good range of utilities, though at the cost, as with IC/1, of considerable software cost and processing overhead. Of minority interest, but doubtless exciting to those directly affected, IBM were also showing an implementation of Japanese APL – who says APL symbols are hard to read! Generally a smart, professional but unexciting display overall.

The final stand on the Ground Floor was **APL People** at B12. The APL People stand had on display, well, what do you expect, PEOPLE! Very friendly and helpful people, and a definite place to stop if your problem was not hardware to use or APL to run on it but fingers to go on the keyboard. As well as APL recruitment and consultancy services, APL People are now agents for Interprocess, who supply the AFM shared file software and other enhancements to VSAPL. They also have several vertical market specialist interests. So there was little to see on this stand, but if your needs were manpower or advice it could have been a valuable stop.

On the busier First Floor the first stand was **MicroAPL**. Now the full owners of the APL.68000 interpreter MicroAPL are busy porting it to a number of popular 68000 machines, including the Macintosh, Atari ST and Commodore Amiga. With APL already on the QL, it certainly won't be MicroAPL's fault if APL does not start to reach a wider 'home' audience, particularly with their enthusiastic support for the WIMP environment (apologies to the gentleman who was offended by the use of this term which he felt denigrated some excellent tools!). (Ed: Windows, Icons, Mice, Pop-up menus). The problem for them, and APL in general, must be how to create the awareness of and demand for APL in a new market area. The main strength of MicroAPL still lies, however, in the business area. As well as possible sales with the above machines, their main products are the Spectrum and new Aurora machines with APL.68000 under the Mirage operating system. The Aurora is certainly impressive in performance, with the possibility of multiple 68020 32-bit processors and a 68000 chip controlling the disk drives. MicroAPL are also branching out into consultancy and promoting the Integrated APL Strategy concept involving not just their own hardware but a variety of software modules to run with a range of configurations. Though slightly out of the mainstream of IBM mainframe or PC compatible APL implementation, MicroAPL offer an impressively functional and powerful option for many situations.

Next to MicroAPL on stands C4-C6 was **Cocking and Drury**, placing emphasis on their role as software suppliers rather than their traditional strength of consultancy. Cocking and Drury now form the sole UK distributors of STSC mainframe products, including APL*PLUS, and the European support for them. They also supply all the STSC PC products, including the new product releases which were announced at the conference. These include release 6 of APL*PLUS/PC, with the object size limitation of 64K finally removed without performance degradation for arrays less than 64K, it was stated. Also announced was release 3 of APL*PLUS/UNIX and version 2 of Statgraphics. Statgraphics is an excellent example of the software packages in APL mentioned earlier. With the boost of a highly favourable review in a major PC magazine it has become a highly visible and accepted PC product in a market much wider than just APL specialists. To the average PC

user its APL origin is irrelevant, but at the same time it provides an extra dimension for the experienced APL'er. Another interesting new release in the PC Tools range was the Spreadsheet Manager, which provides a simple APL spreadsheet environment, but more importantly can read and write Lotus files for creation or manipulation in APL*PLUS/PC. This provides valuable integration between APL and spreadsheet applications. The stand had an impressive variety of IBM compatible micro hardware, with demonstrations available of all the products being promoted. There was, of course, also details of the consultancy and training services offered by Cocking and Drury available. It was an attractive and busy stand, and well staffed with helpful and knowledgeable people.

Just opposite at C7 was **Mercia Software**, who also supply STSC micro products as well as a number of mainframe APL products. The mainframe products include Execucalc/Execuplot spreadsheet/graphics package and Infocentre, a competitor of the similarly named IBM product and Sharp's Viewpoint. All run in either VSAPL or APL*PLUS. The STSC PC products were on display, including Statgraphics and APL*PLUS/PC, plus the EXEC*U*STAT graphics package. The latter has strong similarities with Statgraphics, but with emphasis on the general business rather than highly statistical user. It has a simplified front-end, along with extras such as diary and calculator, but with some of the statistical functions missing. A good product which fills a gap between a very simple business graphics packages and an advanced statistical package like Statgraphics. Most interesting of all on their stand was the low-cost multi-user and networking software Multilink and LANlink. Multilink will allow an AT or compatible to become a multi-user machine supporting 3 users for around the cost of an additional PC, including two terminals, with a theoretical limit of 9 users. The LANlink product has similar software to allow PC's to be networked without network cards by using RS232 connections via serial ports. Both have the speed limitations one would expect from such low-cost solutions, but if they fit your particular requirements the price/performance could be very attractive. Mercia also provided details of their training and consultancy services from their small but business-like stand. They also provided a valuable service at the end of the week in matching badge numbers for the STSC competition!

Next in line was the **Dyadic Systems** stand C8-C9 featuring Dyalog APL. Perhaps the most significant comment on Dyalog was the popularity of the kit on this stand for solving the conference competitions! It may not be what it was there for, but it must say something about both the interpreter and the hardware on display that it was so heavily used. Dyalog is without doubt the closest to APL2 of other "enhanced" APL's, which must be added attraction for users into Unix in a big way. As well as a number of APL2 features, Dyalog has some interesting aspects of its own, such as "external variables". There was also a good deal of technical interest, with Dyalog running on the IBM 6150 (PC/RT) RISC machine in the hospitality suite upstairs. This machine could be the one to bring wider acceptance for both Unix and Dyalog APL, bearing the magic blue letters as it does. On the stand the Sun micro was in use, which both Dyadic and STSC spoke very highly of for sheer performance. There was also the Dyalog APL co-processor, based on the Opus532 Personal Mainframe board which has a 32-bit NS32000 processor. It provides a full Unix V/Dyalog implementation on an expansion board for IBM PC's and compatibles, with PC-

DOS/MS-DOS interface and again opens a potentially fruitful market. For sheer technical interest this good-looking stand was hard to beat, though Dyalog's fortunes are clearly intimately tied up with those of Unix in general and thus currently, in the UK at least, are linked to something of a specialist market.

Even more specialist was the APL Machine from **Analogic** on their stand. Featuring enormous power from its array processing architecture, it has considerable attraction for CPU intensive applications. Unfortunately the stand was usually unattended in the latter part of the week, and I was unable to see the machine in action. It is clearly of interest only for specific applications areas, but within those areas must be hard to match for performance.

The centre of the floor was taken up by **STSC** with stands C11-C20. They were showing all of the products and new releases announced as mentioned above, and had additional booths for their UK and French distributors Cocking and Drury and Uniware. They also had APL*PLUS on the Macintosh, in competition with MicroAPL. APL*PLUS/UNIX was running on a Sun micro, as was Dyalog on the Dyadic stand, and the seriousness with which STSC take the competition from Dyalog in the Unix area can be judged by their attempt to tempt users of "other" Unix APLs with a 25% rebate offer for "trade-ins". MicroAPL also offer APL under Unix, of course, but their emphasis is more on the Mirage or proprietary operating systems. Most encouraging for the spread of APL in general was evidence of STSC's vigorous activity in the educational area and their software publishing scheme. The stand was smart and spacious, but most of the products have been described elsewhere so there is not a lot to say about it.

Although not part of the the exhibition on the two main floors, mention must also be made of the Merlin spreadsheet package on display from **Butel** on the second floor. This is a spreadsheet written in APL to be run in VSAPL on a mainframe or on a PC using a run-time version of APL*PLUS/PC. Whilst having a rather different appearance to the traditional spreadsheet packages it is nevertheless a genuine competitor for them in terms of function and ease of use even for a non-APL'er. As with other APL packages the possibility of calling user-defined functions further increases its flexibility in the hands of someone who does know APL. The screen handling is cleverly structured such that it is virtually identical on the PC to the mainframe without seeming to sacrifice too much of the superior screen handling generally available on the micro. The reporting parameters allow much better reports to be created than from most spreadsheets. Like Statgraphics in the graphics area this product is a worthy competitor for established packages in its own right, and for APL'ers has the added attraction of access to APL. The mainframe version also has now or under development links to a number of the most popular database products. It was a shame that this demonstration was hidden away upstairs and not classified as part of the exhibition proper.

So there you have a quick tour of the APL86 exhibition. I hope it brings back (accurate) memories for those who were there and gives those who weren't an idea of what they missed!.

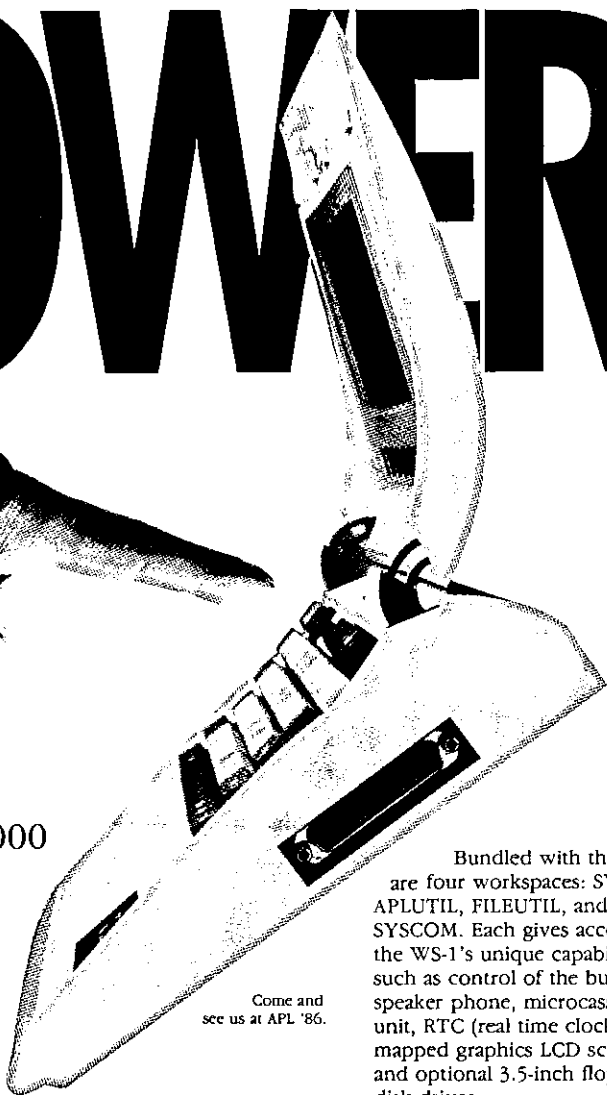
POWER



Enhancing APL 68000

An intriguing computer for an exciting language—the WS-1 and APL 68000. At last the APL programmer can have portability without sacrificing power or capability. Dodge the queue waiting for time on the main-frame and discover the sudden freedom of being able to try out programs anytime, anywhere.

The APL 68000 interpreter is implemented in 86KB of ROM, running under a multi-user, multi-tasking operating system called BIG. DOS. Speed is the essence of APL programming, and now the WS-1 makes development even faster.



Come and
see us at APL '86.

APL 68000 on the WS-1 has attractive enhancements such as a powerful component file system, QUAD. FMT function for alpha report formatting, QUAD. CC function for full-screen control, and extended error trapping facilities.

Bundled with the WS-1 are four workspaces: SYSFNS, APLUTIL, FILEUTIL, and SYSCOM. Each gives access to the WS-1's unique capabilities such as control of the built-in speaker phone, microcassette unit, RTC (real time clock), bit-mapped graphics LCD screen, and optional 3.5-inch floppy disk drives.

Compress these capabilities into a sleek footprint measuring less than 13 inches by 11 inches, and you have the ultimate definition of power.

Ampère

FOR DISTRIBUTORSHIP INFORMATION AND PRODUCT DETAILS PLEASE CONTACT:

Ampère
INCORPORATED

Asahi Bldg., 7-5-20 Nishi-Shinjuku, Shinjuku-ku, Tokyo, Japan. Phone: 03-365-0825.

Telefax: 03-365-0999. Telex: J33101 AMPERE. IP Sharp Mail Box Code AMP (Group Code APLWS).

Pick of the Week

by Adrian Smith

Introduction

This little rag-bag of odds and ends is my attempt to convey some of the excitement and sheer fun that we all had at APL-86. I shall draw a decent veil over the exploding Real Ale, the midnight Croquet, and the extraordinary level of design skill applied to the paper aeroplanes at the conference dinner. Suffice it to say that the general intellectual standard at APL86 made your average gathering of academics look like the autumn meeting of the Beano Readers Association.

Many of the papers were tough going; at the end you thought to yourself "I had to work pretty hard, but I think I got some of that"; then you realised that everyone else was thinking much the same! It made me very proud to be a member of such company.

Now for some of the lighter moments – first a selection of my favourite quotes (I'm afraid that many of these are on tape, so the perpetrators would do well to think twice before they commence legal proceedings!):

From the opening plenary:

"I'm grateful they didn't call it APL"

Ken Iverson on APL2.

From 'APL2 by Example':

"The trouble with APL2 is that you take 10 minutes writing the code, and then spend the rest of the afternoon telling your colleagues about it"

"CPU time – if you don't use it now it's gone for ever"

Alan Graham

From the panel on implementations of new APLs:

"We're building interfaces to IBM acronyms that I really don't know what they are"

"We've got to be able to drive mice, and all those other little critturs that run around."

Bob Bernecky.

"Brackets are the cockroaches of APL; have some respect for them, because they'll probably outlive mankind."

Adin Falkoff (under extreme provocation).

From the first debate (on the effect of enhancements on the spread of APL):

"The 'old' APL is the most fantastic, amazing product the computing world hasn't seen"

"APL2 was a conspiracy to stop the spread of APL(1)"

"We're not getting new customers from APL2 – just confusing the old ones"

Richard Nabavi.

From the panel on APL thinking:

"Encode is something you use when you give change"

Norman Thomson on APL thinking.

"If you disagree with Alan, it's like standing at the wrong end of a cannon"

Norman Thomson on Alan Graham

Mexican Snacks at Dyalog APL

There was almost a surfeit of APL2 at this conference. I don't know quite why, but APL2 bothers me a lot; the odd thing is that Dyalog APL looks far more attractive, and I can't for the life of me see why! Having greatly enjoyed Alan Graham's "APL2 by Example", I had still come away with no lasting impression other than that of his extreme cleverness and showmanship. What I certainly did *not* get was a clear vision of where I could really use APL2 to get my job done better.

That basically means using APL to develop and (much more important) maintain and enhance complex screen-based systems over a wide variety of applications. No-one has yet had to *maintain* a system in APL2; if they are even half as clever as Alan, I think they could be in dead trouble! Which brings me by a round-about way to the point; the only person at the entire conference who significantly helped me along the APL2 road was John Scholes at Dyalog's unscheduled "Fringe".

Of course it was marvellous to escape from conference food, if only for an hour or so, but I cannot credit this as the sole explanation. Much more likely is the fact that Dyalog has been developed by a small team (essentially two people) who actually use APL. Contrast this with APL2, which has been brought about through the enforced union of two APL development groups with very different views on the future of the language. John Scholes could talk convincingly about the value of "Compose" because he knew why he wanted it, and because he was the one who wrote it!

He could demonstrate the power of ideas like "Function assignment":

```
TOTAL <- +/  
TOTAL 1 2 3 4
```

... because he was the one who had wanted it enough to go to the trouble of inventing it! That is the same kind of motivation that Ken Iverson felt when he first sketched out the ideas of APL; I think that is probably why Dyalog APL feels 'right', and why I so much enjoyed those early-evening talks.

The Debates

We had two 'formal' debates at APL-86; the first acceded to the view that enhancements to APL do nothing to help the spread of the language, and the second threw out the conjecture that APL is not (yet) a suitable language for Artificial Intelligence. For me, these ranked alongside the panel discussions as events where the rank and file of the APL world could make its views known. There is always a danger at such a prestigious international gathering that the platform will be dominated by the same circuit of 'names' year after year, and we ordinary folk will never get the chance to say our bit.

The glory of the debating format is that it positively encourages the outrageous, the controversial, the hyperbolic. You can jump up and say something totally ridiculous, and as long as it 'scores' off someone you will very likely get away with it. In the first debate, the floor (and the principals) were to some extent feeling their way; by the end of the second it was beginning to warm up nicely. I hope the committee of APL-87 will take note, and somehow find room for this (very British) tradition in their program. Perhaps a 'Balloon Debate' on APL enhancements would be a revealing occasion?

Postscript

If you can stand the pace, stay awake, hold your own in the bar, and (at times) work damned hard, APL conferences are a lot of fun. This was my first, I doubt if I will make Dallas, but of course it's always in Europe in the even years, isn't it? See you all in '88.

APL86 Fun and Games

by Aelred Tobin

After Monday dinner the hall settled down in anticipation of Dave Ziemann's games evening.

An APL multiple-choice knockout quiz was first on the agenda – the APL Superquiz. A question was posed, and 4 possible answers given. We had to indicate our choice of answer by standing by one of the appropriately labelled pillars in the hall. Only those with the correct answer being allowed to continue to the next question.

The questions started off 'straight' eg. "Is the result of executing the matrix inverse of 2 a RANK ERROR, 0.5, 1 or 2?" (0.5) and "How many characters are therein the shortest possible VS APL expression producing an INDEX ERROR $\geq$ 2,3,4 or 5?" (5). They soon degenerated to the likes of "Does the 'E' in Kenneth E Iverson stand for Eugene, Eustace, Eric or Edward?" (Eugene), and "Was the song APL Blossom Time first performed at APL81, APL82, APL83 or APL85?" (APL81).

It didn't take long to reduce the assembled to a few inseparable stalwarts following the first person who looked like they actually knew the answer. (I wonder what would have happened if they chose the wrong answer?). A few cunning questions finally broke their resolve however, and the IPSA and Cocking and Drury donated prizes of a Sharp APL PC interpreter and a Pocket APL interpreter were won by Fergal Golding and Jim Brown – two APL86 stewards with little knowledge of APL. So much for the experts!

Dave then added a little spice to the next days debate by calling upon the principal protagonists: Jim Brown (the APL2 one) and Neil Mitchison, to each captain a team for 'APL Name That Tune'. Each team was supplied with an identical set of APL symbols on pieces of cardboard. A snatch of music was played – something with a reference to APL in the title or artiste's name, and the first team to hold aloft the appropriate APL symbol gained the points.

It all appeared simple until, when faced with a right arrow symbol for instance, you realise that it could represent TAKE, DROP, ARROW, GOTO, BRANCH, ASSIGN, IS, GETS or FIRST. Not only had the contestants to listen for many key words, but horrible puns were also allowed, eg. Cracklin' Rose by Neil Diamond (RHO and DIAMOND). Overstrikes were also permitted, so both quad and divide had to be held up for the Fats Domino song!

The choice of music was deliberately catholic eg 'Bolero' by Ravel (,), 'Della and the Dealer' by Hoyt Axton (DEL, AND and QUESTION MARK) and for a truly international flavour 'Waltzing Mathilda' (TILDE) and 'Wunderbar' (W).

Neil's team seemed to always have the edge, though Jim Brown never gave up – at one stage holding up all their cards at once in a sort of APL Name That Tune system crash. Neil's team never lost the lead, but graciously shared the prize of 3 bottles of champagne (donated by Inner Product Ltd), with the losing team. One-nil to Neil, but who would win the AI debate?

The organised hilarity at an end, 'twas time to repair to the bar.

Report on the Plenary Panel Session at APL 86

by Anthony Camacho

"The Differences Between Implementations of Enhanced APLs"

Jim Brown was given the largest round of applause of the whole conference when he said "I want to go on record today as saying that I'm also convinced that Ken [Iverson] is correct in his direction for extending APL". When the applause died down he continued by asking us not to jump to conclusions: there is more than one correct way to extend APL "and all it means is that different reasonable people that have different objectives in mind can come to different correct conclusions . . .".

The full transcript of the session took seventeen pages: I will therefore summarise drastically to leave room in Vector for other things. Please forgive me if I leave your question out or misrepresent what you said: I'm sure the editor will be glad to publish your comments.

Jim Brown offered to talk about nested arrays (and the reason why an enclosed scalar was still a scalar), vector notation (and to maintain that there is no function left out between the elements of it), and to explain brackets (which people complain are neither function nor array nor operator).

The fundamental change that APL2 makes is to extend greatly the extent to which the shape of the data can control the work that functions and derived functions will do. There is no need to learn all the rules. Nearly always the simple view of the way things will work is correct; for the occasions when it isn't it's probably simpler to insert brackets than to go into the theory of the syntax which explains why. But there is a unified theory, thanks to Phil Benkard, and it is correct. If the brackets are occasionally redundant, who cares?

John Scholes said that Dyadic's objective was to enhance APL to make it more useful for writing commercial systems by adding error trapping, component files, commercial format, and a wide variety of system functions. Unix was chosen as the operating system because they foresaw its growth in popularity. Dyalog APL has floating arrays like APL2 and is written in C for portability. It has an excellent UNIX interface so programs written in C can be taken into an APL workspace and called just as if they were locked functions. All the features of UNIX are available from within APL. The design of Dyalog APL closely follows STSC's NARS and John gave credit to Bob Smith "Most of the ideas in there are Bob's".

James Wheeler said that the correct origin of NARS is Nested Arrays Research System. From 1979 it was used as a test bed for new language design ideas. In 1982 STSC had chosen the enhancements that had proved most effective in the eyes of the APL community and these are now part of the APL*PLUS/1000 product. STSC also realised that APL needed to be enhanced in other ways to make it easy to use; it needed better editors a user interface with screen control and it needed ways to overcome speed limitations. These are the aims which have guided the people producing implementations on micros and the APL compiler.

James felt that the great debate about axiom systems is over and that the positions taken by the major vendors are now fixed. The set of all possible arrays in APL*PLUS/1000, Dyalog and APL2 is now the same. STSC do not yet have defined operators, selective specification and a few of the less frequently used new primitives. The great range of system functions has made shared variables unnecessary and there are some features that STSC have which APL2 lacks. One he recommended for Jim Brown's attention is the partitioned enclose which has proved very popular with the users. There are some differences in the way bracket indexing works that ultimately he would need to settle with Jim. In short there are some differences still but STSC is converging with APL2.

Luan Thompson explained that although he has been working in machine code on the Motorola 68000 for some years and with APL for four years, MicroAPL only took control over the 68000 interpreter six months ago and have spent the time since mainly in optimisations for speed. As the interpreter is written in machine code its natural targets among small machines are the QL, Atari, Macintosh and Amiga. On enhancements MicroAPL feel that there is already a dominant vendor in the APL market and that they are not going to try to buck the trend. There are already a few features such as dyadic grade in the interpreter and there are hidden features in it which they believe will be able to handle nested arrays, although these are not at present available to the user. Luan said he believed that the enhancements to user and system interfaces would probably be more important than enhancements to the language. APL competes in a market where BASIC, Pascal, C, and FORTRAN all have multi-window WIMPS (window, ikon, mouse and pop-up menu) interfaces and better editing and error control. MicroAPL will enable programmers to write WIMPS applications which nobody can tell use APL. The main problem is that these features are different on every machine which makes it a lot of work, but MicroAPL think it is worth while to make life easier for the programmer.

Bob Bernecky said that the Sharp philosophy is to make enhancements simple, consistent and general. The more complexity the harder it is to teach and learn and use (and complexity also affects the implementations). Bob too would like to see APL2 and Sharp APL converge and suggested that STSC and IBM might like to consider 'link' and 'rank' which would help them converge.

Link does the same as strand notation (Bob asserted that strand notation complicates syntax analysis) but because it is a function you can apply an operator to it. With the rank operator you can link say matrices or vectors from two arrays. There's no obvious way to do that in APL2 because there's no function for an operator to work on. [Alan Graham interjected that one of his idioms did the trick.]

The benefit of the rank operator is apparent when you look at the bracket axis notation it replaces. In APL2 bracket axis gives rise to many special cases; the definition of ravel takes several pages because of the different ways that things in brackets can affect it. When Ken Iverson and Arthur Whitney proposed the rank operator Sharp realised that brackets had been inconsistent from the beginning; caentate laminate, rotate, reduction all are affected by brackets in different ways and each has to be learned separately. The rank operator

replaces the bracket axis notation and applies identically to all functions, even user defined functions. Furthermore if the rank operator is made more efficient it will make every function that uses it more efficient, whereas the brackets would have to be improved one use with one function at a time.

Consistency brings benefits; Sharp had a minor but uninteresting enhancement to take or drop – if the argument shape was a scalar or one element vector and less than the rank of the right argument then the result has a shape of the same rank, defined by the left argument catenated onto the remaining shape of the right argument. So if $EH > 2\ 3\ 4\ 5$ of something then the shape of a $6 \wedge EH$ is $6\ 3\ 4\ 5$. Combined with the rank operator this can be made to yield results of shape $2\ 6\ 4\ 5$, $2\ 3\ 6\ 5$ and $2\ 3\ 4\ 6$ as well. So you start to see some synergy between the enhancements and the ISO APL.

Sharp is trying to put APL back into the mainstream of data processing and this won't be done by language enhancement but by improving the environment. Sharp are providing a consistent interface which will allow any language to drive APL or an APL function to drive programs written in any language. They are also building special purpose interfaces to all manner of peripheral equipment and a network shared variable processor which enables a user to share a variable with a user in another city or with a PC and it still looks just like a variable in the program.

A lot of Sharp's work has been to improve large applications with hundreds of users so that the work bring benefit to many people. Now they have signed a co-operation agreement with STSC and hope that this will enable them to do more in the micro field. Bob would like to see $\square FX < \square \text{prime} < \square CR < \langle \text{function name} \rangle$ allowing you to edit a function using the operating system's full screen editing and agreed that the latest things like WIMPS had to be available with microcomputer APLs.

Philip Goacher then called for questions; below is a brief summary of the main discussion.

Alan Graham endorsed the rank operator: when he first heard of it he put it into his workspace as a defined operator and nowadays even has it defined with its own symbol so that he doesn't have to begin every new workspace with $\rangle COPY UTILITIES RANK$. He has set his students the exercise of writing all the Iverson operators in APL2 and although there are one or two tricky ones he believes he has correct versions of all of them. Anthony Camacho asked whether he would circulate his code, saying "Send it to Vector and we'll publish it" and Alan replied "I'll do that". He also said that he very much regretted brackets and wished that the fathers of APL had never put them in. Bob interjected that the rank operator made brackets redundant.

This exchange stimulated Adin Falkoff to come to the stage to defend brackets. While he was trying to get enough ink out of a dead pen to put his name on the overhead projector he was asked whether he originated the acronym APL. He confirmed that it was his idea. Adin maintains that there is a good use for bracket-semicolon notation and that is to pass multiple arguments explicitly. He too endorses the rank operator and doesn't wish to defend every use of brackets. By passing arguments explicitly you avoid having to box things together

which have a merely contingent association and also avoid having to unpack them inside the function before using them. You can also leave out an argument between two semicolons with the same meaning as in ambivalent functions. He seriously recommended this extension as consistent with all implementations.

Jon McGrew, speaking personally and not for IBM, thought that we were now stuck with brackets, which people find convenient and easy to learn. Jim Lucas questioned the notion that the divergence in APL was a bad thing; progress is achieved by experimentation and competition and perhaps APL will get superseded by a language which will not have to carry on old mistakes as APL has to.

Neil Mitchison disagreed with Jim Lucas, and felt pessimistic about APL's chances of wide acceptance if it continued to diverge. He asked what the panel thought should be the role of the ISO in this. Should they create a compromise standard which went over to one side for some things and to the other for others? Jim Brown said that he felt that the Standards Committee were right not to put files into the standard because there wasn't enough agreement about their form and he would like to see the standards committee sticking to the description of agreed usage. There are wide areas of this; everyone has the same definition of dyadic upgrade and of complex numbers and of replicate. Neil was not satisfied with this and asked what should be done about features where there was no agreement and the versions were incompatible. Jim's answer in effect was that those features should be left out of the standard.

Anthony Camacho asked about the size of the interpreters, saying that he wanted to run APL on microcomputers small enough to take into school or give to children to use at home and that the size of the interpreter determined how far down the market it could spread. James Wheeler said that the size depended on the machine. The STSC interpreter is written in C and when compiled varies from about 200,000 bytes on a Vax to about 400,000 bytes on the PC; unfortunately larger on the machine where less space can be afforded. He looked forward to cheaper machines with larger memories which would allow enhanced APLs to be run in schools. Luan Thompson said MicroAPL is about 95,000 bytes varying by plus or minus 10,000 bytes according to machine, and that there will be four megabyte memories available before you can blink. Bob Bernecky said that the Sharp PC interpreter was essentially the same as the mainframe version but even with all the bells and whistles it only took 400,000 bytes. He reckoned that each 30 months the capacity of chips was quadrupling and agreed that long term there was no problem about memory size. When prodded by Adin Falkoff he said that the naked interpreter was about 250,000 bytes. John Scholes said that Dyalog APL is also written in C and also varies. Originally (on the Z8000) it was 200,000 bytes with all the frills then available. The decision on enhancements is always to take more space to gain speed and the current version, on say a Stride 68000 based machine, is 400,000 bytes. Jim Brown asked David Selby what the size of the PC APL was and David said 70,000 plus the auxiliary processors. A quite adequate set can be put together and run in a 256K machine. David added that he didn't agree with the 'memory will be cheap' argument, saying that we should not expect the user to pour hardware in.

Chris Brady wanted APL to be less greedy of mainframe capacity: his employer seemed likely to cut its APL workforce because of this effect. He also asked when he could see code, variables, the stack and the state of the filing system all on the screen at once. Luan Thompson replied "As soon as I've finished the version for the Atari: that is exactly what I'm working on".

Linda Alvord returned to the problem of getting APL into schools. Her main point was that education authorities and heads of school departments who might consider APL will be put off by the choice of versions since they did not have either an International Standard or any agreed consensus to guide them. If you go to a supplier and say 'I want to buy a standard APL' nobody has one for sale. Everybody tries to sell you the special advantages of their own version and the consequence is that people decide to wait until the question is settled by another mechanism. She felt that there should be a standard APL which everyone could buy safely and that the enhancements were not of much value in secondary schools. These comments brought a big round of applause. Roy Sykes called out that the ISO Standard version should be available for forty five bucks and that was obviously accepted by general acclamation too.

Comment by reporter:

Philip Benkard called attention to the great applause for Jim Brown's opening remarks and I would like to do so again.

It shows the strength of feeling in the APL community that regrets the differences between enhanced APLs and wishes to see all APLs converge.

APL used to have the advantage that you could write and test programs on a microcomputer, benefiting from micro editor and response time, and then download the program to a mainframe. Nowadays it would take great self discipline to stick to ISO APL and even that would not work with APL2. And it would take great dedication and unusual opportunity to get to be at home in all versions of APL.

This diversity is not in the APL user or programmers interest. It arises because each vendor wants to offer something better than the others and which, if possible, commits their customer to them.

I believe a powerful alliance of users could and should apply pressure on vendors to converge and perhaps this could be done by persuading the ISO to set a standard for enhanced APL which does exactly what Neil Mitchison suggests above.

Anthony Camacho
2 August 1986

APL86 Quotes

compiled by Dave Ziemann

There follows a short collection of interesting quotes heard at APL86. My apologies to those concerned if any distortions have crept in – they are purely unintentional.

"If your code has more pepper than your scrambled eggs you should rethink it" — Alan Graham, talking about APL2's 'each' operator.

"Plus reduction is the most common use of reduction...for less imaginative people" — Ken Iverson.

"I was fighting IBM on these matters for twenty years before I started doing it from the outside" — Ken Iverson.

"I can't remember if it's encode or decode....." — APL86 speaker, followed by this comment from the audience: "Neither can we..."

"There's only one thing worse than an APL bigot – an APL bigot who works in zero-origin" — Alan Graham, prior to displaying his zero-origin APL2 code.

"I don't care how much CPU it takes – if you don't use the CPU time now it's gone forever" — Alan Graham.

"Scientists have discovered the missing link between Neanderthal man and intelligent man – it's us" — Jim Brown, in his lecture on APL2 and A.I.

"If this APL thing is so good why isn't it in my car?" — Roy Sykes's mother, as quoted by Roy in his closing speech.



Katy Webster poses Timo Seppala a tough question during a game of APL Trivia.

'Photograph by permission of Ed Cherlin, Editor APL News'

Thoughts on APL design

by Bob Pullman

There are many facets to the design of an application. As far as APL is concerned there have been many papers and even texts that concern system design. My feeling is that although some of these works are excellent, there has not been enough published on design, and that most of the existing works are hard to find.

I think that the novice applications programmer is particularly hard up in this regard. Even someone who has a couple of years' experience of APL programming, in university perhaps, is ill-prepared to tackle an applications project. I can recall, for instance, a programmer who asked me plaintively:

"How do I write an application? What do I do to get started?"

Perhaps this fellow was not the self-starter type, but he was certainly bright (he is in fact a lawyer now) and had both a computer science degree and a solid understanding of APL. He had read several texts on APL (Gilman & Rose, Peelle, and others) and was comfortable with NARS (APL2). Just the same he was at a complete loss for what to do.

This predicament, given his background, supports the assertion that APL literature is inadequate, that it is missing critical topics in the general area of applications programming. There is a wealth of papers on particular applications, but these are rarely of much use. Some present algorithms but very, very few reflect the sequence of steps in which the application was actually done.

There is a school of thought which believes that there is no substitute for experience. This school might be called the "experientialists". The radical wing of this school believes that one should just plunge in, experiment and learn by trial and error. Some believe that this is the way to learn APL to begin with. I would say that it is suitable for a rather small minority, and even for such types it is not necessarily the most expedient approach. One does not learn physics by experiment alone, so why should APL be any different?

On the other hand, experience is the best teacher. There is nothing that can substitute for real-world problems. One only learns by facing new problems. My point is that the learning time could be shortened if the novice had supportive material to study and refer to during the learning phase.

The questions are what sorts of material should be used, and for how long. It's easier to answer the second. Learning time is infinite, although the percentage of learning time in a project will eventually become quite small. The answer to the first question is more elaborate and I cannot claim in any sense to give it fully. I only hope to open up what has been neglected by the literature so far. The most suitable medium would be an anthology on design, combining several different approaches, even contradictory ones.

If such an anthology were published, it would serve a very useful purpose. It would make APL applications programming more accessible. It might blur the distinction between end-user and applications programmer. It certainly would help to increase the popularity of APL, even perhaps to make APL "the people's choice".

Much has been said and done about the problem of making APL more popular. I would guess that there have already been some who have cited education as one of the ways to increase APL's popularity. More often, however, the focus has been on enhancements to APL, ways of bringing APL to the people. Considering that many APLers fell in love with APL\360, I'm sceptical that enhancements have much essential weight in the matter. My general conclusion is that rather than bringing APL to the people's level, we need to use the proper educational tools to bring the people to APL. One way to start would be to collect, review and republish the literature on design. To proceed from there one could encourage more people to contribute and to expand the anthology once it has been collected.

I would suggest, as a very first step, that an index of such publications from VECTOR, Quote Quad, APL News, or elsewhere, be developed. VECTOR would seem to be the most appropriate collation point, and the most effective place to disseminate the index.

Shortened Transcript of Roy Sykes' Closing Plenary Address at APL 86

by Anthony Camacho

Transcriber's note: The full transcript of Roy's speech was thirteen closely written pages (about six full pages of Vector). This abbreviated version omits the jokes and the politenesses to many contributors to the success of APL 86 and tries to do justice to Roy's serious theme. Words which are verbatim are in quotations.

After a few introductory jokes Roy asked the serious question why APL is not dominant. "It has been estimated that APL accounts for between two and six percent of programs written in the world today." He offered the following main reasons why we had not yet persuaded the rest of the world to follow us.

Inertia "Organisations have little reason to discard old yet operational software and are not prepared to retrain entrenched hordes of COBOL programmers. The twelve figure investment in computer software and standards [which] considerably exceeds the investment in computer hardware. . . " is not on our side.

Cost "The memory required by APL has only recently become . . . affordable . . . the speed differential between interpreted APL and compiled scalar languages remains. Furthermore APL implementations are still priced at a premium."

Insularity APL took longer to gain links to computer peripherals, files and databases than the other languages. This has delayed our integration with the rest of computing. "APL programmers have ignored the garden in favour of a single rose bush."

"There are hosts of other reasons for the orbit in which we find ourselves with our language. We may think we have *charm* but the rest of the world thinks we have *strangeness*. We excel in parallel but collide sequentially. The stable molecules we build from our periodic table are seen by others as unstable *gluons*. Overall APL and its advocates have a reputation for inhabiting an esoteric world of strange symbols, peculiar terms, dense programs and evangelical fervour."

After congratulating the organisers (but not on the food – "lunch tickets anyone?") Roy asked whether APL would ever come to fruition.

"I see the seas parting and the promised land beckoning Our technology has burst explosively into the mainstream of computing. In less than a decade we have seen the integration of APL with scores of other languages and packages." APL can now use any equipment, read any file and its speed disadvantage is gradually being eroded. The new data structures, primitives and operators "expand the expressive power of APL explosively yet they retain the fundamental flavour of the language."

Although Roy voted for the motion that APL enhancements do not help the spread of APL (which was carried) he voted against the proposition that APL was not yet suitable for AI (which was lost). "I wonder if the debate had been based on APL 1 (in Jim Brown's

parlance) would this motion have been defeated? Is VS APL sufficient for the task of AI? I think not." So he revised his opinion and now believes that enhancements do help the spread of APL and implied they will do so increasingly.

"... the most important technological explosion for APL is the most obvious. The advent of the personal computer and the extraordinary implementations of APL based on it ... If APL\360 was the first revolution in APL then APL*PLUS/PC is the second."

The biggest factor in the future of APL is education. "IBM has donated tens of thousands of PCs to universities. Thousands are now running APL ... Fertile minds are being seeded with APL."

"It seems to me that we have a more diverse and committed commercial sector devoted to APL than ever before. The UK now has an independent base of APL professionals approaching the size of that in the entire United States at the late 1970s peak of timesharing."

What can we do now? Roy's answers were; correct the old misunderstandings about APL; show it to people who should appreciate it; use it to write applications; *think big* – don't sell just to programmers which is a small market compared with that of users (the largest used APL system is ADRS); use APL for prototypes (slogan: 'Get it up fast with APL'). And he gave an example: he had produced an address book and telephone control system in about five days work, superior for his purposes to any of the packages on offer. "The fact is: APL can produce good products. Why don't we produce them? Why don't we market them? APL will get us there better and faster and the architecture of future machines favours the parallel structure of our language."

We need to train APL programmers properly. Too much APL is done by amateurs and users who are not trained and their mistakes bring APL a bad reputation. There are still under a thousand professional APL programmers. Perhaps most of all we need to spread APL downwards in the educational world from universities to schools.

Roy's closing message was that the great period of enhancements to APL is over. "The APL that we are now using is the one that we will be developing products on now and for the foreseeable future. Let us resolve to exploit APL: to expand its usage and application far beyond its current scope. Our livelihoods depend on it. Our dreams will be satisfied by nothing less. Remember APL is not a programming language. It's a way of life."

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Stop Press

The Amstrad PC1512

I have just had a *very* brief encounter with this machine. It looks and feels rather smart, although the screen resolution (in monochrome white) is only as good as the (standard) IBM colour monitor. However it does make quite sensible use of grey to imitate colour effects.

It ran APL*PLUS/PC with no problems, including screen hardcopy on the Amstrad printer. I am told that you can swop the character ROM, but obviously I was unable to check this! It benchmarks at 2.4 times the speed of a standard PC (i.e. a little below an AT), running the 'FN 1000' test in 15 sec.

Further details in next issue.

Adrian Smith

SPECIAL OFFER (while they last) APL86 White Plastic Wallets (with red Logo)

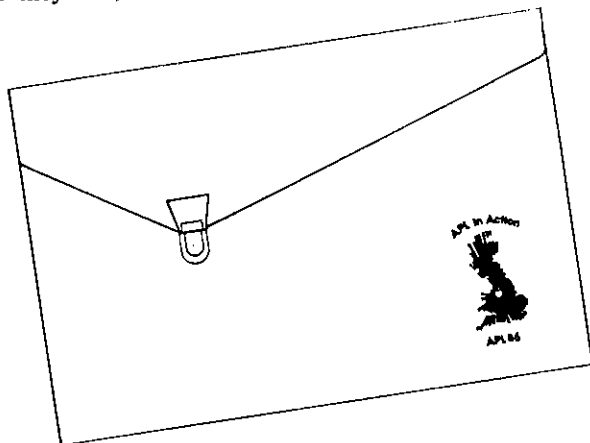
packed with

APL86
Proceedings
APL86
Tutorials
and
a Pack of
APL Trivia Cards

order from

APL Book Service
PO Box 1000
St Albans AL3 6NE

Price £22.50 plus £5
postage and packing.



Technical Editorial

BAA Public Domain Software Library

by Dave Ziemann and Jonathan Barman

At APL86 the BAA proudly announced their latest project, the Public Domain Software Library. We hope that the library will become the definitive source for public domain software of interest to APL programmers. The BAA have not embarked upon a project of this kind before, and so for the first six months we will be offering a restricted service; only software exchangeable on DOS-format disks will be offered, and we will not undertake the distribution of any paper documentation. Although the software will be free, the disk on to which it will be copied and the postage has to be paid for. Charges are expected to be quite low as it will be possible to purchase the disks in bulk. At this early stage, the software distribution procedure has not yet been finalised, so only the mechanism for donating software will be described. Watch for full details in the next issue of Vector.

The basis of the Software Library is the APL 86 Software Exchange. This produced 16 items, many of which are of a substantial size. A directory of the software available will be a feature of this and future issues of Vector. Although each item in the directory has an abstract which describes briefly what is available, we will publish reviews explaining the purpose and use of selected software in more detail.

Whilst briefly scanning through all the software we kept on coming across things that looked really useful, or had to be saved to be played with later, until we ran out of disk space! Not everything is APL, there are some really good games/play items in machine code, and some useful DOS utilities.

A project of this sort needs plenty of activity to keep it alive, so it is essential that your contributions keep on coming in. Mel Chapman, our treasurer, has kindly agreed to be postbox for any software that you wish to send in before the arrangements are finalised. Details are at the end of the Software Library listing.

Technical Correspondence

From Eugene McDonnell

May 27, 1986

Sir: The second letter from Phil Last in volume 2 number 3 of your magazine asked in its last paragraph '...why haven't IBM (and others) done anything about this before?' He is complaining about the fact that he can write defined functions for dyadic iota and membership that execute faster than the primitives. I'm sure he will be gratified to hear that I.P. Sharp Associates has done something about it. In fact, Bob Bernecky of IPSA reported on his speedups for these primitives at the APL73 conference in Copenhagen, so users of IPSA APL have been the beneficiaries of Bernecky's speedups for thirteen years already. I give below some timings I've just made for Last's cases on the IPSA timesharing system. The same measurements apply (roughly) to IPSA APL on the PC and on the XT/370 and AT/370, since all of these systems are code compatible.

For Q=100+75000p2	
expression	time(normalised)
((Q1Q)=1pQ)/Q	1
DUPSQUT Q	5.9
Q1Q	1
Q DIOTA Q	16.7
QEQ	1
Q MEMBER Q	16.5

The fact that these IPSA speedups (and there are many others in our APL system) are so little known has given rise to an occasional amusing incident. Several years ago we had a summer student come to work for us here in the IPSA Palo Alto office whose APL training had come on IBM APL systems. He challenged us to come up with the fastest expression that would do

((Q1Q)=1pQ)/Q

Our then branch manager Joey Tuttle submitted exactly that expression as his entry, and the summer student was sure that Joey had missed the point, whereupon Joey asked the student to time the entry. He did so and was startled to find that the primitive was far faster than the arcane code which he had been intending to dazzle us with.

Sincerely,

Eugene McDonnell
Suite 201, 220 California Avenue
Palo Alto, California 94306-1683
U.S.A.

Prize Competition Result

My Other Car's a Porsche

by Dave Ziemann

The lateness of VECTOR 2.3 seriously reduced the number of entries to this 'bumper sticker' competition. This had the fortunate effect of making the judging a lot easier. Twenty five little bromides were boiled down to three winners; Martyn Adams supplied

The longer the function the shorter my patience.

While Claude Henriod from Switzerland gives the following yardstick:

The number of parentheses in an APL statement is inversely proportional to the experience of the programmer.

The first prize goes to Zeke Hoskins from Canada, who warns us that

A good language with a bad editor is a bad language.

Surely he can't be thinking of APL can he? £25, £15 and £10 (or equivalent) will be sent to Zeke, Claude and Martyn.

APL86 Competitions Round-up

by Dave Ziemann

APL86 was specied by a number of puzzles and competitions the winners of which received prizes kindly donated by the Sustaining Members of the BAA. For those of you not fortunate enough to attend the conference, these pages constitute a round-up of the competitions, their results and their winners.

APL86 Competition: Identify the Photographs

A montage of sixteen photographs taken at the last five APL conferences was posted on the notice board for delegates to identify. Space prohibits their reproduction here, but suffice it to say that were quite difficult – only two people correctly identified all sixteen photos; Gerard Langlet and Ludmila Lemagnen, both from France.

APL86 Competition: What is APL?

Competitors were asked to supply five alternative and original acronyms to those familiar three letters 'APL'. Among the individual entries that amused the judging panel most were:

Aristoteles Programmed Less	Rudolf Freihalter
Another Performance Lag	David Laur
A Perverse Love	Michael Rys
Above Prior Limits	Kim Andreasen
APL Provides Liberty (recursive!)	Kim Andreasen
Alters Programmer's Life	Banek Traude
Assistance Provided Later	Banek Traude
A People's Language	Frederick Macaskill
Artistic People's Lingo	Chester Holmes
A Pub Lunch	Chester Holmes
Avoids Professional Lunacy	Andre Bairamjan
Advocates People Laziness	Andre Bairamjan
All Purpose Laboratory	Howard Peelle
A Political Liability	Howard Peelle

Most poignant, that last one. The rules did not insist on the English language, and so we have:

Amere Pilule Laxative	Gerard Langlet (French for 'Bitter Laxative Pill')
A Prower Lie	Kim Andreasen (Danish dialect for 'I'll Just Try')
A Processer Langsomt	Kim Andreasen (Danish dialect for 'I Process Slowly')
Amicitia Patria Lex	Rudolf Freihalter (Latin for ??????????????????????)

Denis Samson felt that 'APL' are the initials of the unknown muse and inspiration of APL:
 Alice Pleasance Liddel.

Charles Winton produced the following five entries:

Allows	Programmers	Leisure
Arrays	Plus	Lovers
Absolutely	Precision	Like
Always	Practically	Learned
Almost	Perfect	Language

which have the amazing property of reading down the columns as well as across the rows!

There were many clever individual entries, but the two winners were selected on the strength of all five of their expansions. Yap Chua came second with:

All Purpose Language
 Amazingly Practical Language
 Accurate, Precise, Logical
 Almost Perfect Language
 A Pun-y Language

First prize went to Les Hollingbery who gave us the following five consistently excellent expansions for 'APL':

Acronyms Puzzle Learners
 All Pervading Language
 Always Pressing Limits
 Always Proceed Leftwards
 Any Problems Left?

APL86 Competition: Test Your Mental Flexibility

Delegates were asked to determine what the initial letters stand for in the following set of phrases, where each phrase is associated with a whole number. The phrases are:

1. 26 = L of the A
2. 7 = W of the A W
3. 1001 = A N
4. 12 = S of the Z
5. 54 = C in a D (with the J)
6. 9 = P in the S S
7. 88 = P K
8. 13 = S on the A F
9. 32 = D F at which W F
10. 18 = H on a G C
11. 90 = D in a R A
12. 200 = P for P G in M
13. 8 = S on a S S
14. 3 = B M (S H T R)
15. 4 = Q in a C
16. 24 = H in a D
17. 1 = W on a U
18. 5 = D in a Z C
19. 57 = H V
20. 1000 = W that a P P
21. 11 = P in a F T
22. 29 = D in F in a L Y
23. 64 = S on a C
24. 40 = D and N of the G F
25. 40 = W
26. 26 = Q in this C

For example, 16 = O in a P would be 16 = Ounces in a Pound. In case you would like to try it yourself, the answers are printed later in this issue. The prize winners at APL86 were Andre Bairamjan, Paul Chapman and Fergal Golding.

APL86 Competition: Matrix Rotation

Consider a matrix as composed of concentric rectangle-shaped layers. The problem was to write a dyadic function <MATROT> which rotates each layer a given number of positions about an axis perpendicular to the plane of the matrix and centred at the centre of the matrix. For example:

```

      M
      1 2 3 4
      5 6 7 8
      9 10 11 12
      13 14 15 16
      3 MATROT M
      4 8 12 16
      3 10 6 15
      2 11 7 14
      1 5 9 13
  
```

A positive left argument causes rotation in an anti-clockwise direction. Solutions were judged on their clarity, robustness and generality.

Each entry was classified into one of three types; there were 9 recursive solutions, 4 looping entries and 3 'direct' answers (no recursion or looping). Robustness and generality were tested by executing each function with prepared sets of arguments and ensuring that no errors occurred and that the result in each case was correct. The tests applied were:

1. Non-square matrix
2. Matrix with one row and one column
3. Character matrix
4. Empty matrix
5. Index origin independence
6. Zero left argument
7. Negative left argument
8. Vector left argument

Number 8 was a test for generality – did the function use successive elements of a vector left argument to rotate successively deeper layers of the matrix? This test was created in case a tie-break was needed. In the event two function passed the first seven tests, and only one passed all eight. The tests which defeated most entries were numbers 1 and 5. Only four solutions were independent of the global index-origin.

The recursive solution was probably best expressed by Kim Andreasen who took second prize. Kim's entry passed the first seven tests and was a model of well-commended clarity, incorporating excellent error-checking. Here it is:

```

V Z+N MATROT2 M;R;C;I;OIO
[1] A APL86 'MATROT' competition entry from Kim S. Andreasen, Denmark.
[2] A Uses recursive technique to rotate matrix 'M' 'N' positions.
[3] A Works with any size of 'M' and independent of global OIO.
[4] A 1. Errormsg. if 'M' is not matrix or if 'N' is not an integer singleton
[5] A 2. Returns 'M' unchanged if 0<ρM
[6] A 3. If only one row in 'M', rotates it leftwards if 'N' is positive.
[7] A 4. If only one column in 'M', rotates it downwards if 'N' is positive.
[8] A 5. If at least 2 rows and 2 columns in 'M': Rotates the outer layer and
[9] A exit if 'M' has no inner layers (Z=⌈ρM). Otherwise, fills the inner
[10] A matrix with the result of 'MATROT' applied to that inner matrix.
[11] Z←M ⋄ OIO+1 ⋄ R+1+ρM ⋄ C+1+ρM
[12] A Error checking as described above:
[13] →(0<1+0ρN)ρERR1
[14] →((1<ρ,N)∨0<1|'N)ρERR1
[15] →(2<ρρM)ρERR2
[16] A Nothing to do if 'M' is empty:
[17] →(0<ρM)ρ0
[18] N←'ρN
[19] A Special case if 'M' has a dimension of 1:
[20] →(1<ρM)ρL1
[21] Z←M
[22] A Find index in ravelled 'M' of the outer layer:
[23] I←(⌈C),(C+C×1R-2),(⌈(C×R-1)+1C),⌈1+C×1R-2
[24] A Do the rotate and reshape:
[25] Z[I]←N⌈Z[I] ⋄ Z←(ρM)ρZ
[26] A Finish if no inner layers:
[27] →(2=⌈ρM)ρ0

```

```

[28] A Do the same with the inner layers:
[29] Z[1+iR-2;1+iC-2]*N MATROT2 M[1+iR-2;1+iC-2]
[30] +0
[31] A Special case of one row or one column:
[32] L1:Z+(pM)p(N*xC-R)phi.M
[33] +0
[34] A Error handling:
[35] ERR1:+0,p[+]>> ERROR: ERROR IN COUNT ARGUMENT'
[36] ERR2:'>> ERROR: NOT A MATRIX ARGUMENT'

```

▼

The competition winner was Morten Kromberg from Denmark, whose entry passed all eight tests and used no recursion, looping or even branching. Morten's function included a comment that directed the reader to a one-page explanation of how the function works. Here is the function, followed by Morten's description:

```

V R=FACTORS MATROT1 MATRIX;PIOTA;SHAPE;N;DIM;REP;LENGTHS;INDICES;
PIOTA;OFFSET
[1] A MORTEN KROMBERG. SEE DESCRIBE FOR COMMENTS
[2] PIOTA+1
[3] N+([0.5*]/SHAPE+2+1 1 ,pMATRIX
[4] DIM+(-2*1+iN)*. +qSHAPE
[5] REP+(DIM,DIM)-(1 2 *pDIM)p 0 1 1 2
[6] LENGTHS+/(REP+REP*x\REP>0
[7] A
[8] INDICES+((,REP)/(*pREP)p 1 1 *1,1+SHAPE
[9] PIOTA+((pINDICES)-OFFSET+LENGTHS/+1+1,LENGTHS
[10] PIOTA+OFFSET+(LENGTHS/LENGTHS)|PIOTA+LENGTHS/NpFACTORS
[11] A
[12] R+MATRIX
[13] R[INDICES]+R[INDICES[PIOTA]]
[14] R+(pMATRIX)pR

```

▼

The function takes a matrix and rotates the concentric layers according to the left argument, which may have one or more elements. (They are repeated as many times as necessary). For example:

```

1 1 MATROT1 3 5p115
2 3 4 5 10
1 9 7 8 15
6 11 12 13 14

```

If the innermost layer has only one row or column (as in the example above), it is reversed along the last or first axis respectively. The function will also handle vectors and even scalars, treating them in the same manner.

The principle is to generate the indices of a spiral which starts at the top left, and spirals through all the layers. Once this is done, a partitioned rotation is done on the elements corresponding to each layer. Finally, the elements are put back into their new positions.

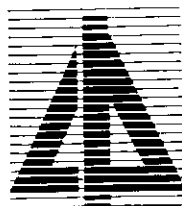
The algorithm used to generate the spiral works because one can calculate the indices of a layer with a sum scan of the differences between the indices of each element and the next. To go round the outermost (r by c) layer, one must step:

- | | | | |
|----|-----|--------------------|------------------------|
| 1. | r | times to the right | (index increases by 1) |
| 2. | c-1 | times down | (index increases by c) |
| 3. | r-1 | times to the left | (index decreases by 1) |
| 4. | c-2 | times up | (index decreases by c) |

This steeping calculation is repeated for all the layers. This is what happens in the function:

Line Comment

- 3: N is the number of layers, SHAPE is corrected in case the argument rank is less than 2.
- 4: DIM is now the shapes of each of the layers, each one 2 rows and 2 columns smaller than the previous one.
- 5: REP is the number of repetitions of the four steps described above, one row per layer of the matrix.
- 6: REP must be adjusted for layers with only one row or column, to avoid negative steps and other nasties.
- 8: We do the replication and sum scan, and get our spiral.
- 9: PIOTA is a partitioned iota, going from 0 to LENGTH[I] for each layer.
- 10: We do a partitioned rotation, by adding the rotational factors, and wrapping around by taking the RESIDUE with the numbers of elements in each layer (LENGTHS). Then add the offset for each layer.
- 12: Ready to go – prepare a result with enough elements.
- 13: Move each element into its new position.
- 14: Restore original shape.



COCKING & DRURY LTD.

THE APL PROFESSIONALS

16 BERKELEY STREET · LONDON · W1X 5AE

TELEX: 23152 MONREF G
TELEPHONE: (01) 493 6172

COURSES

APL Fundamentals

Oct 7 - 9 3 days £375.00

APL*PLUS PC Intermediate

Oct 13 - 16 4 days £525.00

APL Fundamentals

Oct 21 - 23 3 days £375.00

APL Fundamentals

Nov 4 - 6 3 days £375.00

APL*PLUS PC Intermediate

Nov 24 - 27 4 days £525.00

APL Fundamentals

Dec 2 - 4 3 days £375.00

Discounts are available to companies making more than one booking at a time:

Number of bookings	Total Discount
2	£50 (ie £25 per course)
3	£90 (ie £30 per course)
4	£140 (ie £35 per course)

If you require courses on APL2 or other specialised topics we will be happy to provide them on your premises. In-house courses are also likely to be cost effective if you have 4 or more people to train at any one time.

All prices exclude VAT.

For further information, please ring Liz Swann on 01-493 6172.

APL86 Competition Result: Test Your Mental Flexibility

The answers to the questions are:

1. 26= Letters of the Alphabet
2. 7= Wonders of the Ancient World
3. 1001= Arabian Nights
4. 12= Signs of the Zodiac
5. 54= Cards in a Deck (with the Jokers)
6. 9= Planets in the Solar System
7. 88= Piano Keys
8. 13= Stripes on the American Flag
9. 32= Degrees Fahrenheit at which Water Freezes
10. 18= Holes on a Golf Course
11. 90= Degrees in a Right Angle
12. 200= Pounds for Passing Go in Monopoly
13. 8= Sides on a Stop Sign
14. 3= Blind Mice (See How They Run)
15. 4= Quadrants in a Circle
16. 24= Hours in a Day
17. 1= Wheel on a Unicycle
18. 5= Digits in a Zip Code
19. 57= Heinz Varieties
20. 1000= Words that a Picture Paints
21. 11= Players in a Football Team
22. 29= Days in February in a Leap Year
23. 64= Squares on a Chessboard
24. 40= Days and Nights of the Great Flood
25. 40= Winks
26. 26= Questions in this Competition

Apologies to those not familiar with Stop Signs, Zip Codes or Heinz Varieties.

APL86 Competition: Where am I Running?

Competitors were asked to produce an APL function <WHEREAMI> that returns a number indicating the APL system upon which it is run. The following table of interpreters and corresponding numbers was given:

- 10 - VS APL
- 20 - APL2
- 30 - APL*PLUS PC
- 40 - APL*PLUS mainframe
- 50 - Sharp APL
- 60 - Dyalog APL
- 70 - APL.68000

We expected this to be quite tough, and indeed no-one supplied a complete solution. Claude Henriod of Switzerland came closest with this partial solution:

```

V WAI←WHEREAMI;DT;DELTA;OVER;MINIM;BITS;INTG2;INTG4;⎕PP;⎕IO
[1]  A APL86 competition, by Claude Henriod
[2]  A
[3]  A Not all the systems will be available at the conference, so
[4]  A I present a generalised approach which must be completed
[5]  A and/or simplified.
[6]  ⎕PP←4+⎕IO÷0
[7]  WAI←NB+DT NR NR←32⎕NR NB←⎕WA-12 NR+NR.NR '
[8]  WAI←⎕FX 6 11 ⎕WAI,'NB+NB-⎕WA NB+1[[NB+32'
[9]  DELTA←⎕AV[91 187 145 . 112 64 123]='Δ'
[10] OVER←⎕AV[190 239 244 . 77 205 25]='▽'
[11] A v can be replaced by another overstruck character
[12] MINIM←(, 2 10 ⎕'7.237E75 '),'-1.798E308' -7.237E75 '
[13] MINIM← 7 10 ⎕MINIM,'-1.798E308'-3.595E308'
[14] MINIM←MINIM^.=10+▽[10
[15] BITS← 1 1 2 . 1 1 1 =DT 1
[16] INTG2← 4 4 2 . 4 2 4 =DT 30000
[17] INTG4← 4 4 8 . 4 4 4 =DT 60000
[18] WAI←(DELTA+OVER+MINIM+BITS+INTG2+INTG4)/10×1+17
V

```

The dots on lines 9,10,15,16 and 17 represent missing values corresponding to the APL*PLUS mainframe APL, and would need to be replaced by the appropriate values for the function to work.

The principle of the function is to create a binary matrix where each column corresponds to one interpreter and each row is a specific test, such that on any given system only one column will be all ones.

The first part of Claude's solution checks the position in the atomic vector of the 'delta' and 'format' characters. The second part checks the lowest representable number. Note that the character form of this number is tested so that the number -3.595E308 does not blow systems that cannot represent it. (This may in fact be more of a test of the host machine than the interpreter itself . . .). The third part of the function analyses the word-length used for booleans, small integers and large integers. <DT> is an inner subfunction that attempts to determine the storage required for each element of its argument.

Interestingly, Claude's function appears to be ISO standard conforming. Can anyone complete or improve upon <WHEREAMI>?

APL86 Competition: APL86 Yeargame

Although this is an old favourite it nevertheless attracted many entries. The idea is to use the digits 1,9,8 and 6 to create as many of the whole numbers from 1 to 100 as possible. The digits must be used once only and in the correct order. No other constants may be used, but any APL primitives can be included. For example, the number 21 can be formed most succinctly by the expression 19-8-6, which is a 6 character solution.

1=11986	21=19+8-6	41=1019-816	61=11.98*6	81=1*9*8-6
2=11.986	22=119+8*6	42=1918*6	62=1019.86	82=1+9*8-6
3=119+816	23=119+8+6	43=1918*6	63=1019.86	83=19+8*16
4=119+86	24=1+9+8+6	44=119186	64=119*86	84=119*86
5=19-8+6	25=19+816	45=119+8+6	65=1019+8-6	85=119*86
6=19816	26=19+8-6	46=119+8+6	66=1*918-6	86=19186
7=1+9816	27=19+816	47=1+918*6	67=1-9-86	87=1+9186
8=191816	28=19+8+6	48=1918*6	68=1918*6	88=11.9+86
9=1*9186	29=1-9-8+6	49=1+918*6	69=1*19*8*6	89=1-1.98*6
10=19186	30=119*86	50=119+8+6	70=1*19*8*6	90=1-1.98*6
11=19-816	31=19+1*8+6	51=119+8+6	71=1-9*816	91=1-98-6
12=19186	32=119+86	52=119*8*6	72=1*9*816	92=1198-6
13=19-816	33=198+6	53=1+9*816	73=1+9*816	93=1+98-6
14=1918+6	34=1*9816	54=19816	74=119*8*6	94=1+9+86
15=119+8+6	35=1+9816	55=1+9*816	75=1019*8*6	95=1*9+86
16=1+9+816	36=119*8*6	56=1+9+8*6	76=1-9-86	96=1+9+86
17=19-8-6	37=19+816	57=119+8*6	77=1*9-9-86	97=11-9816
18=19-8+6	38=19+8-6	58=1+9+8*6	78=1-9-86	98=1*9816
19=19186	39=1*9-8*6	59=1019186	79=1+9186	99=1+9816
20=19+8+6	40=1-9-8*6	60=1019186	80=191816	100=1+98*6

If expression 12 really does produce a result of 12, it is a spurious answer given by the particular APL interpreter used, and not a mathematically correct expression. Therefore it should probably be replaced by a more suitable expression, eg 1+9+8-6. Expression 4 is also out on a limb, in that it returns a one-element vector result rather than a scalar.

The behaviour of some of the less often used primitives can be clarified by studying this table. For example, note numbers 32,34,44,54,64,66,68 and 80.

Any improvements on this answer will of course be received with interest and published.

APL86 Competition: An APL Story

This one proved a welcome break from the rigours of matrix rotation and the 1986 yeargame. Write a story in 'APLese' was the simple instruction. For example:

```
1 1 1 1+TTS MvKAlD DECIDED 2 GO pING.
```

Which translates to: 'Once upon a time Mork and Mindy decided to go rowing.' Phil Last came second with the following groan-inducing sentence:

```

      THE 1 THAT V 1+- S + e ' 2 [ CH *
I AUGHTA DISCLOSE THE FACT THAT DEL EGATE S GO IN ' TO MIN CH ASTER
' + AN | [/ □TS ^/ □TS[□IO+2] ~ →0
' AD AN ABSOLUTELY SUPERLATIVE TIME EVERY DAY TILDE END.

```

The labours of Mark Wolfson and Eileen Keating proved unbeatable however. They produced a complete APLese story – with a title, beginning, middle and of course a happy ending. Here it is:

The pp of lchestv
The rose of Manchester

by Mark Wolfson and Eileen Keating

```

~ | v l t of fvt. e ned the pps ^ hAed
Without style or one iota of effort, he picked the roses and handed
them 2 hv. She † out the † pp. •ed ^ said:
them to her. She took out the first rose, wheeled and said:

I'm Vighted - but u just do ~ = the A.
I'm delighted - but you just do not make the grade.

α ≡ already - [ . U v * < † me!
I've a match already - Max. You are powerless over me!

wd e !ed. E † the c c it. A love
Oh my God he exclaimed. He took the box and closed it. A love
ted + 10. Now v u † /ed 2 a lute p.
divided is nothing. Now our union is reduced to a minute shape.

I rec when it was i_ {fv e was Gv!}. 2mvp
I remember when it was wunderbar (for he was German). Tomorrow

I will dvl my svpp. " x u c a pp rec me
I will drown my sorrows. Each time you see a rose remember me

^ - 2 'la□□a' n '87.
and dash to Dallas in '87.

```

APL86 Competition: Defined Operators

Dyadic Systems ran a competition for the best user-defined operator to be entered onto Dyalog APL on their stand. The first prize was one hundred pounds in cash or an evening out with John Scholes, the implementor of Dyalog APL. (There were some nasty rumours going around that the second prize was one hundred pounds in cash *and* an evening out with John Scholes, but this proved to be unfounded).

Anyway, the winner was Ray Cannon who supplied the operator <DUAL>, possibly as a nod in the direction of Ken Iverson's Core APL:

```

V R+{A}(F DUAL G)B;DIO
[1] A OPERATOR DUAL
[2] +MON×IO=INC 'A' A IF CALLED MONADICALLY
[3] R+G INV(G A)F G B
[4] -END
[5] MON:
[6] R+G INV F G B
[7] END:
V

V R+{A}(F INV)B;DIO;FN;IFN
[1] A OPERATOR INVERSE
[2] A PERFORMS THE INVERSE OF FUNCTION F ON ARGS A AND B
[3] A EG 6 × INV 3 IS THE SAME AS 6 ÷ 3
[4] DIO+1
[5] FN+'×××<=>[?C?+]'
[6] IFN+'-÷÷÷>=[?C?+]'
[7] -ERR×1~(DIOCR 'F')∈FN.IFN
[8] R+(FN.IFN)1,DIOCR 'F'
[9] +MON×IO=INC 'A' A IF CALLED MONADICALLY
[10] R+1'A'.(IFN,FN)[R],'B'
[11] -END
[12] MON:
[13] R+2(IFN,FN)[R],'B'
[14] -END
[15] ERR:'SORRY INVERSE NOT KNOWN'
[16] -
[17] END:
V

```

As you can see from the example, the operator applies its right operand to each argument, applies its left operand between these results, and then applies the inverse of the right operand to produce the answer. The inverse is produced, where possible, by the associated operator <INV>.

An arguably less useful contribution was the <PERHAPS> operator, which may or may not apply its function operand, according to whim:

```

V R+(F PERHAPS)B
[1] 2(2!2)/R+F B'
V

```

APL86 Competition: Win an ATARI 520 STM with APL.68000

The grand prize of APL86 was undoubtedly microAPL's Atari computer with APL.68000 bundled in. Contestants were asked to rank ten features of an APL system according to importance, and to complete a tie-break slogan, before stuffing their entries into the bulging box on MicroAPL's stand. The lucky winner was Jonny Osterman, who can be seen receiving his prize in the photographic review section in this issue.

Prize Competition: Photo Caption Contest

by Dave Ziemann

The four photographs shown here were all taken at APL86. The competition is to create appropriate or amusing captions for one or more of them. You may submit at most one caption for each photograph. The authors of the two best captions, in the judges' opinions, will share the equivalent of fifty pounds in prize money. The best captions will be published.

Be sure to label each caption with one of the letters A,B,C or D. Entries received after 31st December 1986 may not be considered eligible.

Photo Caption Competition



A



B



C



D

APL Standard Progress Report

by Dave Ziemann

On Saturday 5th July, just prior to APL86, the ISO WG(Working Group) for the standardisation of APL met for the ninth time, in Manchester. Representatives from Canada, France, Germany, Great Britain, Sweden and the USA attended the meeting.

By this time, all the technical work on the document had been completed, and it had been accepted as a second Draft Proposal by a vote of 13 approvals, 6 approvals with comments, no disapprovals and 1 abstention. Karl Soop of Sweden, and Raymond Tisserand, the convenor of the ISO WG, collated the comments and Karl visited the editor of the document, Alex Morrow, in Palo Alto to make the necessary alterations.

The modified draft was sent to the ISO SC22 central secretary for processing as a DIS (Draft International Standard) in June, and it is this document, along with a French translation, that will eventually become an IS (International Standard). Unfortunately these things take time to percolate through the bureaucratic layers, and the document is unlikely to become a DIS before early 1987, or an IS before the end of that year.

After this had been reported to the group Raymond Tisserand passed the convenorship and secretariat of the WG over to Lee Dickey of Canada. Raymond had convened the group for the last five years, and was now retiring.

So with the APL standard all but put to bed, what was left for us to do? Lee Dickey presented three documents to the meeting, each suggesting further work items: one from the ANSI group, one from the BSI group and one from a list produced at the Minnowbrook implementers conference. After much discussion the WG decided that a new work item called 'Extended APL' should be proposed. This would include the following topics for consideration in a new standard APL: generalised arrays, operator extensions (primitive and defined), the scope of names, files, complex arithmetic and exception handling. It was agreed that the group need not restrict itself to this list, or even commit to work on any topic mentioned. The motion was passed by a vote - 9 for, 0 against and 4 abstentions. The abstainers were either unhappy about the items in the list or felt that work should not proceed until the current standard was published.

Lee Dickey then presented a position paper on standardisation and validation. The paper suggested the idea that a test suite for the validation of APL systems purporting to be standard-conforming could be developed and controlled by an international group, such as the ISO WG itself. It was noted that this would be a world first, as it was usual for conformance validation to be performed by independent or national testing houses. After lengthy debate it was proposed that the job of designing a validation procedure for a program product "may lie within the domain of the WG". This wording allows the group to consider the idea before committing to a project of unknown duration and cost, and as such the motion was passed unanimously.

The ISO WG agreed to meet again during the week before APL87, in Dallas.

During APL86 itself, there was a sparsely attended session on the APL standard. Raymond Tisserand presented a brief history of the standardisation effort, from the time the French standards body AFNOR started in 1978 up until the present. At the end of his talk Karl Soop and Paul Barnetson thanked Raymond for all his hard work over the years, and surprised him with a gift from the WG – a bottle of scotch and a bouquet of flowers. Karl then reported the details of the most recent round of technical work on the draft and Lee Dickey explained the decisions taken at the WG meeting. They were then joined by Clark Wiedmann who pioneered ANSI involvement in the work, Bob Bernecky of IPSA and David Livingstone of Newcastle Polytechnic, for a panel discussion. Questions concerning the possible standardisation of files and GKS (Graphics Kernel System) bindings, among others, were discussed.

The session finished with an invitation: anyone interested in getting involved with the standardisation work should contact their national representative. For the UK, you should contact David Ziemann at the address given on the inside back cover of VECTOR.

The tireless work of Raymond Tisserand, Alex Morrow and Clark Wiedmann was honoured during the APL86 banquet, when John Myrna, the chair of the US APL Special Interest Group, presented them with the SIGAPL outstanding achievement award for 1986.

APL/PC 2.0 Auxiliary Processors

by Manuel Alfonseca & David Selby

This tutorial was given by David Selby, on behalf of Manuel Alfonseca and himself. The paper in the volume of Tutorial Proceedings runs to some twelve very informative pages, so this report addresses itself to the presentation at the Conference.

David gave a very accomplished showing of a series of graphics displays, some of them animated, using an IBM PC/AT and a Barco projector. The PC/AT had a co-processor, one megabyte of storage and a standard Colour Graphics Adapter. (In principle, the presentation would run on any PC with 640K storage.) All of the graphic images were produced using APL (naturally), using Auxiliary Processors 124 and 206. Some images were constructed 'on the fly' and others were pre-computed.

The Auxiliary Processors were illustrated in turn.

AP 80 – Printer Control

A very simple protocol is used with one shared variable. The user has to be aware that the printer has a buffer, and will not print characters received until either the buffer is full, or a suitable control character (e.g. newline) is received. AP124 was used to produce an animated picture of a series of characters being passed from PC to printer. (The picture of terminal and printer was constructed from standard display characters.)

AP 2 – non-APL Program Interface

APL obtains storage in 64k partitions in which it can load and run programs from disk. Communication between these programs and APL is possible. DOS, FORTRAN and other programs can be executed in this way, and Auxiliary Processors can be loaded. Thus a user's processing requirements can be met while retaining control through APL, and space constraints can be eased through loading APs dynamically in an overlay structure.

AP 101 – Stack and Session Control

This is broadly similar to the AP of the same number in mainframe implementations, permitting stacked communication with the host system (i.e. DOS).

AP 103 – BIOS/DOS Interrupt Processor

Again, this provides an interface with the host, this time a direct interface. The user can have a very low-level interface with DOS using AP 103 to generate software 'interrupts' to manage space within the APL workspace and to drive I/O ports.

AP 124 – Full-screen Text Processor

Compatibility with the VSAPL AP124 is provided. As mentioned above, some of the 'pictures' produced during this tutorial session were in fact not graphics images, but were composed of characters displayed using AP 124. The more usual text display was illustrated in detail.

AP 190 – Host Communications Processor

This is probably the most exciting of the APs of APL/PC 2.0, because it provides the opportunity to perform workstation and host programming under the complete control of the APL/PC session. The illustration given showed very effectively and in parallel both the underlying APL code and the host session as seen by the user.

AP 210 – DOS File AP**AP 488 – IEEE/GPIB Processor**

These were not covered in detail.

AP 206 – Graphics Processor

Starting with illustrations using APL/PC GRAPHPAK (familiar to mainframe users) – including of course the famous 'apple' – numerous animated displays were shown, with the mechanics being revealed. Animation is achieved through the use of graphics buffers in storage, with sequencing and timing managed by APL functions.

A beating heart was shown very effectively. This is one result from work going on at the IBM Scientific Centre in Winchester, normally with higher-resolution displays. As a grand finale, graphics and music were combined to play out a very successful presentation.

Questions

Q. Is the Enhanced Graphics Adapter supported?

A. No, but we would like to do it.

Q. Who controls your screen when using AP 190?

A. You do, in APL.

Q. Using AP 2, how do you manage file-types?

A. The loader sorts out whether it has a COM or EXE file.

Info Center/1

*an IBM licensed program that helps business professionals
perform their daily tasks quickly and productively*

Info Center/1 provides an integrated, multifunction information center environment compatible with predecessor products such as ADRS II and APLDI II. A full-screen interface, with prompts and extensive help facility, provides easy access to the following powerful general business functions, as well as providing the full power of APL:

Query System

The Query System provides a simple, effective way to interactively access, analyze, manipulate, and report information stored in files of up to several hundred megabytes.

Reporting System

Provides an organization with a single, comprehensive system for generating and maintaining reports. Standard calculations can be defined and stored for future use. Calculations can be made with predefined functions and with APL.

Data Entry and Validation

This tool allows information center personnel to tailor panels for users to display, update, and enter data in column format.

Financial Planning System

The Financial Planning System provides a set of 60 modeling routines that work with the Reporting System and address periodic data. Some examples are:

- Financial analyses and plans
- Statistical analyses and projections
- What if analyses and modeling
- Project evaluations and risk analyses.

Business Graphics

The Business Graphics facility is a particularly powerful yet flexible tool for interactively producing the following types of charts: line graphs, surface charts, histograms, pie charts, scatter plots, bar charts, stacked bar charts.

Technical Data

Info Center/1 is an IBM Licensed Program, Program Number 5668-897.

The program runs under CMS and TSO together with the following IBM programs or their equivalents: APL2 or VS APL, Application Prototype Environment, GDDM (Graphical Data Display Manager). Some examples of terminals supported are: IBM 3277, 3279, 3270 PC/G and GX.



Development of APL through standardisation

by David Livingstone

Introduction

The purpose of this article is to suggest that :

- APL should have major extensions, added in a relatively short time.
- this should be achieved through new international standards for APL.

Why APL should be significantly extended

We are presently in a period of rapid development of computer languages. There are new languages for new areas (eg PROLOG for Expert Systems), new types of languages (eg Functional Languages, Query Languages, Graphics Languages), as well as new languages to replace old languages (eg ADA to replace COBOL, PASCAL to replace FORTRAN). There are also many packages (eg Word Processors, Spreadsheets, File-handling Packages, Expert System Shells) that are effectively specialised languages yielding high programmer productivity. Some even integrate several erstwhile language areas (eg Framework, Symphony, Mimer).

A major objective of all these is to obtain higher programmer productivity by some means or other. The so-called Fourth Generation Languages (= 4GL) are explicitly aimed at this, and claim to give much higher programmer productivity than other languages. Since high productivity has always been APL's main strength, it ought to be very successful. (Is APL of too venerable an age for the label "Fourth Generation" to be comfortable?)

However, currently APL is largely ignored by the computing world. It is ignored by academics because it does not conform to their accepted ideas of what constitutes a good language. It is ignored by industry because for most of the part it has been too inefficient a user of hardware; and recently it has not been marketed extensively as a 4GL.

Will APL survive in this environment in the long-term (10-15 years), given the great pace of language development? I believe to do so the best approach is to aim to make APL the "COBOL of the future", the language that is normally used by most people for most tasks. To achieve this:

- APL must incorporate major extensions, which confer a significant increase in power and value to APL.
- The extensions must be such as to bridge the gap which has cut APL off from both academics and industry, and bring APL into the mainstream of the computing world; otherwise APL will still not be a commonly accepted language. At the same time, it must not lose the essential nature and characteristics of APL.
- The extensions must be added in a relatively short time, to compete in the marketplace.

In the author's view this means pursuing the following 3 strategies :-

1. The extensions must enhance APL as a high productivity programming tool. Since such a language provides real benefits to the user, industrial users will be receptive to it. The big increase in sales of 4GLs and high productivity packages indicates the desire of the market to use such tools. As APL already provides high productivity in the programming language area, much of the enhancement will have to come from integrating other computer language areas into APL, e.g. database. This approach is already well established with packages such as Framework and Mimer. (Technically packages and languages are similar, the difference being one of emphasis; a package usually has a more specialised application than a language).
2. The extensions should incorporate some of those concepts of computer science that academics think are essential to a modern computer language - eg data types and loop constructs. There is a tendency in the APL community to reject ideas thought valuable elsewhere in computing, because their conventional application does not fit the APL philosophy. Nevertheless some of these concepts can be cast in a form consistent with APL, and to do so would make APL more appealing to academics.
3. Extensions must be based on generalisations of existing APL language principles or on principles which fit harmoniously with APL. The "extended APL" must not be a collection of bits and pieces just glued together. A "collection" leaves the problem of having to learn different styles and syntax for each area, and to design the linkages between the areas for each application. To overcome this, the whole language must be one integrated and unified whole. It would also ensure that "extended APL" had those characteristics that make current APL attractive to its users.

Extensions to APL

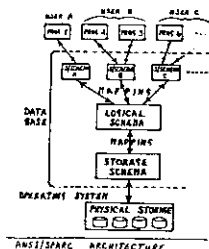
To illustrate the extensions (in addition to Nested Arrays) referred to above, a few suggestions are included here :-

1. Michael Crick (see reference 1) has proposed that APL be generalised into an object-orientated language. This means that insted of having several totally different categories of object - functions, arrays, workspaces, groups, etc - there is only one kind of object in APL. The variety of different things in the language is achieved by having sub-categories of the one kind of object; where necessary, a sub-category can be split into sub-sub-categories, and so on hierarchically down. This means that all objects in APL have a set of properties in common. Each sub-category has some extra properties peculiar to itself.

The benefit of this is that APL is simplified, becomes easier to learn and use, and gains flexibility through the removal of anomalies: i.e. greater programmer productivity. For example, arrays would be able to hold functions, or collections of functions and data; a workspace is just an array of functions and (nested) arrays; a workspace can be held as an element of an array, and hence manipulated like an array, giving greater power over workspaces while removing their special commands like "COPY". (See reference 1 for further details.)

Since object-orientated languages (e.g. Smalltalk) are of great interest to the academic community, it would make APL more acceptable to academics.

2. APL is basically a logical language. That is to say, when writing an APL expression, the programmer is specifying conceptually what is to be done rather than how the computer should physically execute it. Nevertheless in practice, people learn how APL executes expressions and will adjust their code accordingly. Likewise programmers may store data in files rather than workspaces because of the physical differences in their implementation. It is proposed that the physical and logical aspects are separated out, using a layered structure based on that of the ANSI/SPARC architecture used for databases and illustrated in the figure; see also reference 2.



The ANSI/SPARC architecture is an academically sound and accepted way of designing Database Management Software. It allows one to specify a set of database relations (say) as the "Logical Schema" or design of the database, and separately specify the "Storage Schema" or files in which the relations are to be stored. The Storage Schema may be continually altered to optimise machine performance while the Logical Schema remains constant. Consequently programs using the database are insulated from any file changes. Furthermore the ANSI/SPARC architecture permits different programs to have different views of the Logical Schema, ie different Sub-schemas. This allows one program to change its Sub-schema while insulating other programs using the same database from having to change theirs to match it.

If this same structure were to be introduced into APL, the analogous benefits would accrue. A large data array could be held physically as a program array (the normal APL method) or a disc file of any type; the normal logical APL code would remain the same regardless of how it is stored or if the storage method were changed. One would just change the mapping of the logical object onto its physical storage. Similarly it would be possible to map one set of logical code onto another set of logical code; this would permit a simple but crude algorithm to be mapped onto a more sophisticated and efficient algorithm; the latter would be what was executed but the former used as a simple overall description of what logically was to be achieved. If no mapping were specified, the APL code would be unchanged from its current form where the interpreter (or compiler) automatically maps between the logical and physical states.

This facility would be of great practical benefit in prototyping an application. An initial prototype could be quickly developed to show the user and refine the system specification; later an efficient production version could be developed from it by modifying the physical data storage and/or algorithms with the minimum of code changes. This reduces both the work required and risk of introducing new bugs into reworked code.

This architecture would also provide a framework into which numerous of the current ad-hoc techniques for improving machine efficiency (e.g. Auxiliary Processors (see reference 3), special External Variables (reference 4)) could be put to achieve a coherent approach (see reference 5).

3. The above extensions will create a large part of the infrastructure of a database system. The third extension should be to explicitly add APL primitives, concurrent processing and security facilities in order to incorporate into APL a Relational Database using Relational Algebra. Such a database (with algebra not calculus) has a philosophy and principles which correspond directly to those of APL (see 6), and this extension would fit harmoniously with it. The author already has a research project nearing completion which demonstrates their natural integration – reference 7.

Already many 4GLs are based on databases rather than programming languages, and there is no doubt that data storage and retrieval are equally as vital to a general-purpose application development system as are data manipulation and calculation. This extension would allow APL to have “a foot in both camps”.

The extension would yield significant programmer productivity. The development of many general-purpose Query Languages based on databases clearly demonstrates the practical need to integrate the programming and database areas. Relational Databases are well established academically, and there is far more academic agreement on what constitutes a basic set of Relational Algebra functions than there is for the equivalent Relational Calculus.

4. There is much academic and industrial support for the “Programmer’s Workbench” concept, because of its productivity benefits. The Programmer’s Workbench is an integrated set of tools to support the programmer in developing software. There are many views on what should constitute a workbench. The important point here is to specify a framework and philosophy so that a coherent set of tools can be developed which integrate with APL. They should build on the workbench tools that already exist in APL: e.g. the workspace Symbol Table should be extended into a Data Dictionary so that other useful information about variables and functions could be held and easily referenced by the programmer.

The workbench is part of the environment in which the language operates. Integration of environment and language is already important in APL and should continue to be so – see 8.

Some workbench facilities can be precisely specified in a language; others can only have an interface to internal system variables and functions defined, leaving the implementer free to develop them in as creative a manner as he thinks fit. The following are a few suggestions for workbench tools:

- A compiler. It would be useful to define the specific interface and functional arguments for a compiler; e.g. how is source as opposed to object code referenced?
 - Language checking software. Most languages have code checking built into their compilers. Given that most future APL applications will still be debugged using an interpreter, it is better to separate checking and compiling. All sorts of possibilities suggest themselves here, including eventually software to annotate and document functions automatically. Here a language standard should give the implementer freedom to develop ideas.
 - The representation of APL objects (e.g. keywords, symbols) should be separated from APL itself. The aim would be to allow the input and display of APL software using APL symbols, keywords or any other desired representation (while leaving APL syntax unchanged), varying the representation in one session if desired. This would allow the use of non-APL keyboards, give the freedom to exploit future I/O technology, and provide the opportunity to release APL from the "Chinese Pictogram Syndrome" where its vocabulary of primitives expands beyond the limits of representation by a reasonable set of symbols. An input interface could be standardised, with an associated standard describing the current symbols (plus in future other alternative standards to the latter?).
5. The functional equivalents of the loop and IF/CASE control constructs found in other programming languages, plus logical data typing, should be added to APL. They would increase programmer productivity, but more importantly give academic and industrial respectability to APL by providing it with what are seen as basic essentials in a modern programming language.

Samson (reference 9) and Backus (10) have demonstrated that the control constructs can be implemented in a functional manner in keeping with APL and quite different from conventional notation, while still providing the power of these constructs.

The APL equivalent of data typing is the concept of domains. This should be extended to allow a programmer to associate with a variable a specific domain-set of values rather than be limited to just the 2 primitive domains, numbers and characters. These domains would be defined by enumeration (all possible values are held in an array) or conditions (these are executed by a function). The array or function would be "assigned" to a variable as its domain. This is like ADA and PASCAL user-defined types, but much more powerful since the full power of APL can be used to create a type/domain. Only an extra "domain-assign" function is essential. Such domains (equivalent to those in Relational Databases) would provide powerful data validation and debugging facilities.

The role of the Standards bodies in extending APL

The APL standard which hopefully will soon be ratified, is designed to establish the basics of APL. It is deliberately intended not to be controversial, but to specify the "Lowest Common Denominator", the established practice of existing APL systems. This is a wise approach because it is the first standard to be created despite the language having been in use for twenty years. It is a tribute to Iverson's design that the basics have remained the same for so long without a standard.

One possibility for further progress is to proceed from this standard in a bottom-up fashion, adding in a piece-meal way those specific facilities (e.g. error trapping) that are coming into common use.

In the author's view the real need is to look ahead and make a strategic decision as to how APL should develop in the future, a top-down approach, not bottom-up. The above sections have tried to show that several very large scale extensions are required, and in a relatively short time, if APL is not to decline into disuse. Even if the APL community disagrees with this view, or the particular large-scale extensions suggested here, it is still important to make an explicit strategic decision on where APL goes from here. No major extensions are likely to arise in an agreed fashion without it, as the saga of nested arrays demonstrates only too clearly. If APL were to be developed in a limited bottom-up fashion, a de-facto strategic decision would implicitly have been taken to do this, and it would be better if this were made clear to all interested parties.

The only real forum for discussion among the APL community appears to be the ISO Standards organisation and the associated national standards bodies. Therefore after completing the present standard, their next task should be to promote proposals and discussion on this topic *and* make the final strategic decision on APL's future. Were the APL software vendors to form their own private association to do this, it would be inadequate because it would have no user representation. APL users are not really in a position to form such an association, with or without the software vendors. An extra advantage of the standards bodies taking up this job is that the conclusions they reach will carry considerable weight with all concerned, which is not necessarily true of any alternative organisations.

Having taken a strategic decision, the standards bodies should then take a very proactive approach to developing standards for all the extensions, be they great or small, required to fulfill the decision.

If things are left as they are, APL software vendors will naturally develop their products vigorously to compete and become successful. (They would be foolish to do otherwise). The case of nested arrays shows that these developments are as likely to diverge as to converge on a common standard, even when the extension is supposedly the same thing from all vendors; or they will be IBM-compatible extensions because of IBM's clout in the

marketplace. The "Lowest Common Denominator" policy will not work with nested arrays because incompatible versions have been developed. A standards body will either have to choose one implementation or create a new one, which presumably would be a compromise of some kind. This is likely to be very difficult, contentious and time-consuming, with many APL people in entrenched positions. If this were to happen with every new extension, progress would be very slow if not impossible.

Thus the standards bodies should pre-empt these problems by moving ahead of the implementors and creating standards which form targets for the implementors to aim at. This would follow on as a natural consequence of the standards bodies deciding on APL's future development strategy.

There would be nothing new or remarkable about the standards bodies doing this. The ISO Standards organisation has for a number of years been developing the "Open Layer Reference Model". This is a large and comprehensive set of data network standards, built on a layered architecture of seven layers. It is very forward-looking and a major creative exercise, producing something which is very complex and sophisticated, although manufacturers' existing protocols have been utilised where appropriate. All these standards have preceded the software to implement them. Manufacturers have taken the standards after they have been published and written software to meet them. All this is fully the equivalent of what is proposed here for APL.

In the case of APL, the ISO Standards organisation and associated national standards bodies should decide on what extensions are to be added to APL and in what sequence, using some order-of-magnitude timescale. The previous section could be considered as an APL "Marketing Man's" specification of extensions required to compete successfully over the next few years. ISO should produce their equivalent, but far more comprehensively of course. Then for each extension they should produce a standard, detailed in the same way as the current standard, for APL vendors to write software for.

In conclusion, there remain two more points to be mentioned:—

1. Some may doubt that it is possible to add on major extensions without changing the nature of APL radically from what it is now. The author's conviction that it can be done arises from noticing that both Relational Databases (using Algebra) and the Unix Operating System are the result of applying the same principles and philosophy as APL to other computer language areas — see references 11,6,12; data types and control structures can be cast into APL's functional form; hence there is no logical reason why APL should not be enlarged to incorporate these areas in one uniform and coherent language. The policy must always be to tease apart those principles which are currently bundled together in APL and then generalise each one as far as possible. Sometimes it is necessary to look to other computer languages to gain an insight into what is currently bundled together. (In the process of extension, APL may turn into ACL — A Computer Language — reflecting the fact that it is more general-purpose than a programming language; and the change of name may help it overcome the problems of its present image).

2. Some may point out that standards bodies have very long gestation periods in giving birth to their progeny, and APL is no exception to this. Therefore why expect standards bodies to produce major extensions to APL in a relatively short time? There is much truth in this. The problem is that no other alternative seems possible. If you have a faster method, please promote it.

References

1. *Should APL be a Declining Language?* by M F C Crick (APL81 Conference Proceedings)
2. *An Introduction to Database Systems* by C J Date (3rd ed), Addison-Wesley, 1981, Chapter 1
3. *Improving APL Performance with Custom Written Auxiliary Processors* by A K Dickey (APL85 Conference Proceedings)
4. *Dyalog APL User Guide*, Page 7.40
5. Reference [1] page 86
6. *Relational Database: A Practical Foundation for Productivity* by E F Codd (Communications of the ACM, February 1982, Vol 25, No 2)
7. *The Integration of Relational Database Algebra into APL* by D Livingstone (APL86 Conference Proceedings)
8. *Towards Monolingual Programming Environments* by J Heering and P Klint (ACM Transactions on Programming Languages and Systems, Vol 7, No 2, April 1985)
9. *A Proposal for Control Structures in APL* by D P Samson (APL84 Conference Proceedings)
10. *Can Programming be Liberated from the Von Neumann Style? A Functional Style and its Algebra of Programs* by J Backus (Communications of the ACM, Vol 21, No 8, August 1978)
11. *The Design of APL* by A D Falkoff and K E Iverson (IBM Journal of Research and Development, July 1973)
12. *UNIX Time-Sharing System: A Retrospective* by D M Ritchie (The Bell System Technical Journal, Vol 57, No 6, July-August 1978)

Using Quad-FX to facilitate the use of Auxiliary Processors

by David Piper

1. Introduction

□ FX is commonly used in large APL systems for overlaying code in workspaces, fixing local functions containing system specific code etc. □ FX is used as a form of "compiler" which allows the conversion of parameter driven systems into hard-coded systems. If any of the parameters have to be changed during the life of the system, the functions can simply be re-fixed using □ FX.

□ FX can also be of use in connection with shared variable processing in an analogous way. In many functions using shared variables, the names of the shared variables are passed as an argument. The function then uses execute to assign or use the shared variables. Typical of this type of implementation are the IBM supplied functions for use in association with API23.

2. Shared Variable Processing

The use of execute for processing shared variables arises as a result of the dichotomy inherent between the use of the NAME of the variable and the use of the VARIABLE itself:

- At share offer/retraction the NAME is used.
- During processing the VARIABLE is assigned/referenced.

Since the name of the shared variable may change, any function used to cover the use of shared variables must use execute to assign or use the correct variable. This is especially true if more than one set of shared variables is to be used, in which case the names of the shared variables to be used will probably be passed as an argument to the cover function, or as a global variable.

An example of this form of implementation is readily available in the IBM supplied workspace VSAMDATA, which makes use of API23 to access VSAM clusters. Other examples of the use of execute abound in the IBM supplied workspaces which are involved in shared variable processing.

It is especially desirable to avoid execute in situations such as reading or writing files since the volumes of data input/output (and hence the number of times the executes are processed) is potentially very large. Consider also that up to 3 executes may be involved per file operation – this is certainly true for the VSAM data AP, API23. To read data:

- Assignment of the read command to the "control" variable.
- Use of the "control" variable to check the return code.
- Use of the "data" variable to retrieve the data.

To write data:

- Assignment of the data to the "data" variable.

- Assignment of the write command to the "control" variable.
- Use of the "control" variable to check the return code.

3. Using $\square$ FX

To avoid the use of execute, a function must be created which contains the references to the control and data shared variables explicitly. The function is created and fixed at the time the shared variables are offered to the auxiliary processor.

The following terminology is used. Each function associated with a single set of shared variables is referred to as a "PATH". A path is created by "LINKING" to the auxiliary processor and is destroyed by "UNLINKING". The functions listed at the end of the article, link a new path (VSAMLINK) and unlink an existing path (VSAMUNLINK) with the VSAM auxiliary processor, AP123. The right argument to both functions is the name of the path to be linked or unlinked. The result is a boolean return code, 0 if the action was successful, 1 otherwise.

The pathname is used by the LINK function to customise:

- The names of the shared variables.
- The name of the PATH function.

Both sets of names also reflect the auxiliary processor in use, so no name clashes will result if multiple auxiliary processors are used as well as multiple paths per processor. The key function in the LINK process is the VSAMGEN function, the version listed uses features of APL2, but there is no reason why it could not be recoded using VS APL.

The function created by the VSAMGEN function is also listed, essentially it simply assigns values to the control and data shared variables and returns a two item nested vector (APL2 again!). The explicit result consists of the value from the control variable as the first item, with that from the data variable as the second item.

In VS APL, one of the shared variables would have to be referenced as a global variable since nested arrays are not available. As an alternative, a second niladic function could be generated to return the value of the other variable as its explicit result. The pathname used in the example functions to link to the path is "TEST", the function generated is called VSAMTEST, and is used for all subsequent file I/O operations. At the end of file processing, the path is unlinked by using the VSAMUNLINK function.

One result of using the PATH name to customise the shared variable names is that multiple shares with the same AP can easily be achieved. When using the VSAM AP, this allows multiple VSAM clusters to be open at the same time. Contrast this with the approach taken by the IBM supplied functions which always close a file before allowing another to be opened. Similar techniques, if applied to the QSAM and BDAM auxiliary processors would allow multiple file access using these protocols too.

4. Other Benefits

The command system implemented using the VSAMGEN function is identical to that for API23 itself – there is no processing overhead interpreting the commands used. Also, using APL2, the EACH operator can be used to execute many commands without writing looping code. Allowing, for example, many read or write operations to be undertaken without the associated overheads of looping.

User written operators could be used to read or write data from/to a file until a non-zero return code, such as end-of-file, is received (problems with WS FULL may result of course).

The ability to use APL2 operators arises because the same function is used for all file based operations, with only the argument values changing. Arguments can be applied repeatedly without having to reference different functions. In contrast, the approach taken in the VSAMDATA supplied workspace is that of “keywords”, different functions being used to achieve each VSAM operation. When using the keyword approach, there is little or no opportunity to make use of the functional enhancements provided by APL2 operators.

It is worth noting that even in VS APL, the command controlled function can result in more efficient, compact code. For example, it would be relatively simple to rewrite the generated function so that it could either use a vector or array command argument. In the case of an array, each row could be used in turn until the array is exhausted. The only looping code involved in this type of implementation is inside the generated function, rather than in the application code.

A further benefit of the command approach rather than the “keyword” function approach is the reduction in the number of objects (and the volume of code) that has to be copied into the workspace. The only permanent code in the workspace when using the example functions are the LINK, UNLINK and GEN functions. During processing, the path access functions are also created. All these functions are relatively short, so the volume of code involved is small.

The largest function listed is that giving the text associated with the different error codes that may arise. Since an application will typically contain its own error checking and messaging, it may not be necessary to have this function in the workspace at all.

5. Extending the concept

There is no need to restrict the use of `□ FX` to the avoidance of execute for variable names, function execution is also amenable to the technique.

Mainframe APL provides no in-built mechanism for translating input data from binary, packed or other S/370 formats into APL data formats. A frequent requirement when processing files is the translation of such data, functions are provided in one of the IBM supplied workspaces for this purpose. Before the translation is performed, the record must be separated into its component fields.

A function can easily be generated, which, given a series of field names, lengths and translation requirements will take an input record, extract the fields and translate the data into APL format. The translated data can either be returned as a vector representing the data, or in a series of global variables, one per field. The reverse process can be used for writing records back to the file in S/370 data formats.

The generation of two functions (one for input and one for output) given the layout of the record as input is a relatively trivial problem. The gains in performance by avoiding use of the execute function to perform translation and assignment into the explicit result (or the global variables representing the fields within the record) will be considerable. This is all the more important since it is likely that, as with any file processing, large volumes of data will be processed.

6. Conclusions

The system function `⎕FX` can be used to replace the use of the execute function in many situations. It acts as a form of "compiler" to hard-code the references to variables which would otherwise be referenced through execute.

Some overhead is incurred in the use of `⎕FX` as the function being fixed is verified and established in the workspace. It is important therefore that `⎕FX` is used in those situations where the gains from the avoidance of execute will outweigh the costs of using `⎕FX`.

This technique will be of most use in situations where the execute would be invoked many times. Typical of these are file processing applications where relatively large volumes of file I/O are involved.

Other situations where execute is used, such as the conditional execution of a line of code do not lend themselves to this technique. None the less execute is probably better avoided in these cases.

I-APL – A Proposal for a Public Domain Interpreter for Small Computers

by Anthony Camacho

Three meetings were held at APL 86 to discuss this project – on 9 July at 5.30, on 10 July at 5.45 and on 11 July over lunch. The main things that were agreed at the first meeting were:

Object: to promote the use of APL

Preamble At present anyone persuaded of the desirability of using APL cannot try it on a computer without significant expenditure. This barrier can be surmounted. The combined resources of the APL Associations with Vendor support will suffice to raise the £30,000 (\$45,000) or so needed.

With that barrier removed it will be worth mounting a sustained campaign (in particular in education) to take advantage of the new low threshold cost of trying APL. To do this effectively will require further finance, the amount of which has not yet been determined.

Main steps to provide a public domain APL interpreter which will run on school and home computers without enhancements (BBC micro, Apple II, Commodore 64, PCs and look-alikes etc.).

1 Raise sufficient financial backing:

- i) for the interpreter
- ii) to advertise it widely
- iii) document it beautifully
- iv) distribute it to all who want it

2 The interpreter is to be simplified ISO (conforming in all respects for features that are included). It will have to be written to save space at the expense of speed, and in a language which makes the job of porting it onto the various machines easy. Thus it will be too slow to compete with commercial APLs. Paul Chapman could begin to write the interpreter in October 1986 and estimates that it will take about five months to do it.

At the second meeting many people volunteered to help and a committee of five was elected to manage the project:

Ed Cherlin
Anthony Camacho
Howard Peelle
Norman Thomson
David Ziemann

Chairman – New World
Secretary and Joint Chairman – Old World
Education – New World
Education – Old World
Technical (Paul Chapman declared he would prefer not to be on the committee)

The interpreter is to run on the commonest home and education microcomputers such as those above and such other machines as are proposed to us as worth including. It is to include as much of ISO APL as Paul can manage in the space. The interpreter will have to be written to conserve space at the expense of speed, and for some machines may have to be supplied in EPROM to leave a usable workspace. Paul's target is to fit the code into 25K bytes. There may be a problem on some machines with the character set, so the interpreter will be written in such a way as to allow either simple porting using an ASCII symbol set, or, with more work, provision of the APL character set on selected machines.

The timescale is very tight. Paul could start work in October 1986 if we have sufficient money raised by then, and would expect to have a working interpreter with porting instructions by May 1987. It may even be possible to show it at Dallas!

The two Chairmen were put in charge of fund-raising. Plainly they have to raise about half of the required total by September if we are able to commit ourselves to support Paul for the period needed to do the work. The two chairmen were also to set up an organisation which would make the best use of the funds raised.

The Education Officers have a harder task to perform, and probably are harder pressed to do it in the time than the fundraisers. They are to plan approaches to the educational world in the old and new worlds which will get the new interpreter into use. It may be desirable to do this through the children direct as well as via the established institutions and the computer manufacturers. We should obviously try to get manufacturers to include APL in the box with every new computer. The major task will be to produce educational material so that teachers who can be persuaded to try APL don't have each to perform the daunting task of writing their own. The costs of each proposal need careful estimating to allow decisions to be made about which to carry out. Offers and suggestions welcome!

The committee feels sure that everyone who uses APL will want to see this project succeed. Please contribute as much as you can afford.

I Appeal to Members: International APL (I-APL)

You can't persuade people APL is wonderful without showing them. You can't show them because it won't run on their home or school computer. End this frustration! Here is an opportunity not to be missed. Now you can really do something to help the spread of APL – support I-APL !

The I-APL Project is in two parts:

- 1 Write a public domain 'International APL' and distribute it just for the cost of copying. The software will be free and unprotected – even the source code!
- 2 Promote it everywhere, particularly in education.

It will be based on and will conform to the ISO Standard (hence the name) and will be as complete as possible while running on many kinds of school and home microcomputer (BBC microcomputer, Apple, Atari, MSX, Commodore 64, Sinclair Spectrum 48K, PC and many others).

We intend to start work in October 1986 and to finish by May 1987. We need £30,000 (\$45,000) and we need it NOW. A committee is raising funds on both sides of the Atlantic. Ed Cherlin (editor of APL News) is Chairman for the new world and Anthony Camacho (secretary of the British APL Association) is Chairman for the old world.

Subscribe and you can also have the opportunity to test the first release of I-APL. The suggested subscription is one or two years subscription to your own APL group. How much do you care about APL? How much is it worth to you?

Send your subscriptions to your local group to forward to us or direct to Ed or Anthony. Make cheques payable to I-APL. Enclose the form below or a photocopy if you want the first release.

Yes: let me know as soon as the first release interpreter is available for my computer.

I enclose a donation of:

Name

Address

.

.

.

Postcode or zip

Make, model and memory capacity of computer

.

.

SEND TO

Ed Cherlin:

6611 Linville Drive, Weed, CA 96094, U S A

or Anthony Camacho:

2 Blenheim Rd, St Albans, AL1 4NR, U K

Ed Cherlin and Anthony Camacho, Chairmen

ANNOUNCING THE

dyalog APL

COPROCESSOR FOR THE IBM PC

The Dyalog APL coprocessor allows large scale second-generation APL applications to be developed and run on an IBM PC or plug-compatible computer.

The Dyalog APL coprocessor consists of:

- a 32-bit coprocessor board based on the NS 32000 chip set, hardware floating-point, up to 4Mb of on-board RAM, and 16Mb of virtual memory space.
- a complete implementation of UNIX* System V including C, f77, Fortran and Pascal.
- a complete implementation of Dyalog APL.
- software that integrates the Dyalog APL coprocessor subsystem into the PC-DOS environment.
- a Dyalog APL character generator and set of keycap stickers.

The Dyalog APL coprocessor is easily installed in an IBM PC or plug-compatible system with a standard BIOS and a hard disk. The board simply slides into a single slot in the PC chassis, and the software is installed via an interactive menu.

The Dyalog APL coprocessor provides both DOS and UNIX System V environments without compromise. Data can be shared between the two environments, and a simple keyboard command switches between operating systems. DOS commands and files can be accessed directly from Dyalog APL, and Dyalog APL can be run directly from DOS.

The Dyalog APL coprocessor supports very large APL objects and workspaces, to a maximum of nearly 16Mb; and floating-point performance is up to 3 times faster than an IBM PC AT.

Prices, including a multi-user Dyalog APL licence, start from around £3,000. A complete ready-to-run system with a fast 40 mb disk and tape back-up is also available for under £7,000.

For further details and ordering information, please contact **Peter Donnelly**
Dyadic Systems Ltd., Park House, The High Street, Alton,
Hampshire GU34 1EN, United Kingdom
Telephone: Alton (0420) 87024 (10 lines) Telex 858811

Zero Deficiency in Floating-Point Environment

by Dick Gray

One of the important qualities in a high-level language is that it should be divorced from the technicalities of computer jargon – the bits, bytes and binary syndrome that are such a joy to the computer professional and so frustrating to others. Everyone cannot be expected to thrill to the knowledge that asymptotic lines go on for ever getting closer and closer but never actually meeting, or that points can float.

As a devotee of APL, I believe that it is the best of the high-level languages, partly because it is not only a language in its own right but also a logical mathematical notation. It is very practical for people who are not noticeably philosophical, for people like me who used to thumb out the notes from my wages packet into little piles and put one pile into the tea-caddy for housekeeping, one under the mattress as savings for old age, and the remainder into my pocket for a night out at the pub, carefully moving the change from one receptacle to another. Now, with APL and my home-computer, I can calculate immediately that so-much for housekeeping and so-much for savings leaves me enough for 20 pints plus a flutter on the 2.30 at Doncaster. This simplicity is why APL appeals to me and why I have persuaded a number of clients, who are practical-minded business people, that APL is a "good thing" and a ray of hope in the tortuous and complex world of computing.

But due to an event which occurred recently with APL I am now in a state of complete shock.

I was asked to demonstrate APL to a prospective client and decided to illustrate it with a simple Ledger routine, a procedure I have adopted successfully many times before. The routine took each customer in turn, calculated a balance by subtracting payments from the amount due, and wrote a new record when the balance did not equal zero. The relevant statements looked something like this (written in Sinclair Keyword for the benefit of non-APL readers):-

```
.....
BALANCE is AMOUNTDUE -+on PAYMENTS  (the sum of payments)
GOTO NEXT if BALANCE eq(u)al(s) 0
WRITE (Balance Record)
.....
```

Although I had already partially tested the function, the first hint of trouble came when we entered about 40 records from the client's existing ledger and found that we had generated, from about 39 genuine zero balances, at least 15 new records which stated:-

```
Customer: BALANCE BT FWD £00.00
```

"Ha!" said the client's inevitable Mr. Knowall who was very anti-computers. "Hope we don't send them all threatening letters for that."

"No problem", I assured him. "Just a logic error. We can take a look and see what the computer thought the BALANCE was." I thought that maybe we had rounded up a half-penny out or some such.

But not a bit of it – we were out even less than that. Imagine my consternation when I displayed BALANCE and saw the value as:

$-7.105427358E-15$

The statement BALANCE = 0 produced the truth-vector response of 0, or untrue!

"What does that mean?" asked Mr. Knowall. "Have we won the pools?"

I will not flog the point, but I had thought that the purpose of Comparison Tolerance was to iron out silly, unrealistic situations like this. I know that I can avoid it happening by using integer pence instead of pounds to two decimal places, by formatting, by rounding, or in several other different ways. I know that zero is a special case, that the smaller the number the lesser the relevance of Comparison Tolerance. I know all that. But is it desirable that a user should have to cope with these hardware qualifications?

If we do a simple calculation like:

$X + 46.23 - (40.20 + 6.03)$

and then display X and find:

$-7.105427358E-15$

What user-friendly, simple, mathematical notation is that, I ask you? Words fail me. Do I have to explain to my trusting and valued client that there are times, according to our so-called advanced computers, when nought does not equal zero?

Next day

I am in anguish. Oh! my beloved APL, what hast thou done to me? I have a vision of the instructor on the first BBC lesson in APL saying,

"Now the first thing you have to get into your heads is that, in this advanced logical language, zero is simply represented as:

$-7.105427358E-15$

got it?"

By the waters of Severn & Trent I lay down and wept!

Some days later

Now I should be feeling better. I have managed to speak about the problem to my bewildered wife, my astonished G.P., my wily psycho-analyst and, today, to a competent APL therapist. But the latter assures me that there is nothing to get upset about, that the machine is correct, and that the response is in accordance with the new ISO standard; there is, in fact, no problem.

I am distraught. Little symbols are darting out of nested arrays and feeding on tildes. My world has collapsed. The Romans didn't know about zero, but the Arabs, thank goodness, invented it. Now it seems that computers and APL have destroyed it. Is this a nightmare? Am I going asymptotic? Cannot true zero exist in floating-point?

Matron's bulletin, Dick Gray Associates' Nursing Home

Patient is in intensive care. If the BSI APL standards committee feels it can do anything to help, please contact me urgently. Otherwise it may be:

GOTO END

(Editor: I feel compelled to offer a sympathetic voice, if only one that cannot offer a therapeutic solution. My own particular frustration is to find that the answer to the mathematically equivalent test:

$1 = 1 + \text{BALANCE}$

is (correctly) 1 or true. No doubt the definition is part of a devious plan by hardware suppliers to ensure that we need additional CPU resource to carry out the extra addition every time we want to compare with zero.)

Public Domain Software Library

Software contributions

Any item of use or interest to APL programmers is welcomed in the library. For a six month experimental period the operation of the library will be kept as simple as possible. After that extensions may be made.

The initial scheme is:

- 1 The library item is a PC compatible disk (9 sector 360K).
- 2 Each software contribution should be accompanied by a document in the format below which:
 - a asserts the software is yours to give and authorises the British APL Association (BAA) to give it away on your behalf.
 - b indemnifies the BAA and its members against claims from third parties for breach of copyright, libel or breach of the law.
 - c contains a description of the disk and a one line description of each file on the disk (for the catalogue).
- 3 The BAA make no claim or warranty about the content of the library.

The material must be supplied on a DOS-format disk which can be copied on a PC compatible machine. The contents of the files may be in any format for any variety of APL. Documentation should, if possible, be provided on the disk.

Software contribution form

To: Mel Chapman
North Staffs Polytechnic
Blackheath Lane
Stafford ST18 0AD

I confirm that the enclosed disk labelled contains only material which I am entitled to give away and I authorise the British APL Association to give it away on my behalf and I hereby indemnify the British APL Association and its members against any loss they may sustain arising from such authorised gifts of material copied from this disk such as claims from third parties for breach of copyright slander or libel or the consequences of any breach of the law.

Attached is an abstract of the purpose and use of the software, a list of all the files on the disk, and my full name and address.

Signed and dated

Public Domain Software Library

APL 90 Interpreter for Macintosh

Ref: 86001

Machine: Macintosh.

Donor: J. J. Girardot

Documentation: On line & 2 pages. Size: 2 Disks.

APL 90 is a complete APL system for Macintosh, and includes some of IBM's APL2 facilities (strand notation, nested arrays, user defined operators . . .).

Life

Ref: 86002

Machine: IBM PC.

Donor: P. Chapman

Documentation: On line. Size: 1 Disk, 8 Files, 40K.

A version of Conway's game of life for the IBM PC. Hundreds of times faster than previous version. 65532 by 65532 universe. Full editing and library functions. Some sample patterns. See tutorial volume of conference proceedings for background. (RUNCODE Life by Paul Chapman).

Ed: Great fun.

Fluctuation analysis of multichannel spectra

Ref: 86003

Machine: IBM PC, APL: APL*PLUS/PC.

Donor: Claude Bastian

Documentation: On Line. Size: 1 Disk, 1 File, 8K.

Workspace WALSH - Analysis/condensation of counting fluctuations in multichannel spectra or time series. Further description in APL 86 proceedings pages 50 to 58.

Free software from APL385

Ref: 86004

Machine: IBM PC, APL: APL*PLUS/PC.

Donor: Adrian Smith

Documentation: £50 or \$75. Size: 1 Disk, 13 Files, 270K.

- "Is it a Plane: is it a Bird?" (Expert System Shell)
- Digital Cuckoo Clock (Honest)
- File Utilities (Function Swopping and DOS files to and from APL)
- PASCAL-Written DOS file lister.

Documentation is available from: APL385, Brook House, Gilling East, York, England

Ed: Lots of useful functions.

Piano

Ref: 86005

Machine: IBM PC, APL: APL*PLUS/PC.

Donor: G. Langlet

Documentation: On Line. Size: 1 Disk, 1 File, 3K.

Play the PC like a piano.

Ed: OK for nursery rhymes.

Ref: 86007

Frame

Machine: IBM PC, APL: Sharp APL/PC.
Documentation: 11 Pages. Size: 1 Disk, 2 Files, 67K.

Donor: Robert Pullman

Editing multidimensional numeric arrays.

Ref: 86008

Get/put files

Machine: IBM PC, APL: APL*PLUS/PC.
Documentation: On Line. Size: 1 Disk, 2 Files, 16K.

Donor: M Kent

Functions for manipulating data in APL*PLUS files by "item name", rather than by component number. The associated file structure is called a "Get/Put" file, whence the workspace name "GP".

Ref: 86009

Binary Combinations

Machine: IBM PC, APL: APL*PLUS/PC.
Documentation: On Line. Size: 1 Disk, 1 File, 2K.

Donor: Orway Pardee

Very fast method for generating the binary image of combinations. Useful and fast where the number of combinations is large. Can easily be adapted to generate the combinations in subsets. This method has only one explicit APL loop.

Ed: Needs assembler for real speed.

Ref: 86010

Workspace Lister

Machine: IBM PC, APL: APL*PLUS/PC.
Documentation: On Line. Size: 1 Disk, 1 File, 14K.

Donor: Roger Harper

Pretty-printer for APL*PLUS workspaces, both functions and variables, with index and paginations. Assumes an Epson LQ1500 printer, but can be converted to other printers.

Ref: 86011

TEXT

Machine: IBM PC, APL: APL*PLUS/PC.
Documentation: On Line. Size: 1 Disk, 7 Files, 330K.

Donor: Ian Feldberg

Typesetting for the IBM-PC using APL*PLUS. Documented in APL 86 conference proceedings.

Ed: PC users will need GRAPHPAK for the PC, ref: 86012.

Ref: 86012

GRAPHPAK for the IBM PC

Machine: IBM PC, APL: APL*PLUS/PC.

Donor: Ian Feldberg

Documentation: On Line. Size: 1 Disk, 3 Files, 31K.

Companion to the TEXT software to draw the characters on a PC graphics board or on an HP7550 plotter.

Ref: 86013

Pianoman

Machine: IBM PC.

Donor: Stuart Yarus

Documentation: On Line. Size: 1 Disk, 14 Files, 340K.

Play your PC like a piano! Also, 'capture' music into .COM file for later replay. Much music included. PIANOMAN is a User-Supported product. You have the opportunity to try the program as see if it suits your needs. If you find that you like and use it, please send the requested contribution of \$25 to the author:

Neil Rubenking 300 Page Street San Francisco CA 94102 USA.

Ed: Sophisticated system.

Ref: 86014

Functions for Linear Algebra

Machine: IBM PC, APL: APL*PLUS/PC.

Donor: Garry Helzer

Documentation: APL 86 paper. Size: 1 Disk, 1 File.

Ref: 86015

DOS Utilities

Machine: IBM PC.

Donor: Timo Kiravi

Documentation: On Line. Size: 1 Disk, 4 Files, 33K.

CWEEP.EXE – Mass copy files, erase files etc.

DOSEDIT.EXE – Command line editor.

BROWSE.COM – View text files.

D.COM – Formatted disk directory.

Ed: Nice utilities.

Ref: 86016

Reverse Polish APL Interpreter Model

Machine: IBM PC, APL: Most.

Donor: Richard Naugle

Documentation: APL 86 paper. Size: 2 Disks, 30 Files, 600K.

The functions were described in the APL 86 paper: APL Compilation and Interpretation by translating to F83VEC. Versions for IBM APL, PortaAPL, Pocket APL, APL*PLUS interpreters, and a file in workspace interchange format for Sharp APL. Demonstration version of PortaAPL included.

Public Domain Software Library

Keyword Index

<i>Ref</i>		<i>Keyword</i>
86014	Functions for Linear	Algebra
86003	Fluctuation	analysis of multichannel spectra
86016	Reverse Polish	APL Interpreter Model
86001		APL 90 Interpreter for Macintosh
86004	Free software from	APL385
86009		Binary Combinations
86009	Binary	Combinations
86015		DOS Utilities
86008	Get/put	files
86003		Fluctuation analysis of multichannel spectra
86007		Frame
86004		Free software from APL385
86014		Functions for Linear Algebra
86008		Get/put files
86012		GRAPHPAK for the IBM PC
86012	GRAPHPAK for the	IBM PC
86016	Reverse Polish APL	Interpreter Model
86001	APL 90	Interpreter for Macintosh
86002		Life
86014	Functions for	Linear Algebra
86010	Workspace	Lister
86001	APL 90 Interpreter for	Macintosh
86016	Reverse Polish APL Interpreter	Model
86003	Fluctuation analysis of	multichannel spectra
86012	GRAPHPAK for the IBM	PC
86005		Piano
86013		Pianoman
86016	Reverse	Polish APL Interpreter Model
86016		Reverse Polish APL Interpreter Model
86004	Free	software from APL385
86003	Fluctuation analysis of multichannel	spectra
86011		TEXT
86015	DOS	Utilities
86010		Workspace Lister
86001	APL	90 Interpreter for Macintosh

Index to Advertisers

Ampere	76
APL-385	26
APL People	66
Camacho (Public Domain APL)	92
Cocking & Drury	13, 50, 102
Dyadic Systems Ltd	14, 130
HMW Programming Consultants	22
IBM PC	64
IBM (Sweden)	114
Inner Product Ltd	2
Mercia Software	42
MetaTechnics	6
MicroAPL	16, 41
Mine of Information (APL Booklist)	36
Vector - Back Numbers	4

All queries regarding advertising in VECTOR should be made to the advertising editor, Steve Lyus, at the following address:

Metapraxis Ltd
Hanover House
Coombe Road
Kingston, KT2 7AH

Tel: 01-541 1696

Advertisements should be submitted in typeset, camera-ready A5 portrait format with a 20mm blank border. Illustrations should be black-and-white photographs or line drawings. Rates are £250 per page. A6 and A7 sizes (at £150 and £75 respectively) are available, subject to layout constraints.

BRITISH APL ASSOCIATION**Membership Application Form**

Please read the membership information in the inside front cover of VECTOR before completing this form. Use photocopies of this form for multiple applications. The membership year runs from 1st May – 30th April.

Name: _____

Department: _____

Organisation: _____

Address Line 1: _____

Address Line 2: _____

Address Line 3: _____

Address Line 4: _____

Post or zip code: _____

Country: _____

Telephone Number: _____

Membership category applied for (tick one):	86/87	
Non-voting student membership (UK only)	£ 5	
UK private membership	£ 10	
Overseas private membership	£ 18	\$ 27
Airmail supplement (not needed for Europe)	£ 8	\$ 12
Corporate membership	£ 85	
Corporate membership Overseas	£140	\$210
Sustaining membership	£360	

For student applicants:

Name of course: _____

Name and title of supervisor: _____

Signature of supervisor: _____

PAYMENT

Payment should be enclosed with membership applications in the form of a UK sterling cheque or postal order made payable to "The British APL Association". Corporate or sustaining member applicants should contact the Treasurer in advance if an invoice is required. Please enclose a stamped addressed envelope if you require a receipt.

Send the completed form to the Treasurer at this address:

Mel Chapman, 12 Garden Street, Stafford ST17 4BT, UK.

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society and a member of EuroAPL, an organisation supported by the Commission of the European Communities. It is administered by a Committee of eight officers who are elected by the vote of Association members at the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

1986/87 Committee

Chairman:	Dick Bowman 01-634 7639	CEGB, 85 Park Street, London, SE1.
Secretary	Anthony Camacho 0727-60130	2 Blenheim Road, St. Albans, Herts., AL1 4NR.
Treasurer	Mei Chapman 0785-53511	12 Garden Street, Stafford, ST17 4BT.
Activities	Roy Tallis 01-405 7841 (ext 5482)	Kodak Ltd., P.O. Box 114, 190 High Holborn, London, WC1V 7EA.
Journal Editor	David Preedy 01-541 1696	Metapraxis Ltd., Hanover House, Coombe Road, Kingston, KT2 7AH.
Education	Norman Thomson 0962-54433	IBM Mail Point 188, Hursley Park, Winchester, Hants., SO21 2JN.
Publicity	Gerard Paul-Clark 01-622 0395	MicroAPL Ltd., Unit 1F, Nine Elms Industrial Estate, 87 Kirtling Street, London, SW8 5BP.
Technical	Dave Ziemann 01-493 6172	Cocking & Drury Ltd., 16 Berkeley Street, London, W1X 5AE.
Recruitment	Christine McCree 01-560 5151 (ext 3414)	Beecham UKCS, Beecham House L/2, Great West Road, Brentford, TW8 9BD.
Projects	David Eastwood 01-622-0395	MicroAPL Ltd., Unit 1F, Nine Elms Industrial Estate, 87 Kirtling Street, London, SW8 5BP.

Journal Working Group

Jonathan Barman	0374-588835
Anthony Camacho	0727(56 from London)-60130
Steve Lyus	01-541 1696
David Preedy	01-541 1696
Adrian Smith	0904-53071
David Ziemann	01-493 6172

Activities Working Group

Peter Donnelly	0420-87024
Steve Margolis	01-670 7959
Tim Perry	04626-77375
Roy Tallis	01-405 7841

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society and a member of EuroAPL, an organisation supported by the Commission of the European Communities. APL stands for "A Programming Language" — an interactive computer programming language noted for its elegance, conciseness and fast development speed. It is supported on many timesharing bureaux and on most mainframe, mini and micro computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

APL People

17 Barton Street,
Bath, Avon.
Tel. 0225-62601

APL Software Technology

14 Rosewood Avenue,
Alveston, Bristol, BS12 2PP
Tel. 0454-415767

Cocking & Drury Ltd.

16 Berkeley Street, London, W1X 6AE
Tel. 01-493 6172

Dyadic Systems Ltd.

Park House, The High Street, Alton, Hants.
Tel. 0420 67024

H.M.W. Programming Consultants Ltd.

142 Feltham Hill Road,
Ashford, Middlesex, TW15 1HN
Tel. 07842-41232

Inter-Product Ltd.

Eagle House, 78 Clapham Common Southside,
London SW4 9DG.
Tel. 01-673 6354

Marcia Software Ltd.

Aston Science Park, Lovell Lane,
Birmingham, B7 4BJ.
Tel. 021-659 5095

Metatechnics Systems Ltd.

Unit 216, 62 Triton Road,
London, SE24 6DE
Tel. 01-670 7959

MicroAPL Ltd.

Unit 1F, Nine Elms Industrial Estate,
87 Kirling Street, London, SW8 5BP.
Tel. 01-622 0395

I.P. Sharp Associates

10 Dean Farrar Street,
London, SW1H 0DX
Tel. 01-222 7033



The British Computer Society, 19 Mansfield Street, London W1M 0BD.